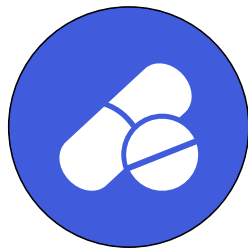


Residential home Medication Tracker



PILL POWER

Zaccaria Marin

Contents

RESIDENTIAL HOME MEDICATION TRACKER.....	1
ANALYSIS SECTION.....	3
<i>Introduction.....</i>	3
<i>Defining the Problem.....</i>	3
<i>Current Implementations.....</i>	5
<i>Intended Recipients.....</i>	6
<i>End User Interview(s).....</i>	6
<i>Moddelling the solution.....</i>	8
<i>Research.....</i>	10
<i>Final Numbered Objectives.....</i>	14
DOCUMENTED DESIGN.....	16
<i>Contents.....</i>	16
<i>File Structure Overview.....</i>	17
<i>Website Overview.....</i>	18
<i>The Data Layer.....</i>	26
<i>Backend Layer.....</i>	36
<i>Frontend Layer.....</i>	64
IMPLEMENTATION.....	72
<i>Contents.....</i>	72
<i>MedTracker/.....</i>	73
<i>Templates/.....</i>	120
TESTING.....	173
EVALUATION.....	176
<i>Fulfilment of Objectives:.....</i>	176
<i>User feedback:.....</i>	178
<i>Overall evaluation:.....</i>	179

Analysis Section

Introduction

Residential Homes can be registered to deal with all sorts of responsibilities for the people in their care. Amongst these responsibilities is the management of an individual's medication, which requires precision, reliability and efficiency. Traditional Methods, such as paper-based records or use of spreadsheets can not only be time-consuming but result in human error. In the market there are many web-platforms that allow end users, that being residential home staff, to track residents digitally across a care home, and it is this type of product that I am interested in making for my project. More specifically, I want to create a tool for managing the medication of care home residents. The focus will be on making the responsibility on carers lesser, assisting them in handling day-to-day medication administration.

Defining the Problem

To make a tool that is accessible, I intend to create a website that will allow for both management and care staff to easily use. By using a website, it will enable for a wide variety of devices and operating systems to access the application, especially useful when a Care home uses both desktop and mobile hardware. The website will allow for managers to register their organisation and manage residents medication schedules. Carers will then be able to efficiently track and administer medications via a user-friendly interface. The key features of a robust solution will be secure user authentication, role-specific access and real-time records, ensuring accurate and consistent data.

Care homes

Researching into care home's¹, I became more familiar with the demands of the project. Care homes provide essential support to individuals who require assistance with daily living tasks in a tailored environment. They do vary in purpose and services, ranging from residential care homes (focused on personal care) to nursing homes (more focused on medical care). Additional care homes include dementia and dual registered homes, which combine the services of nursing and residential. While a nursing home may focus more on the medical side than a residential one, they both shoulder the responsibility of dispensing medication to residents. This means that my application should aim to service a wide variety of medication-based needs.

¹ https://en.wikipedia.org/wiki/Nursing_home



Figure1: Example of a registered residential home²

The hierarchy of Care home staff, who are involved in the welfare of residents, typically consists of: The Managers, who oversee operations and are responsible for compliance; Senior Carers, who organise care plans and who are registered to dispense and apply more serious medications and Carers/Nurses who handle more daily support tasks and administering medication. When designing my user structure, my project must reflect this hierarchy, taking into consideration who is allowed to see and dispense certain residents medication.³

From my research, I concluded that the workflow within a care home is typically defined as:



I discussed this model with my end-user, and it resulted in this workflow definition:

1. The pharmacist creates a medication record on the system, defining the medication name, dosage and period of medication. Due to the scope of my project, as a student and GDPR I would be unable to directly work with pharmacy systems, so this would be manual input in my project.
2. Medication is then scheduled based off the provided information. The schedule includes the dosage, name, administration route, frequency and time(s) in the day.
3. During the allotted times, the medication is then successfully administered or dispensed to a resident.
4. The medication is then marked as given, and a physical or digital record of this is stored.
5. Physical medication stock levels are recorded against the records.

² Identifying source removed from public edition.

³

Current Implementations



Figure 2: Atlas eMAR, an electronic medicines management system

Before starting my project, I researched if any products similar to mine exist already. What this would establish is a baseline for features to implement and also made sure that my project had appeal to end-users. For nursing homes, there are many existing applications that assist with recording the multiple variables within a care home environment.

In the case of medication tracking, I came across many. Nourishcare was one popular platform I found. A key feature of their platform was accessible on PC and mobile through a downloadable app. Due to my unfamiliarity with dedicated mobile software development, I didn't feel their implemented solution was most fitting for my project.

However, another team developing a medication tracker was Person-Centred Software and their application Atlas eMAR. I found it had significant overlap with my idea of a final product. It was aimed at care home staff, to be used by both care staff and management, and enabled the tracking of medication administration for residents. Their application was primarily accessible online through a web interface using a client-server cloud-based model.

After researching current industry solutions, the approach I initially modelled was a client-server web-app. I concluded that this seemed to be a feasible medium for providing a carehome medication tracking service, as shown through the current industry implementations.

Intended Recipients

The intended recipients of the project are going to be care home staff, the project will aim to manage the entire medication workflow, from staff management to resident scheduling. The project will encompass both services for management users and nursing staff. For the development of the project, I will have specific end user and intended recipient for development. This end user is an experienced care-home manager of a dual-registered home, meaning it is authorised in both nursing and residential care.

End User Interview(s)

Over the course of the project, I held multiple interviews and communicated ideas to my end user. I ran an initial interview, asking a few questions to narrow down project scope and objectives:

What are the biggest challenges you face in managing medication for residents in your care home?

When dealing with a care home of our size, the biggest challenge is ensuring accuracy. With multiple residents, each with different medications and schedules, it can be easy for staff to make errors. Another issue is documenting everything correctly during busy shifts, which affects the quality of auditing.

How do you currently track medication schedules and administration, and what limitations do you find with this solution?

We currently use a digital tracking system, accessible on both desktops for management and Android devices for carers. It's generally reliable, and we prefer it over paper-based records, but it does have limitations. The main problem is the user interface: it's not intuitive, especially for staff who aren't confident with tech.

How important is it to have different levels of access for managers and carers in such a system?

It's extremely important, mainly for data protection reasons. Under GDPR, we're required to make sure only the right people have access to sensitive data. For example, we're a dual-registered care home, meaning we operate both residential and nursing care. We'd expect senior nursing staff to only see data relevant to residential residents, and the same for the nursing team. Beyond that, carers shouldn't be able to edit or view critical information, such as resident profiles or medication inventory, unless they've been given permission to do so.

Is it important for you that the system generates reports or summaries?

Yes, that's a very important feature. The most critical report we need is the MAR chart: the Medical Administration Record. This tracks every medication given to a resident: it should include the date and time, the dosage, the name of the medication and who administered it. MAR charts are required both for our own internal record keeping and for when we're audited or inspected. We also use them for agency or temporary staff, who may not have access to our full digital system. Being able to generate printable or exportable MARs is a must.

What kind of information do you expect to be recorded about each resident in the system?

Each resident's profile should include basic details like their full name, date of birth and room number. More importantly, it should hold their medical information: current medications, known allergies/ conditions and dietary needs. This would provide myself and authorised users an overview of each resident.

What are your expectations for the usability of such a system? For example, how important is it for the software to be simple for staff who aren't tech-savvy?

Usability is crucial. Not all our staff are confident with digital systems, If a system is too complicated or cluttered, they'll avoid using it or make mistakes. I'd expect large, clearly labelled buttons and minimal steps to complete everyday workflows/routines.

Conclusion:

From this interview, several priorities for the project were identified. The user focuses on medication record accuracy, staff usability and data protection (GDPR compliance) as essential. Therefore, audit logs and MAR charts must be detailed and easy to generate, Resident profiles and medication scheduling straightforward but comprehensive and role-based access and user authentication are not optional but legal necessities for such a program.

Based off the first end-user interview, I was able to compile an initial set of features that my end-user would expect of the final product. These are defined as followed:

1. Secure User access – This includes measures such as password authentication, time-based access and encryption.
2. Resident/Carer Groups – Allows the organisation of residents into specific groups (e.g. by floor or care type) and assigns carers to their respective groups
3. Resident profiles – Each resident has a detailed profile including current prescriptions, allergies, conditions and room number
4. User permissions and roles – Differentiate between the two individual workflows of users, managers and carers, and limit access and permissions of each respectively.
5. Medication scheduling – A modular system that enables a daily schedule for each resident, ensures that carers dispense the correct dosage at the correct time.
6. Alerts/Notifications – Sends alerts/notifications for missed or upcoming doses.
7. Medication log/record - records every instance of medication administration, including date, time, dosage, and the staff member responsible. Ensures compliance with medical regulations.
8. Medication Inventory Management - Tracks stock levels of medicines, alerts user when supply is low and updates automatically when medicines are dispensed.

I had my end user evaluate these features in a questionnaire, ranking them in priority:

Feature	Importance Ranking
Residential Profiles	1
User permissions and roles	2
Medication Scheduling	3
Residential/Carer groups	4
Medication inventory management	5
Secure User Access	6
Medication log/record	7
Alerts/Notifications	8

Modelling the solution

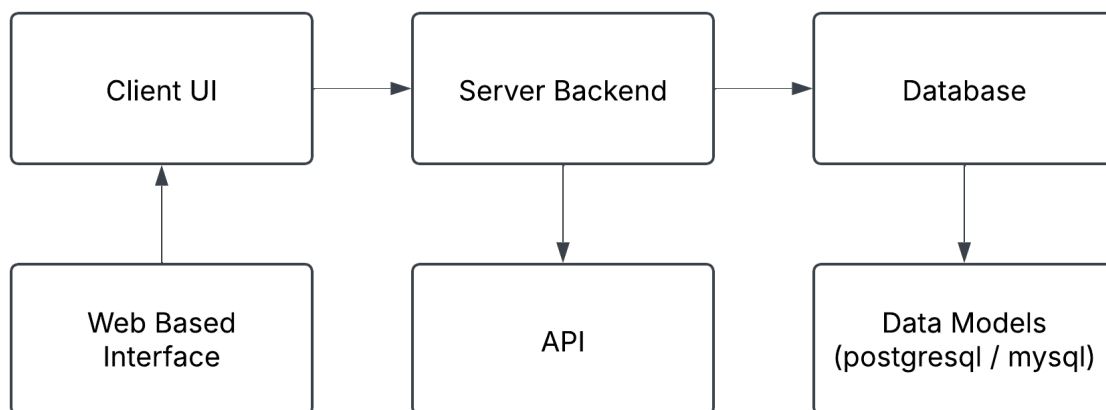
Diagrams are made using Balsamiq Wireframes⁴ and Lucidcharts⁵

For a solution for a carehome wide medication administration tracking, a web-based client/server approach felt most appropriate.

The diagram below serves as an outline, at the highest level, for the client/server relationship my project will have.

Users will interact through a web based interface, most likely HTML, and POST and GET requests will be managed by the backend, which will prompt requests to the database.

Model of the client-server architecture. Initial High-level overview.



Initial data model:

The demands of this project, which focuses heavily on data transactions, means modelling the database was a priority.

Modelling what the database would require and the tables it would be using, below is pseudocode for a prototype database:

- ⇒ CREATE TABLE carehomes (id, name, location, manager)
- ⇒ CREATE TABLE users (id, username, password_hash, user_type, carehome_id)
- ⇒ CREATE TABLE residents (id, name, room_number, carehome_id)
- ⇒ CREATE TABLE medications (id, name, carehome_id)
- ⇒ CREATE TABLE medicationitems (id, medication_id, dosage, form, stock, carehome_id)
- ⇒ CREATE TABLE schedules (id, resident_id, inventory_item_id, frequency, route)
- ⇒ CREATE TABLE individual_schedule_time (id, schedule_id, time)
- ⇒ CREATE TABLE medication_log (id, schedule_id, time_id, given_by, timestamp, notes)

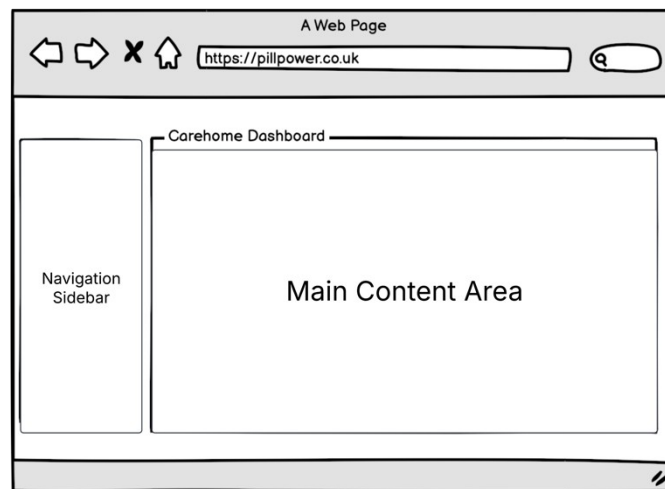
Carehomes will be assigned a manager User as a foreign key. Residents will then be created and assigned to a carehome. Each medication will have separate items, which record the unique dosages that medication take.

⁴ <https://balsamiq.com/wireframes/>

⁵ <https://lucid.app/>

UI mockup

Users will interact through the program via a web interface. The design will centre around a



navigation sidebar, displaying respective dashboards in a main content area. This first ui design is built to utilise a multiple view system, having different templates for each feature. For example, these could be different dashboards, for managing residents, inventory or staff.

Research

Following my initial research into the problem, and gathering the requirements expected of my end-user, I began analysis into how I would go about creating a finalised solution.

Website

When developing a web application, selecting the most appropriate backend language is crucial. During my research⁶, I discovered that a variety of languages may be suitable for this task, such as Java, Node.js and PHP, however, Python emerged as the ideal choice. This is due to my familiarity with the language, finding it the most comfortable for me to develop a large-scale project in, and also its versatility in web development. Python excels due to its readability and maintainability, as well as extensive frameworks and libraries for handling data manipulation and back-end logic, aligning perfectly with the needs of my project. Following my decision to develop the backend in python, I started research into Python's wide array of web back-end frameworks. I evaluated several web frameworks⁷, and my final consideration was the three most popular libraries, Flask, Django and Fast API. Below is a comparison table summarising the key differences I found:

Framework	Strength	Limitation
Django	Good documentation, robust security features, large library of extensions, good file structure for large projects	Performance heavy web-framework, steep learning curve
Flask	Lightweight, flexible, easy to learn, well documented	Larger projects require additional setup
FastAPI	Asynchronous support, high performance framework, lightweight	Fewer built-in-tools, less mature and so has less guides.

After deliberation, I chose Django for the project. While I was unfamiliar with developing in the Django environment, I chose it because of its scalability with larger projects and its extensive library of extensions and built-in tools, which allowed me to focus on functionality and with its focus on security features, It would help me follow best practices when it comes to managing key data like personal medication records.

Databases

For the project, databases would be fundamental. The workflow of a care home involves constant transactions of data, be it: storage, retrieval and management of sensitive information such as medication schedules and medication logs.

The choice of database had to have a balance of reliability, framework integration and ease of use. Python supports a wide variety of different databases⁸, as does Django with its own

⁶ <https://roadmap.sh/backend/languages>

⁷ <https://www.netsolutions.com/insights/top-10-python-frameworks-for-web-development-in-2019>

⁸ <https://builtin.com/data-science/python-database>

ORM language, including PostgreSQL, MySQL and SQLite. As they are all relational databases, they would suit my demands for a query-based database. I compiled my research into a table, evaluating the three most popular database⁹ types for Python:

Database	Strength	Limitation	Best use case
PostgreSQL	Advanced features (including JSON support and full text search), ACID compliant	Higher setup complexity, more resource intensive	Large-Scale applications, complex search queries, high reliability
SQLite	Lightweight on resources, easy to setup, highly compatible with Python.	Limited in scalability, single-file storage.	Small projects, development and testing
MySQL	Fast read operations, wide community support, easy setup	Less robust ACID compliance, fewer features in comparison to PostgreSQL	Medium-scale projects, web applications

For this project, PostgreSQL came out as clearly the best option. A medication tracker involves critical personal data, high levels of concurrency and data integrity is crucial. PostgreSQL is ACID compliant, and so transactions involving, for example, storing medication logs will be fully consistent. PostgreSQL is also a good choice for long-term scalability if the project were to go on into the future.

APIs

Medication and drugs within the healthcare industry are constantly being researched, therefore, the most up-to-date information is changing and evolving. Therefore, I felt it was most appropriate to use an API for the project to provide any of the medication data I would be using.

The benefit of using an API means that my data set would never be outdated. When new medications are released, or old medication made obsolete, the data provided to the end user can reflect that.

Some API's are made accessible with authentication methods, as such they require access to an API key, where as some do not require any authentication method. For my project, this limited the pool of API's I could access as a student, however after deep research there were many to consider.

For my project, I researched REST (representational state transfer) APIs, which use the HTTP protocol. I narrowed down the most suitable APIs to my project that I could find, the final shortlist was:

API	Link to API provider
NHS	https://www.nhs.uk/medicines/

⁹ <https://codedamn.com/news/python/3-best-databases-for-python>

NICE syndication services	https://api.nice.org.uk/
DrugBank	https://www.drugbank.com/

After research, I concluded that each Api would all suit my project needs. However, Drug bank seemed to offer the most complete service. Their API included a lot of medication data and was designed to be easy to implement within projects, with very detailed documentation and modular extensions.

I contacted DrugBank requesting for an API key for a trial account, but they recently had closed sign ups to the service.

Hi Zaccaria,

Thank you for reaching out! The API is a DrugBank commercial product and the pricing for the API is customized to your needs and use case, and is usually between 5 to 6 figures. If you would like more information about the API, I can connect you with one of our sales representatives.

Figure 3: Drugbank response to API access request

After this response I ruled out commercial APIs from my project, Instead focusing on free to access APIs. Both NHS and NICE offered their API for free usage in the United Kingdom, making them a suitable option.

Ultimately, it was the NHS web-content API I went with. The NHS are the main provider of healthcare in the United Kingdom, with their service I can be assured that data would be of a high standard, up-to-date and the API would have little downtime and be always accessible.

GDPR

Following the discussion with the end user, GDPR¹⁰ emerged as a key talking point. This prompted research into what my project would require abiding by GDPR specifications

Data covered by data protection is:

- Data relating to people
- Data that can determine or predict critical 'things'

Uk legislation mentions Health data as characteristic of critical data that must be protected.

To abide by GDPR practices, its important to:

- Keep it secure
- Dispose of it when no longer needed
- Seek consent
- Disclose to subject

Processing is legal with either Consent, Contract or requirement by law, and data subjects should be able to withdraw at any time.

After consulting my end user, they shared that contractually carehomes are provided consent to manage their residents data, which meant I had to focus on securing the data and making disposal of data.

Keyways to 'Protect' data involve:

- Encryption of sensitive data
- Maximise security for those who can access data

¹⁰ <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/>

- Limited access to data to specific users
- Limit time frames where data can be accessed
- Anonymisation of data links between people and data
- Pseudonymisations, do not hold data directly against a name

Final Numbered Objectives

1. User Account creation
 - 1.1. Provide a user registration module to create distinct user accounts
 - 1.1.1. Present a manager registration form, allowing the creation with name, email and password
 - 1.1.2. Present a Carer registration form, allowing account creation with name, email password and a existing carehome code
 - 1.1.3. Process, clean and store user data in the database
 - 1.1.3.1. Store User passwords in an encrypted format
 - 1.1.4. Create a Manager/Carer entity from stored Users in the database
 - 1.1.4.1. Present a carehome creation form process for Managers after registration
 - 1.1.4.1.1. Require managers to setup carehome after registration
2. Role-Based access controls
 - 2.1. Implement role-based access to views, which are dependant on user permissions or role
 - 2.1.1. Restrict carehome views to authenticated users associated with the correct carehome
 - 2.1.2. implitment permission-based access for Carers,
 - 2.1.2.1. limit the views and functions they can access based on permissions.
3. User session management
 - 3.1. Enable users to Login using an authentication form
 - 3.1.1. Require both an email and password for logging in
 - 3.2. Redirect authenticated users to the appropriate dashboard for their role
 - 3.3. Redirect unauthorised access back to the login page
 - 3.3.1. Display error messages displaying appropriate contextual message
4. Resident Management
 - 4.1. Enable Managers to add residents to the carehome
 - 4.1.1. Display a form, requiring valid fields like name and d.o.b
 - 4.1.2. Create a record of the resident in the database
 - 4.2. Enable creation of carehome groups
 - 4.2.1. Display a name and colour field
 - 4.2.2. Allow residents to be assigned to groups associated with their carehome
5. Medication Schedule management
 - 5.1. Manage Medication Schedules and the administration
 - 5.1.1. Provide a dashboard for Carers to view upcoming medication schedules
 - 5.1.1.1. Only display the residents for residents of an associated group
 - 5.1.2. Allow authorised carers to mark a medication as administered
 - 5.1.2.1. Log the time of medication administration
 - 5.1.2.2. Provde a optional field for care notes
6. Medication Inventory Tracking
 - 6.1. Track the carehome inventory
 - 6.1.1. Allow managers to add medication to the carehome inventory
 - 6.1.1.1. Provide a form to define a variant of a medication
 - 6.1.1.2. Store the variant of the mediation in the database
 - 6.1.1.3. Notify users when medication is low on stock/ requires reorder

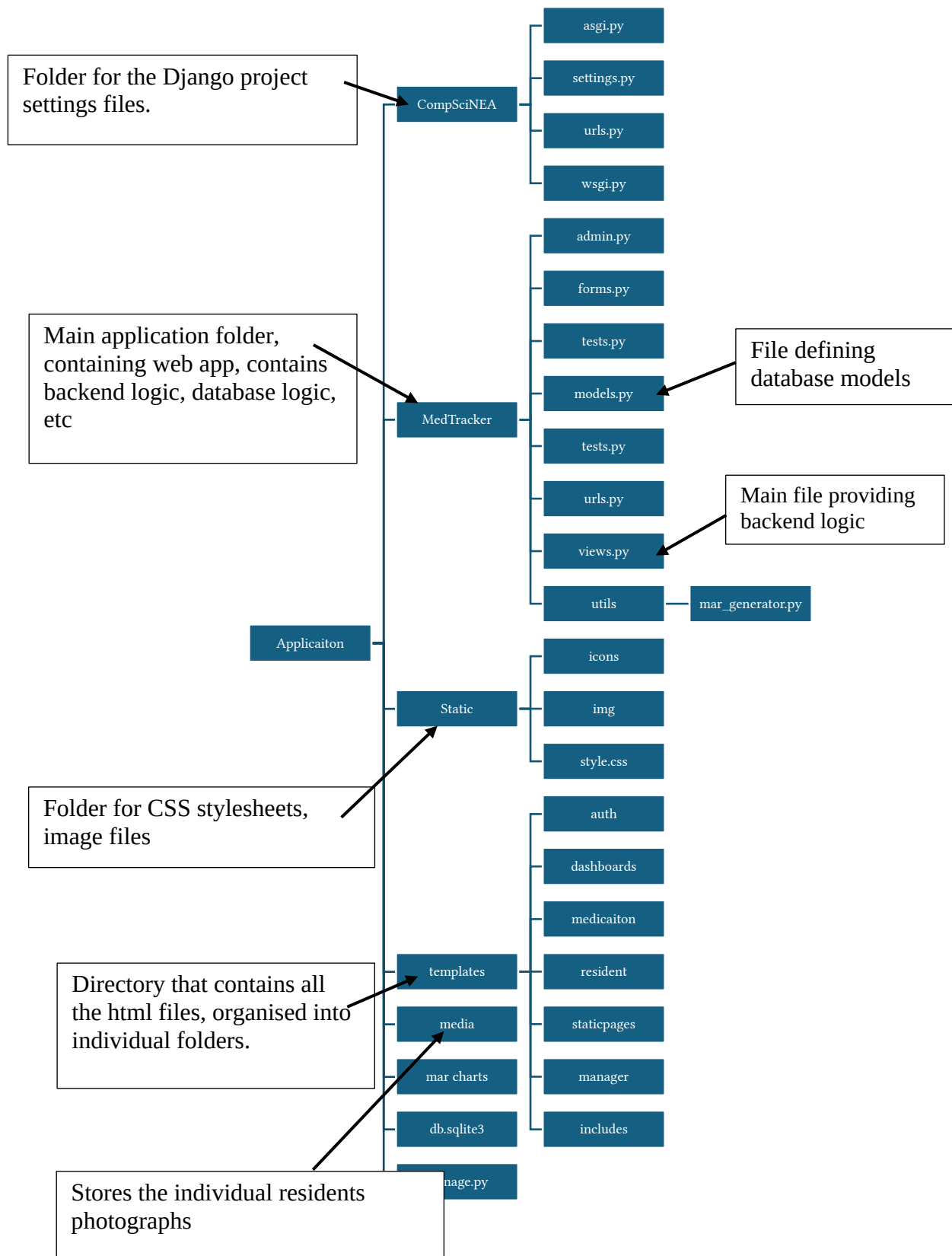
- 7. Carer Permissions
 - 7.1. Impliment controllable permissions for carer access
 - 7.1.1. Allow managers to assign permissons for an individual carer
 - 7.1.2. Allow managers to revoke permssions for an individual carer
 - 7.1.3. Create a custom decorator to control permissions
 - 7.1.3.1. Allow decorator to refuse/allow access dependant on different permissons
 - 7.1.4. Trigger error message/redirect for unauthorised access
- 8. Medical Administration logging
 - 8.1. Impliment a audit trail/log of medicine administration
 - 8.1.1. Log every instance of given medication
 - 8.1.1.1. Record timestamp at moment of administration
 - 8.1.1.2. Record which cares staff provided medication
 - 8.1.1.3. Record optional notes of medical administration
 - 8.1.2. Allow audit logs to be filtered by date/time
- 9. Carer Dashboard
 - 9.1. Provide carer dashboard with dynamic updates
 - 9.1.1. Automatically refresh the list of upcoming medication
 - 9.2. Calculate urgency of pending medications
 - 9.2.1. Display urgent medications with colour coded identifier

Documented Design

Contents

DOCUMENTED DESIGN.....	16
<i>Contents</i>	16
<i>File Structure Overview</i>	17
<i>Website Overview</i>	18
High Level Website overview.....	18
User workflow overview:.....	19
<i>The Data Layer</i>	26
Databases and Django Models.....	26
Entity Relationship Diagram:.....	26
Core tables/models.....	27
Carehome model.....	27
Invitation Code.....	28
User Models.....	29
Resident Models.....	30
Medication related model.....	30
Example Transaction statements.....	33
<i>Backend Layer</i>	36
User account creation and authentication.....	36
Registration view.....	39
Flowchart of the registration view.....	41
Login View.....	42
Resident Management.....	46
Resident creation.....	46
Editing a resident.....	47
Medicaition management.....	48
Medication inventory.....	48
Adding a medication.....	48
Search Medciation.....	48
Adding the medication.....	49
Creating a Medication variant.....	50
Medicaition Administration.....	51
Creating a schedule.....	51
Recording medication administration.....	52
Displaying the medication.....	52
Dashboard updates.....	55
Marking the medication as ‘Given’.....	56
Medical Administration Record (MAR chart) generation.....	57
Permissions and Security:.....	60
Authentication layer.....	60
User role verification.....	60
Carehome Association:.....	60
Carer Permissions:.....	60
Logout system.....	63
<i>Frontend Layer</i>	64
Base.html.....	64
Form rendering:.....	64
Error handling:.....	65
Role of jinja in dashboards:.....	66
Medication search rendering.....	68
Carer Dashboard.....	69

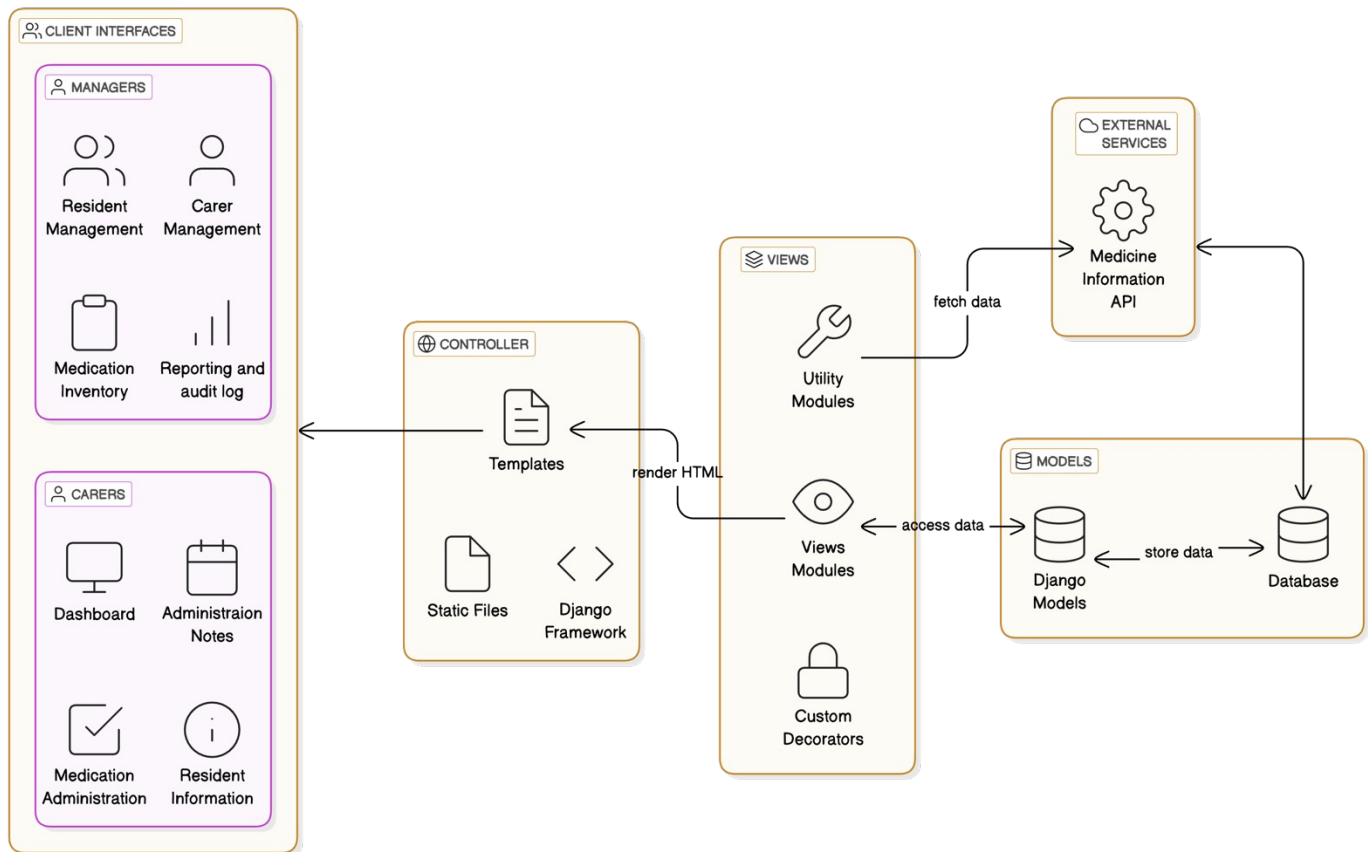
File Structure Overview



Website Overview

The design section will outline the functionality of the website, based upon overviews provided below. The high-level overview defines how my client/server interacts. Additionally, Outlines will be provided for the user workflow/journey.

High Level Website overview



The system is a web-based platform on a three-tier structure:

1. The data layer – PostgreSQL (data storage) and Django models (connects database to python backend)
2. Backend Layer – Python Logic through the Django web framework.
3. Frontend Layer – Jinja, Html5, bootstrap and Javascript enabling UI rendering

The project follows the Model, View, Controller software pattern. The models are represented as python class objects and are stored in the database, the views are individual functions that present and accept user information, and the controller is the Django URL routing system, that uses Jinja and HTTP. This approach allows for modularization where data handling, business logic, and user interface are kept separate, making the project both scalable and easy to maintain.

User workflow overview:

User creation:

- Manager registers a carehome
- Manager invites carers using invitation code
- Carer sign up and gain access to the assigned carehome

Medication management:

- Managers and Authorised carers add medications and assign them to residents
- System tracks the schedules and calculates urgency
- Dashboard displays the upcoming medications

Access control:

- Managers can assign permissions to different carers
- Carers can only interact with assigned residents and medications
- Carer logins can be restricted to certain timespans, until reauthorised by management

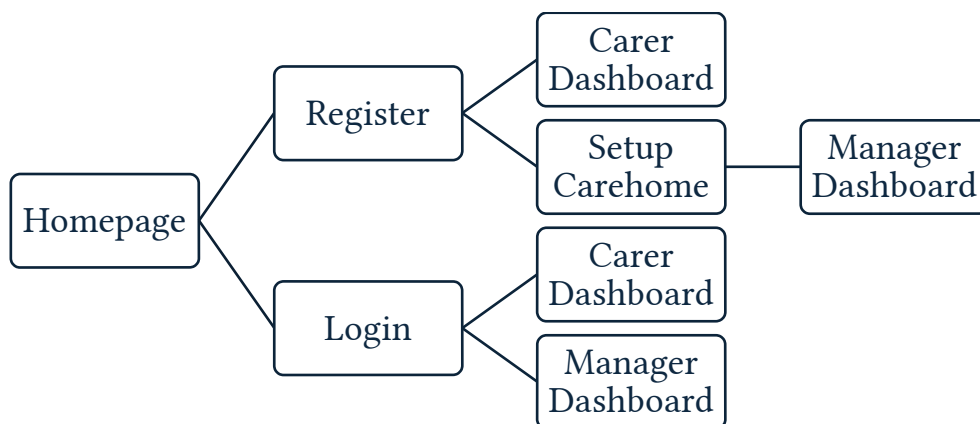


Figure 4: Overview of registration and log-in process

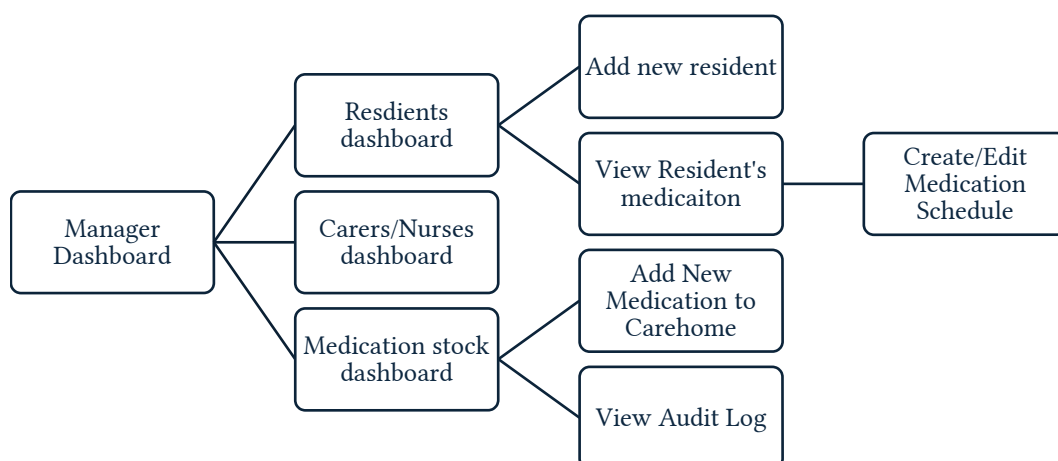


Figure 5: Overview of the Manager's workflow, including all managerial dashboards

Annotated Screenshots displaying general overview:



Figure 6: index.html - the homepage

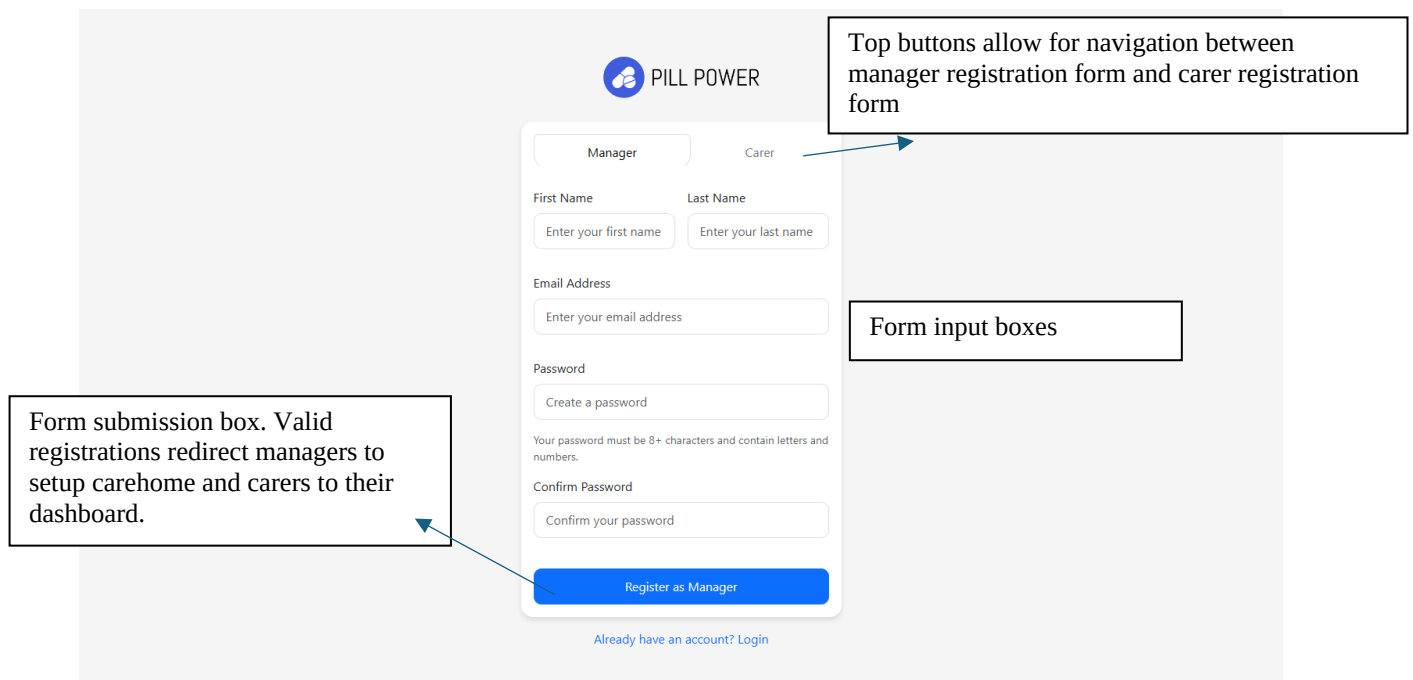


Figure 7: Registration page

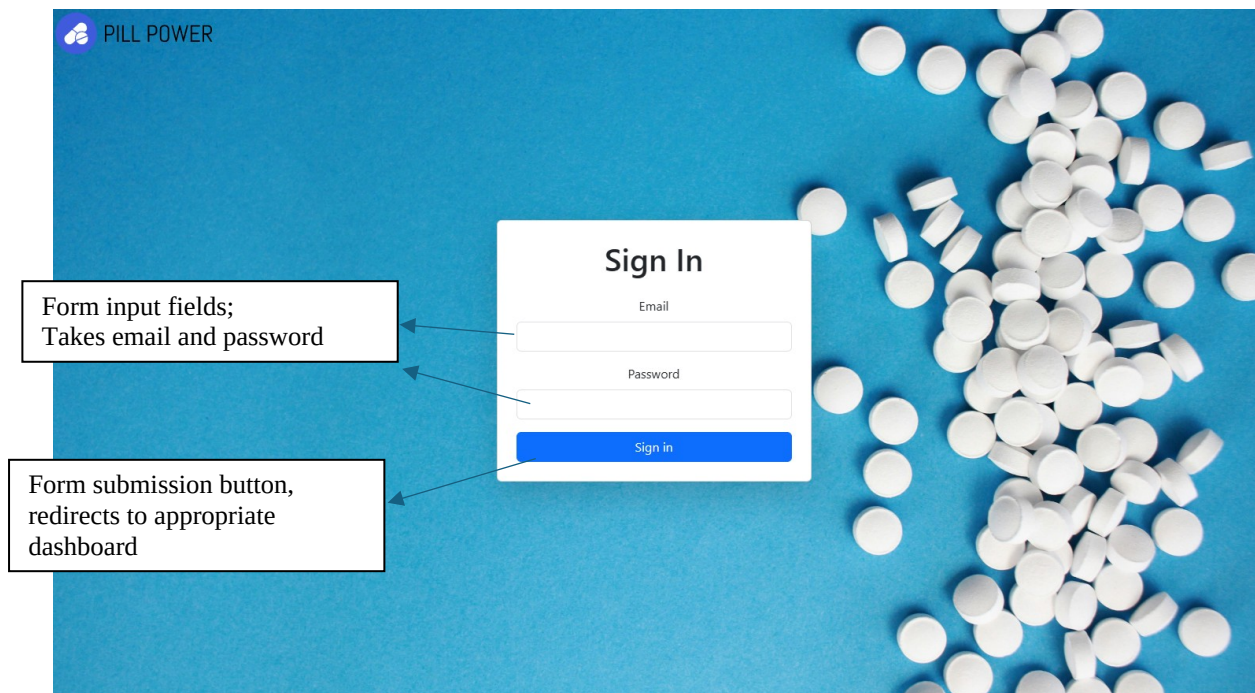


Figure 8: Sign-in Page

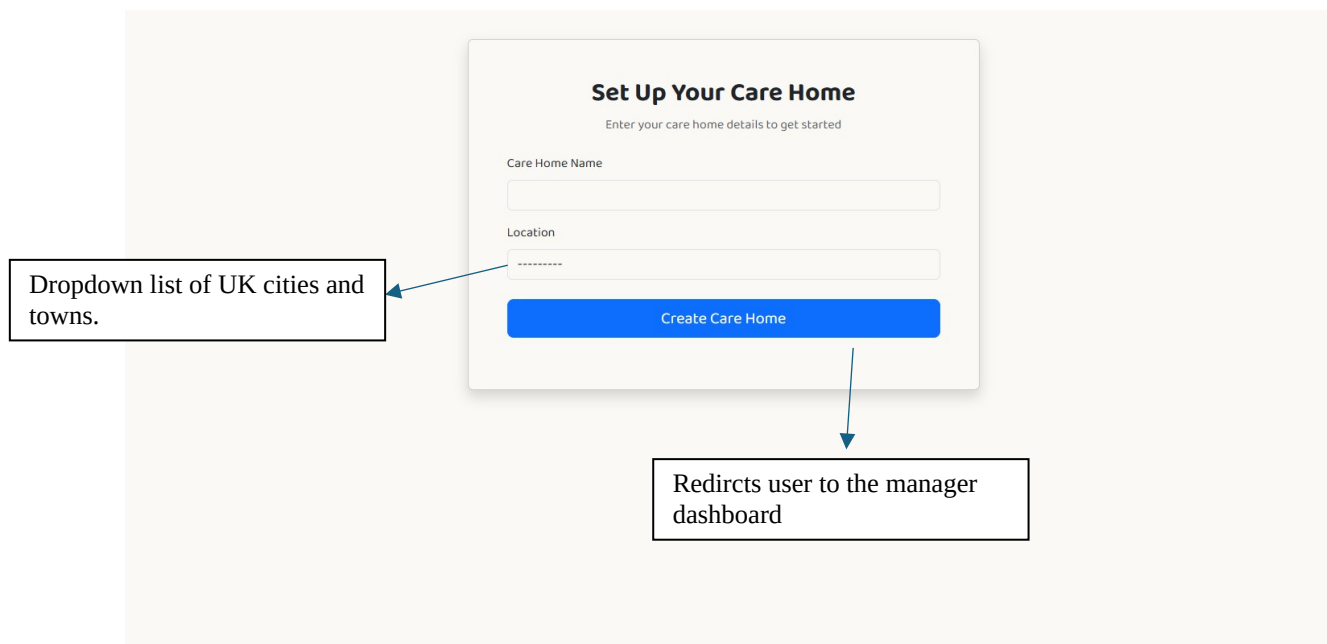


Figure 9: Setup Carehome, redirect after a new manager registration

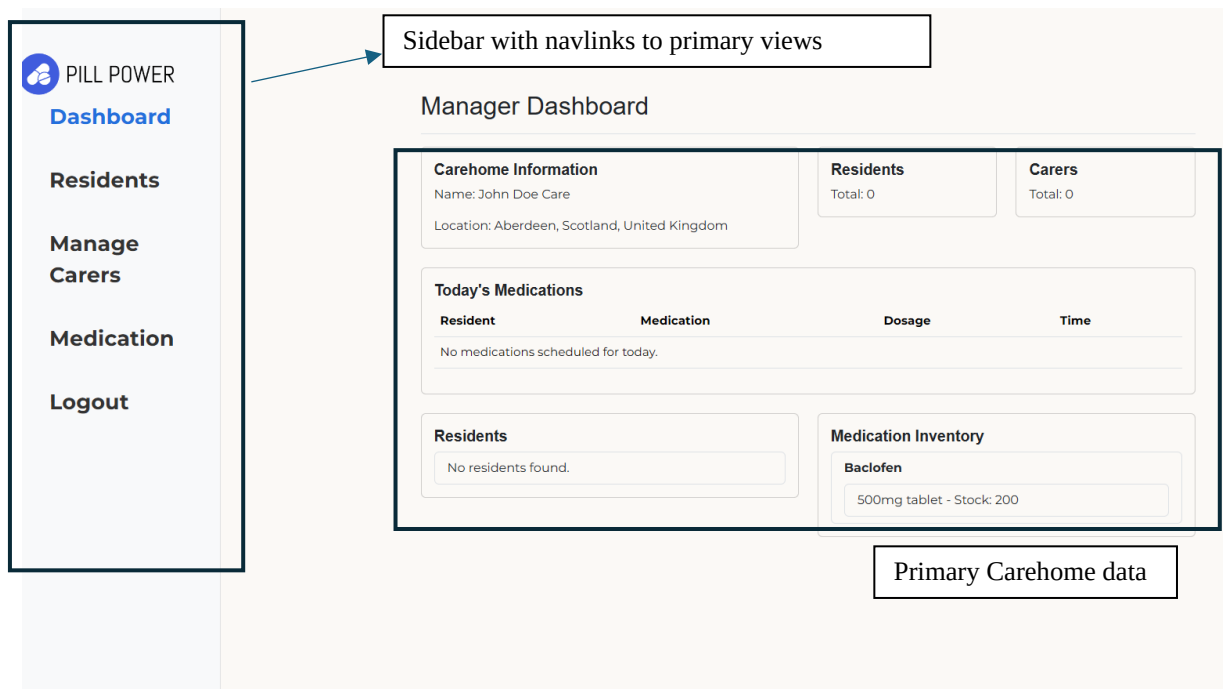


Figure 10: Managerial carehome Overview

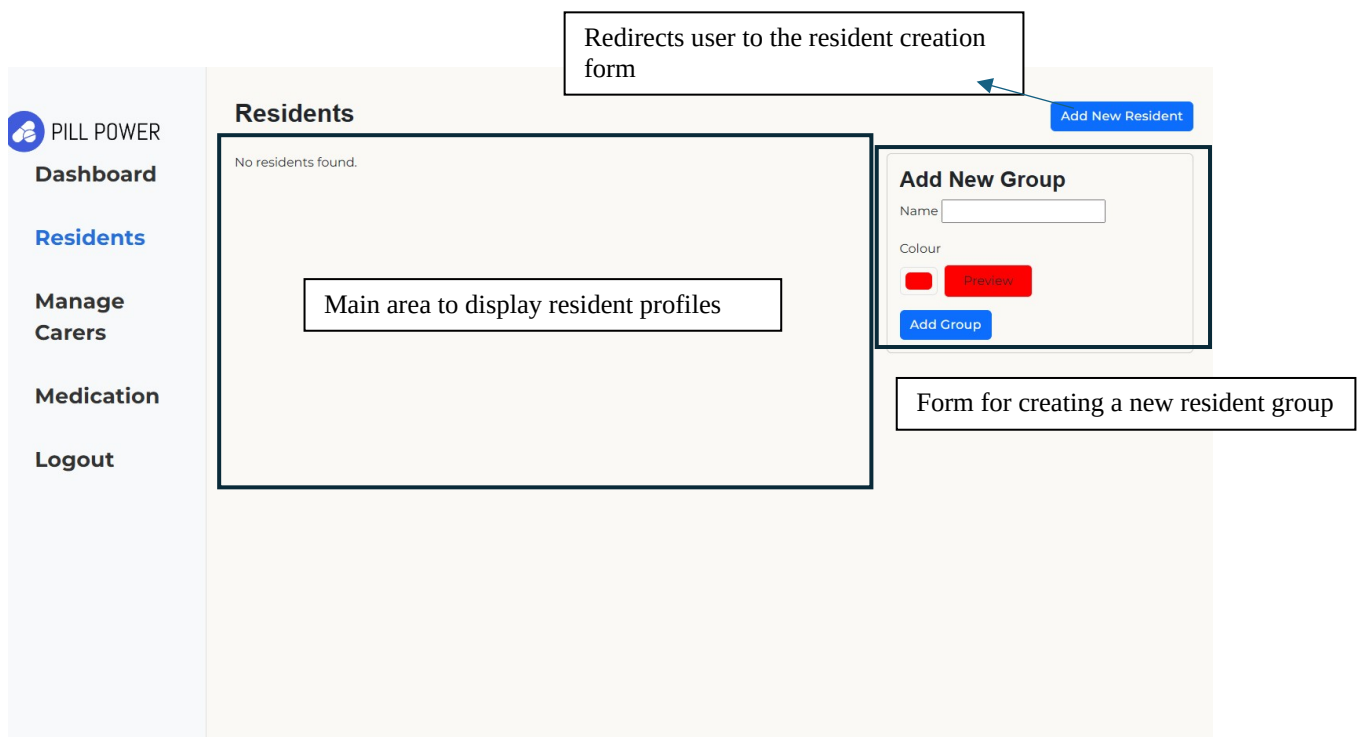


Figure 11: Residents Dashboard

PILL POWER

Dashboard

Residents

Manage Carers

Medication

Logout

Add New Resident

Personal Information

[Grab your reader's attention with a great quote]

Date of Birth*
dd/mm/yyyy

Resident Group

Medical Information

Medical Conditions
List any medical conditions

Allergies
List any allergies

Photo

Resident Photo
Choose File No file chosen

Cancel

Add Resident

PILL POWER

Dashboard

Residents

Manage Carers

Medication

Logout

Manage Invitation Codes

Generate one-time invitation codes for new carers to join John Doe Care

Generate New Invitation Code

Active Invitation Codes

	Actions
Invitation valid for (days): 7	

Carers

No carers found.

Care Home Code

Share codes with carers to join your care home:

Manage Staff Invitations

Resident Groups

No groups created yet.

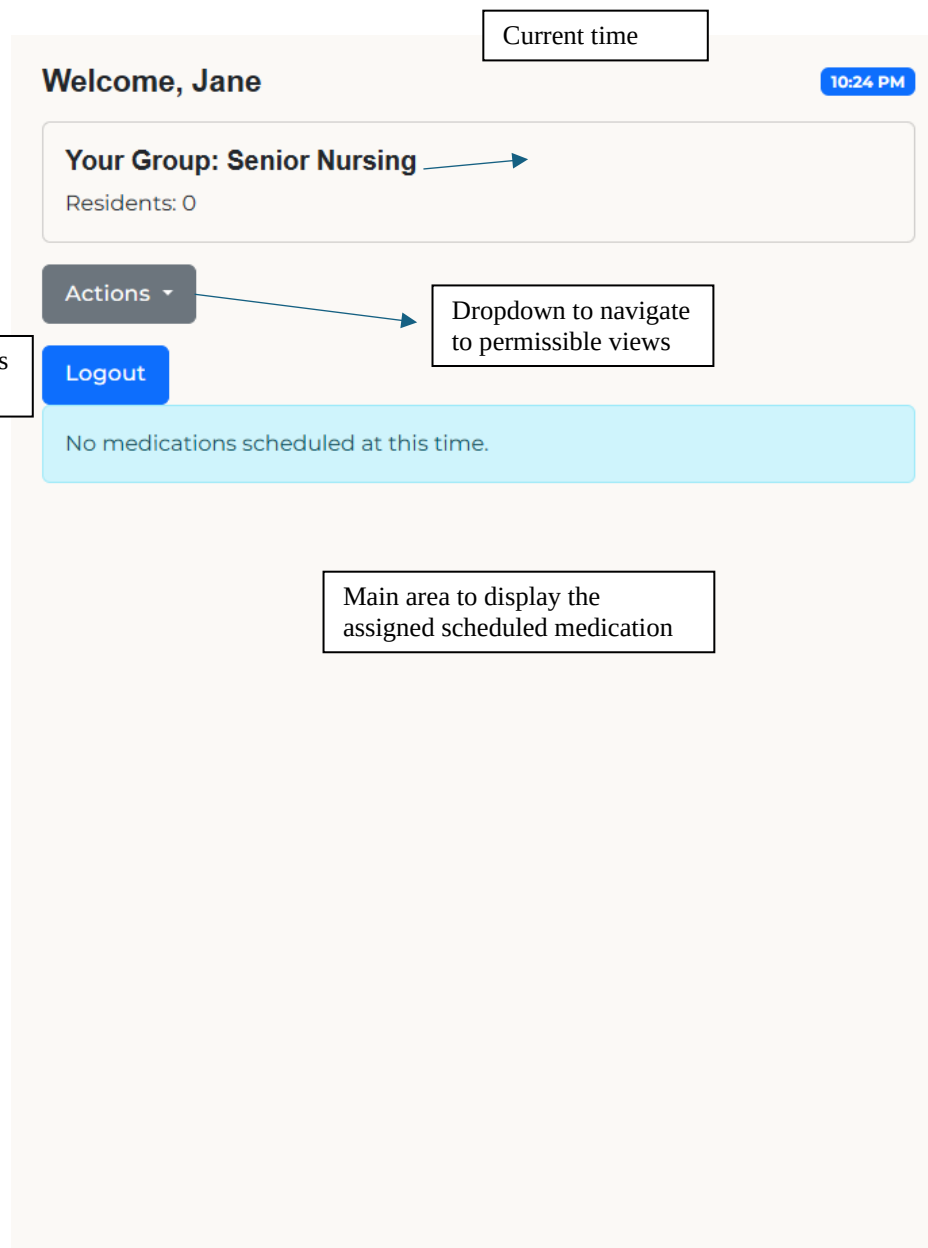
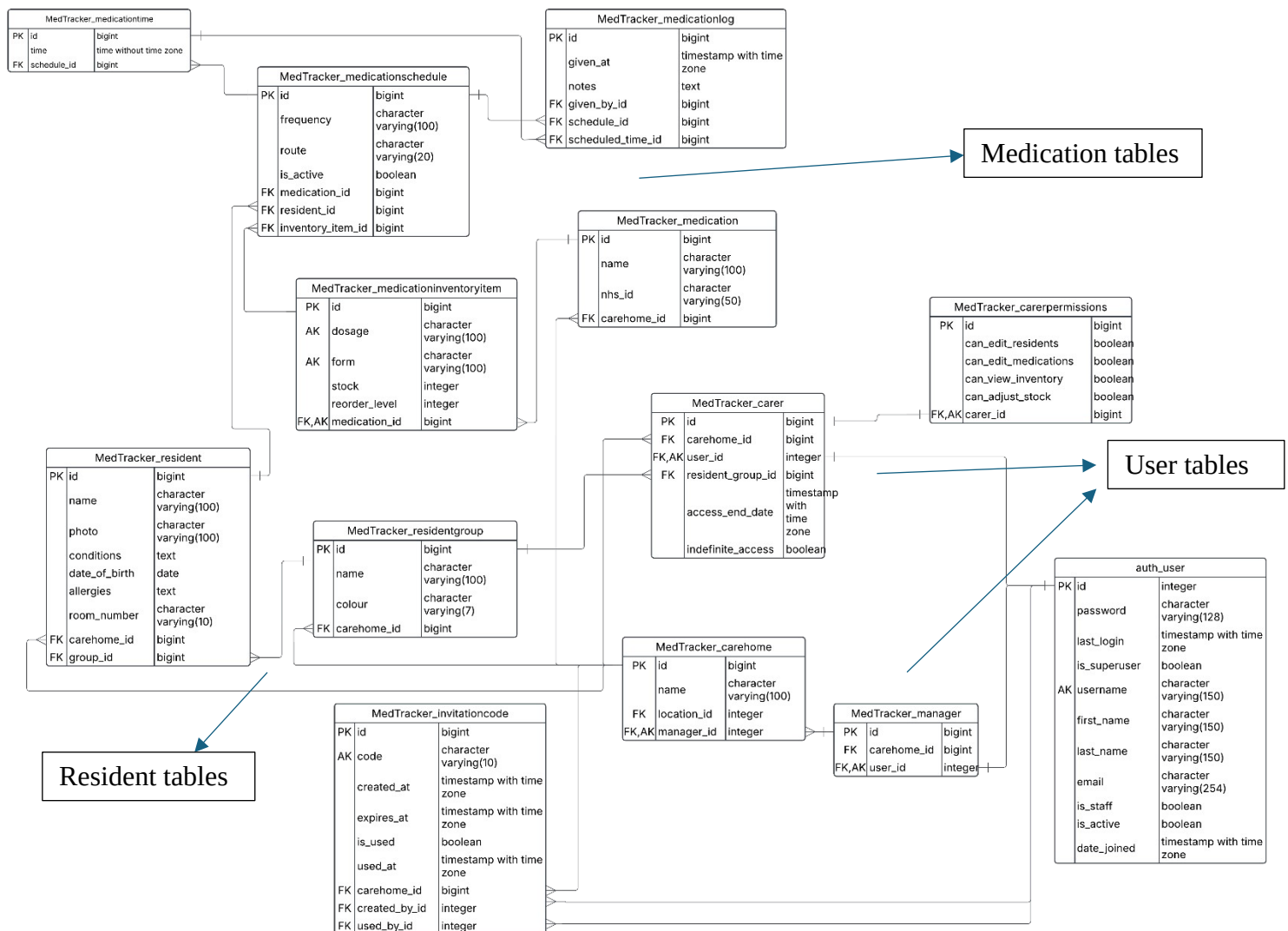


Figure 16: The Carer Dashboard

The Data Layer

Databases and Django Models

Entity Relationship Diagram¹¹:



The database is a relational database implemented using postgresql and Django models. The system is designed to ensure data integrity, role-based access control and efficient medication tracking.

Database models are defined in Models.py and are mapped one-to-one with database tables. This allows Django to work directly with the python objects, reducing the risk of SQL injection attacks, which is integral when dealing with sensitive medical data.

¹¹ Diagrams created using lucidcharts @ <https://lucid.app/lucidchart>

Core tables/models

The system's database consists of multiple tables, with anonymization as a key design principle. While Django automatically generates an 'id' primary key for each table by default, this database design explicitly defines a custom ID for each table, overriding Django's default, for use with table relations.

Pseudonymisation is mentioned 14 times in GDPR and is a key part of the database's design. Using ID's creates a pseudonym, as nothing can be inferred from a Users/Residents ID and relations are formed using ID's as foreign keys for table relations. This creates a separation of people from their data.

Below is a breakdown of the key tables and relationships:

Carehome model

MedTracker_carehome		
PK	id	bigint
	name	character varying(100)
FK	location_id	integer
FK,AK	manager_id	integer

The carehome table serves as a central entity. It represents a care facility with:

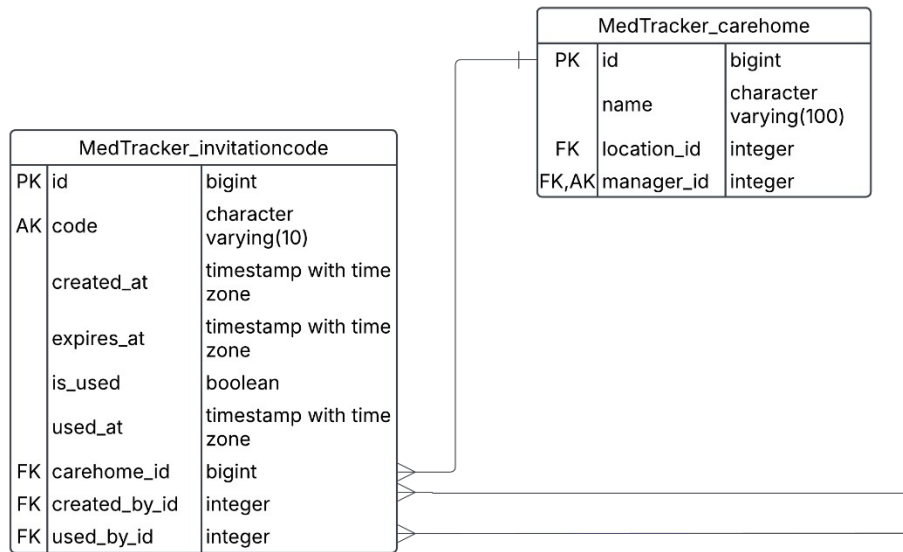
- Name
- Location_id
- Manager_id

Its Location_id has a link to a foreign key to a city record stored in the Django cities_light cities table within the database.

The manager_id field creates a relational link to a manager user's id in a one-to-many relationship. This modular design allows for one carehome to have multiple managers if required.

However, For the scope of my project, my end-user is the sole manager of their carehome, therefore, I only require functionality for one manager, however the database is designed for wider scope if the project is expanded.

Invitation Code



The InvitationCode model is designed to store information about invitation codes, which are used to allow carers to sign up to a carehome, and their usage. Its fields and relationships are as follows:

Fields:

- **code:** A CharField with a unique constraint. This ensures that each invitation code is unique.
- **carehome:** A ForeignKey to the CareHome model, with a CASCADE deletion policy. This associates each invitation code with a specific care home.
- **created_by:** A ForeignKey to the User model. This tracks the manager who created the invitation code.
- **created_at:** A DateTimeField which stores the timestamp when the invitation code was created.
- **expires_at:** A DateTimeField that specifies when the invitation code will expire.
- **is_used:** A BooleanField with a default value of False. This indicates whether the invitation code has been used.
- **used_by:** A ForeignKey to the User model, tracks the user who used the invitation code.
- **used_at:** A DateTimeField which stores the timestamp when the invitation code was used.

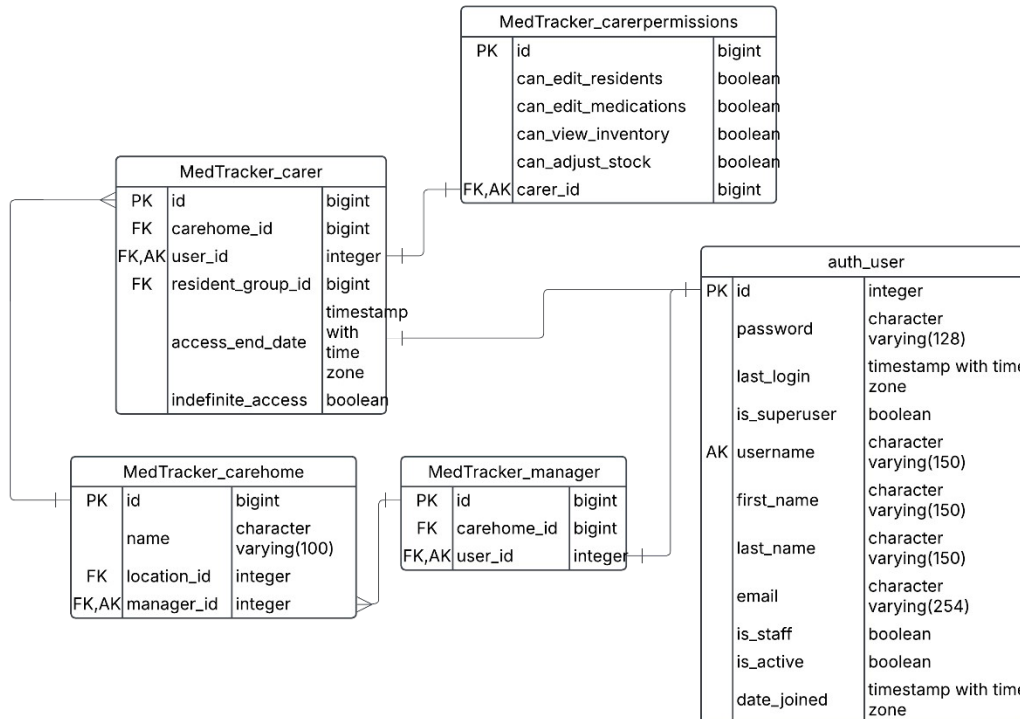
Relationships:

carehome: Links the invitation code to a specific care home, with a many-to-one relationship, as a single carehome can have multiple invitation codes.

created_by: Tracks the manager who generated the code.

used_by: Tracks the carer who used the code to sign up.

User Models



The system uses Django's built-in `auth_user` model as a foundation, due to its high security and authentication.

Core usermodel

The Django user model takes in Username and password for authentication.

The email is stored in the username field and allows for email + password login. This follows a unique constraint, as the username is an alternative primary key, alongside the ID, for identifying the user.

Role-specific models:

1. Manager

Managers have a one-to-one relationship with the Django user model. They are associated with a carehome (which they manage).

2. Carer

The carer has a one-to-one relation with the Django user.

They must be associated with a specific carehome.

Cares have limited time access through the fields:

Access_end_date – a variable datetime field

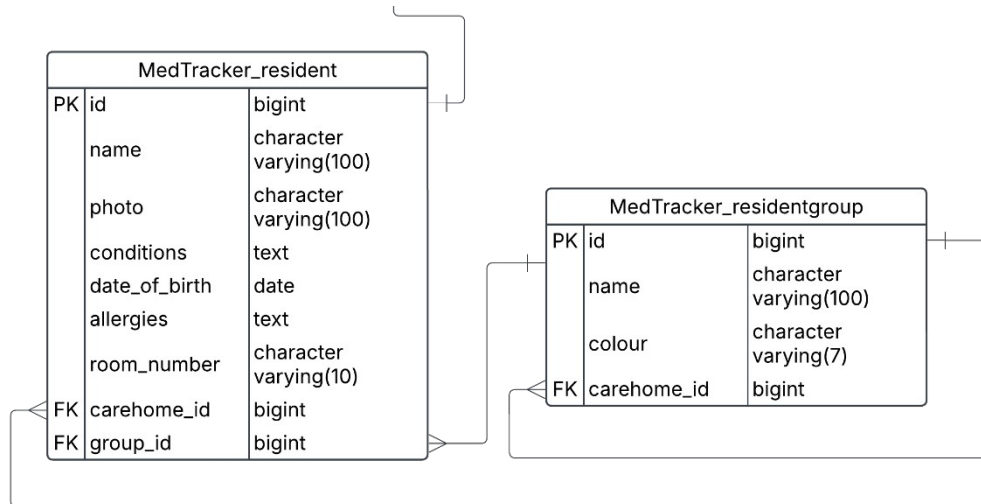
indefinite_access – a Boolean field

They have a one-to-one relationship with the permissions table.

3. Carer permissions

This table is created automatically when a new carer is created. Maintains a one-to-one relationship with a carer. It is a series of Boolean fields which allow for control of Carer access.

Resident Models



The Resident table represents individual residents in a care home. It stores personal and medical information about each resident.

Fields:

- **name:** CharField, Stores the resident's name as a string.
- **photo:** ImageField, An optional field for storing a photo of the resident
- **date_of_birth:** DateField, Stores the resident's date of birth, used to calculate their age.
- **conditions:** TextField, A text field for listing any medical conditions the resident has.
- **allergies:** TextField, A text field for listing any allergies the resident has.
- **room_number:** CharField An optional field for specifying the resident's room number.
- **Carehome_id:** a foreign key linking the resident to a carehome.
- **Group_id:** A foreign key linking the resident to a ResidentGroup. Enables grouping residents based on specific criteria (e.g., care needs, location, etc.).

Relationships:

Group: a many-to-one relationship with a resident group, a resident can only belong to one group, but a group can have multiple residents.

Carehome: creates a many-to-one relationship with a carehome.

The resident group table

A one-to-many relation with residents.

Its fields store the name of the resident group, as well as a hexadecimal colour used for clear identification of different groups on the frontend.

Medication related model

Each medication related table in the database serve different roles:

1. Medication – base medication table
2. MedicaitonInventoryItem – stores a specific variant of medication for their specific dosage/form/stock
3. MedicationSchedule – Links residents to the medication they need to take
4. Medication time – records the exact time a medication should be administered
5. Medication Log – records the medication administration for audit log

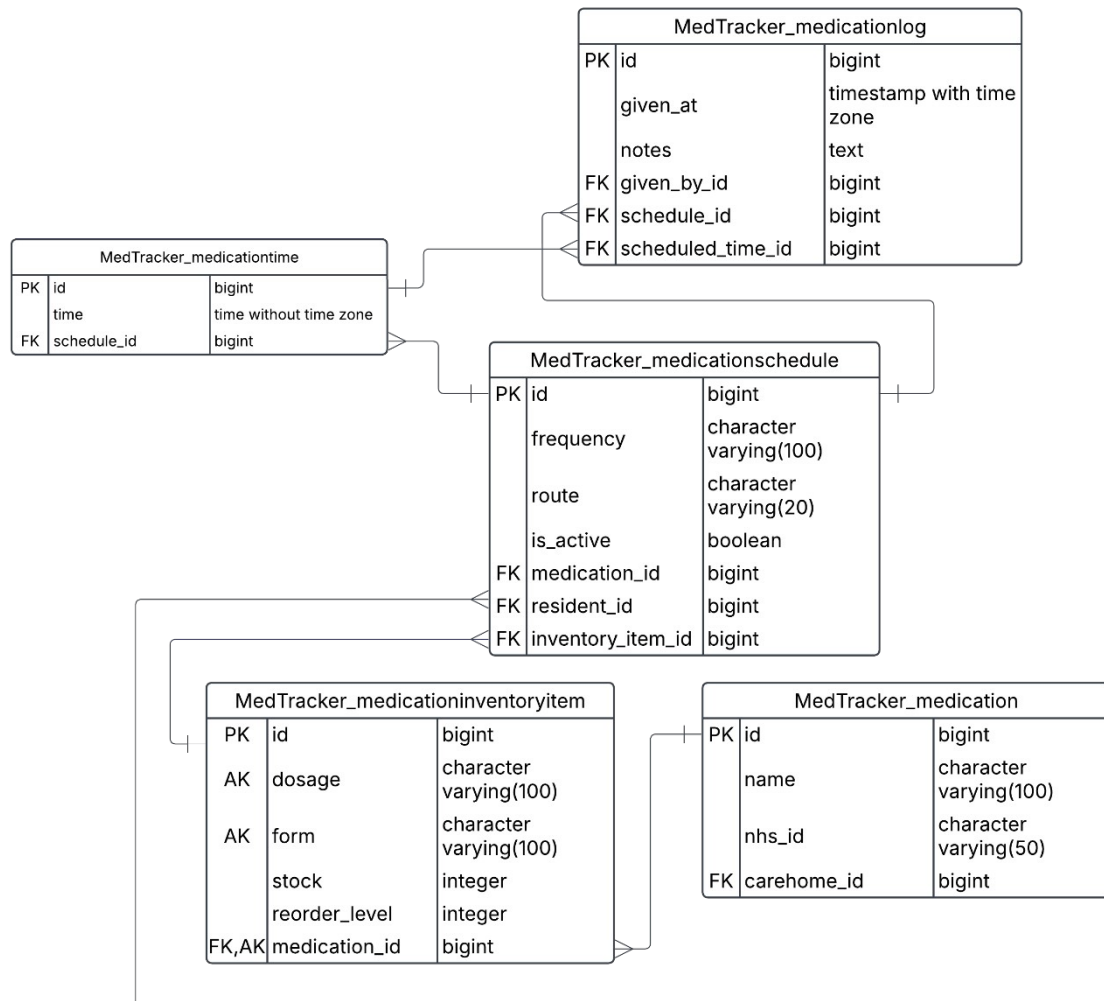


Figure 17: The medication tables and their realtions in the database. Other tables and their relations have been abstracted.

Medication table:

Stores the basic details of a medication available in a specific carehome, the data for medication is from the NHS api, and the name of the medication as well as the URL id it points to is stored in the table.

MedicationInventoryItem:

This represents a specific variant (i.e 500mg tablet) of a medication and tracks its stock. It has a many-to-one relationship with medication, as a single medication can have multiple variants as each drug has its own individual dosage.

Medication Schedule:

This table defines the adminsitraion plan for a specific medication variant (Stored in the MedicationInventoryItem) for a particular Resident.

MedicationTime:

This table storess a record of the specific time of the day that a medication needs to be administerd. It has a many-to-one relationship with a medication schedule, corresponding to its frequency. For example, a schedule with 2-time frequencies being 9:00 AM and 9PM would create two records in MedicaitonTime.

MedicationLog:

This table is the Audit log for medication, tracking every instance medication was actually given, linking each to a specific Schedule, Time and Carer who administered it.

A log entry is created when a carer clicks “mark given” on the carer dashboard.

The table is used for both the Audit log and MAR chart generation.

Example Transaction statements

The project was written using Django Object Relational Mapping. However, to help the documentation of the program, transactions have been translated into their equivalent SQL statements (Postgresql), provided below:

Process of creating a new carer user:

-- Create User

```
INSERT INTO "auth_user" ("password", "last_login", "is_superuser", "username", "first_name",  
"last_name", "email", "is_staff", "is_active", "date_joined")  
VALUES ('hashed_password', NULL, false, 'carer@example.com', 'FirstName', 'LastName',  
'carer@example.com', false, true, NOW())  
RETURNING "auth_user"."id";
```

-- user_id = 2

-- Find valid Invitation and CareHome ID (e.g.invitation code 'ABC123XYZ', carehome_id = 5)

```
SELECT "MedTracker_invitationcode"."carehome_id"  
FROM "MedTracker_invitationcode"  
WHERE "MedTracker_invitationcode"."code" = 'ABC123XYZ' AND "MedTracker_invitationcode"."is_used"  
= false AND "MedTracker_invitationcode"."expires_at" > NOW()  
LIMIT 1;
```

-- Create Carer object

```
INSERT INTO "MedTracker_carer" ("user_id", "carehome_id", "resident_group_id", "access_end_date",  
"indefinite_access")  
VALUES (2, 5, NULL, NULL, false)  
RETURNING "MedTracker_carer"."id";
```

-- Create CarerPermissions (triggered by Carer.save signal/method)

```
INSERT INTO "MedTracker_carerpermissions" ("can_edit_residents", "can_edit_medications",  
"can_view_inventory", "can_adjust_stock", "carer_id")  
VALUES (false, false, false, false, 3);
```

-- Mark Invitation as Used

```
UPDATE "MedTracker_invitationcode"  
SET "is_used" = true, "used_by_id" = 2, "used_at" = NOW()  
WHERE "MedTracker_invitationcode"."code" = 'ABC123XYZ';
```

COMMIT;

Adding medication to the Care Home's database:

-- carehome_id = 5

```
INSERT INTO "MedTracker_medication" ("name", "nhs_id", "carehome_id")  
VALUES ('Aspirin', aspirin-for-pain-relief', 5);
```

Adding a Medication Inventory Item (Variant of medication):

-- medication_id = 10


```
INSERT INTO "MedTracker_medicationinventoryitem" ("medication_id", "dosage", "form", "stock",
"reorder_level")
VALUES (10, '500mg', 'Tablet', 100, 20);
```

Creating a medication schedule:

```
-- resident_id=15, medication_id=10, inventory_item_id=12
INSERT INTO "MedTracker_medicationschedule" ("resident_id", "medication_id", "inventory_item_id",
"frequency", "route", "is_active")
VALUES (15, 10, 12, '2', 'oral', true)
RETURNING "MedTracker_medicationschedule"."id";
```

```
-- schedule_id = 7
INSERT INTO "MedTracker_medicationtime" ("schedule_id", "time")
VALUES (7, '08:00:00');
```

```
INSERT INTO "MedTracker_medicationtime" ("schedule_id", "time")
VALUES (7, '20:00:00');
```

```
COMMIT;
```

Administering medication:

```
-- 1. Retrieve the inventory item ID associated with the schedule
```

```
SELECT "inventory_item_id"
FROM "MedTracker_medicationschedule"
WHERE "id" = 7;
```

```
-- Assume this returns inventory_item_id = 12
```

```
-- 2. Check current stock and lock the row for update
```

```
SELECT "stock" FROM "MedTracker_medicationinventoryitem" WHERE "id" = 12 FOR UPDATE;
-- Assume stock is > 0 based on the check
```

```
-- 3. Create the Medication Log entry
```

```
INSERT INTO "MedTracker_medicationlog" (
  "schedule_id",
  "scheduled_time_id",
  "given_by_id",
  "notes",
  "given_at"
) VALUES (
  7, -- schedule_id from input
  25, -- time_slot_id from input
  3, -- carer_id from request.user.carer
  'Taken with water', -- notes from input
  NOW() -- Timestamp of when the log is created
);
```

```
-- 4. Decrement the stock for the specific inventory item
UPDATE "MedTracker_medicationinventoryitem"
SET "stock" = "stock" - 1
WHERE "id" = 12;

COMMIT;
```

Updating a residents information:

```
UPDATE "MedTracker_resident"
SET "room_number" = '10B', "allergies" = 'Penicillin'
WHERE "id" = 15; -- Assuming resident_id = 15
```

Backend Layer

The backend of this project is implemented using Django, a Python-based web framework. It is structured to handle user authentication, care home management, medication tracking, and resident management.

User account creation and authentication

The Registration and Authorization system is responsible for managing user sign-up, authentication, and session control.

Registration is handled by a CustomUserForm which can be found in forms.py, it inherits Django's UserCreationForm. The form differs from Django's default registration form as it takes in Firstname, Lastname and Email inputs, as opposed to just the username and password Django's default form takes.

The CustomUserForm found in Forms.py

```
class CustomUserForm(UserCreationForm):
    first_name = forms.CharField(
        max_length=15,
        required=True,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your first name'
        })
    )
    last_name = forms.CharField(
        max_length=15,
        required=True,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your last name'
        })
    )
    email = forms.EmailField(
        required=True,
        widget=forms.EmailInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your email address'
        })
    )
    password1 = forms.CharField(
        widget=forms.PasswordInput(attrs={
            'class': 'form-control',
            'placeholder': 'Create a password'
        })
    )
    password2 = forms.CharField(
        widget=forms.PasswordInput(attrs={
            'class': 'form-control',
            'placeholder': 'Confirm your password'
        })
    )
```

```

    })
)
class Meta:
    model = User
    fields = ('first_name', 'last_name', 'email', 'password1', 'password2')

```

The carer and manager forms inherit this base form and override the base Django save () method.

ManagerSignUpForm, which inherits from the CustomUserForm, found in Forms.py

```

class ManagerSignUpForm(CustomUserForm):

    class Meta(CustomUserForm.Meta):
        model = User
        fields = ('first_name', 'last_name', 'email', 'password1', 'password2')

    def save(self, commit=True):
        user = super().save(commit=False)
        user.username = user.email.lower()
        if commit:
            user.save()
            Manager.objects.create(user=user)
        return user

```

CarerSignUpForm, which inherits from the CustomUserForm, found in Forms.py

```

class CarerSignUpForm(CustomUserForm):
    invitation_code = forms.CharField(
        max_length=10,
        required=True,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your invitation code'
        })
    )

    class Meta(CustomUserForm.Meta):
        model = User

    def clean_invitation_code(self):
        code = self.cleaned_data.get('invitation_code')
        now = timezone.now()

        # Check if code exists and is valid
        try:
            invitation = InvitationCode.objects.get(

```

```

        code=code,
        is_used=False,
        expires_at__gt=now
    )
except InvitationCode.DoesNotExist:
    raise forms.ValidationError(
        "Invalid or expired invitation code. Please check and try again."
    )

return code

def save(self, commit=True):
    user = super().save(commit=False)
    user.username = user.email.lower()
    if commit:
        user.save()

    # Get the invitation code and its related carehome
    invitation = InvitationCode.objects.get(code=self.cleaned_data['invitation_code'])
    carehome = invitation.carehome

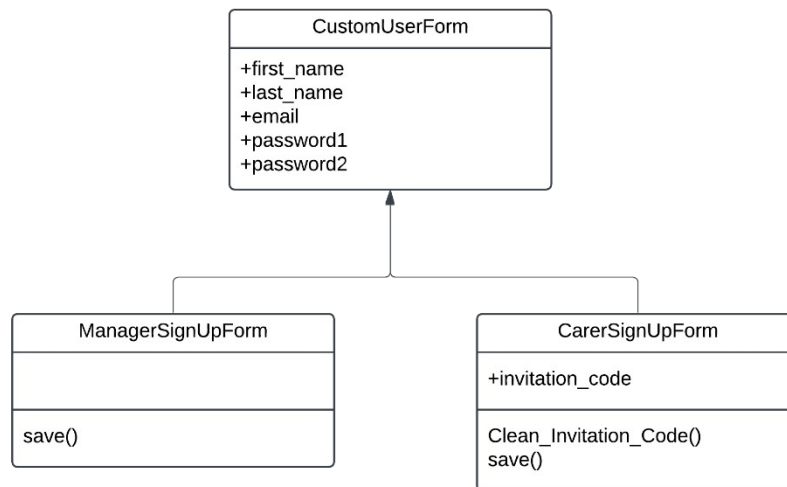
    # Create carer object
    Carer.objects.create(user=user, carehome=carehome)

    # Mark invitation as used
    invitation.mark_as_used(user)

return user

```

The inheritance is reflected in this diagram:



Save() method

Instead of Django's default username behaviour, the email is assigned as the User's username ensuring uniqueness as a composite primary key.

The `user.save()` Stores a User instance in the database.

Adhering to GDPR data standards, password encryption is utilised. Django uses a PBKDF2 algorithm to encrypt and hash passwords. It generates a random salt for each password, using the SHA256 algorithm. The hash function is applied multiple times (for this project, based on Django 4.0, it has a default 260,000 operations) making brute-force attacks computationally expensive.

The resulting hash, along with salt and algorithm used, is stored in the database in the format: Password ::= <algorithm>\$<iterations>\$<salt>\$<hash>

The `save()` function has also been edited to create either a manager or Carer instance in the database. These instances are created with their associated `user` id's as foreign keys. Carers who used a carehome invitation code have the code automatically marked as used.

The carer form extends the functions of its parent class by adding support for the invitation code. The `def clean_invitation_code` method checks 3 things, if the invitation code provided:

- Exists in the database
- Has not been used (used = False check)
- Has not expired (expires_at__gt (greater than) = `timezone.now`)

If a condition fails, an error is raised and prevents the user from registering.

Registration view

The registration view handles the registration process for both user types. It's protected by two decorators:

`@csrf_protect` – Cross-Site request forgery protection

`@ensure_csrf_cookie` – Makes sure that CSRF cookie is set in browser, used for the continuation of registration when managers are redirected to carehome setup.

Initially, the function initializes the two forms:

```
form1 = ManagerSignUpForm(request.POST)
```

```
form2 = CarerSignUpForm(request.POST)
```

The POST data dictates the form that gets processed, Manager registrations check for 'sign_up_manager' and carer registrations check for 'sign_up_carer'.

For Manager registrations:

```
if 'sign_up_manager' in request.POST:
    form = ManagerSignUpForm(request.POST)
    if form.is_valid():
        user = form.save()          # Create the user and manager profile
        login(request, user)        # Log the user in
        return redirect('setup_carehome') # Redirect to setup a care home
```

For Carer registrations

```
elif 'sign_up_carer' in request.POST:
    form = CarerSignUpForm(request.POST)
    if form.is_valid():
        user = form.save()          # Create the user and carer profile
        login(request, user)        # Log the user in
        return redirect('carer_dashboard') # Redirect to carer dashboard
```

The view also has error handling

```
for field, errors in form.errors.items():
    for error in errors:
        messages.error(request, f"{error}")
and checks specifically for if a email is already in use:
except IntegrityError as e:
    if "unique constraint" in str(e).lower():
        messages.error(request, "An account with that email already exists.")
```

Following a Managers registration, they are redirected to `def setup_carehome`. This checks if the manager already has a carehome assigned to them, if yes they are redirected to their dashboard, otherwise, they are provided the carehome creation form.

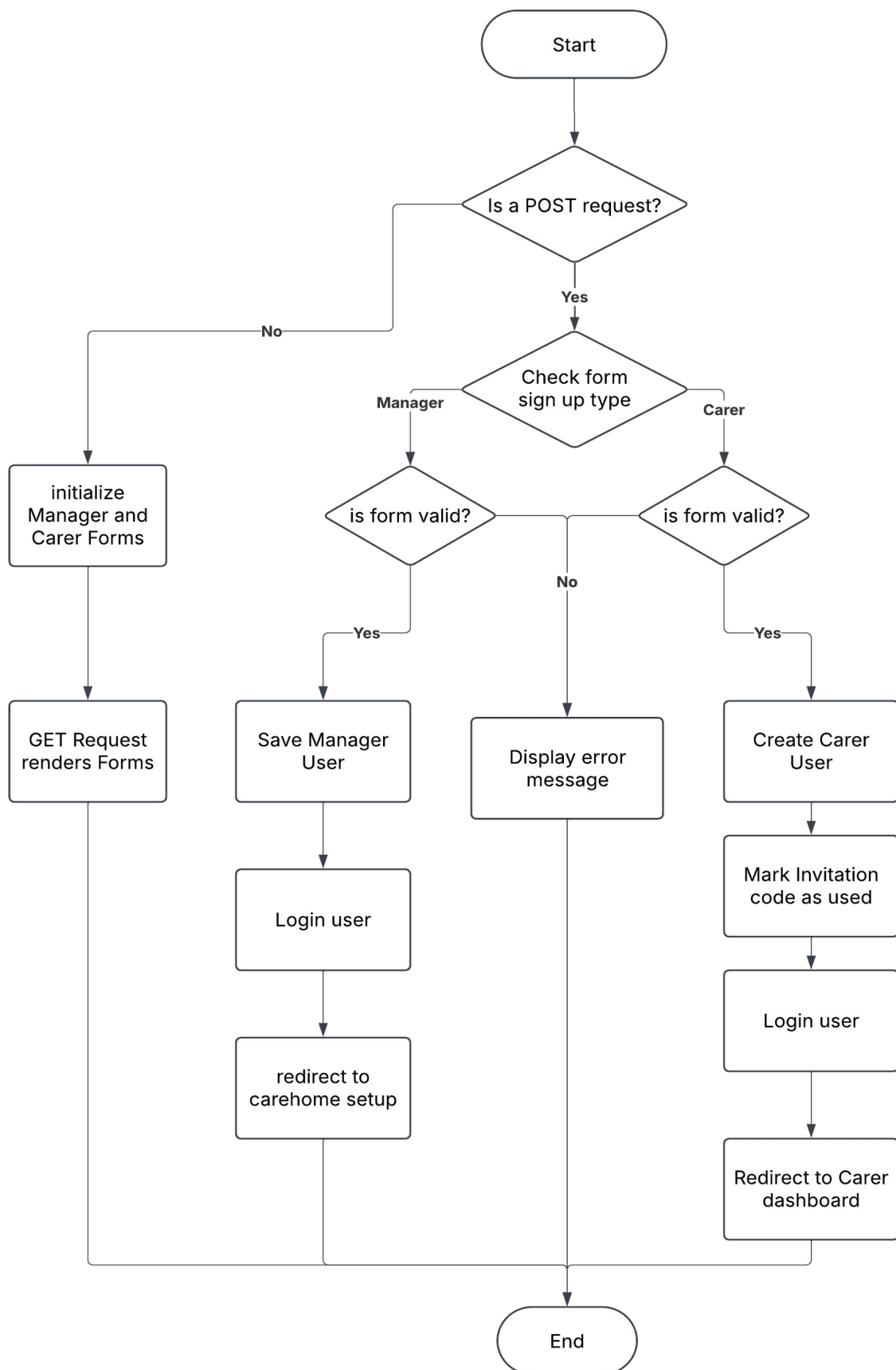
```
if request.method == 'POST':
    form = CareHomeSetupForm(request.POST)
    if form.is_valid():
        carehome = form.save(commit=False)

        carehome.manager = user
        carehome.save()
        manager.carehome = carehome
        manager.save()
        return redirect('manager_dashboard')
```

The form takes in a name, and the location is chosen from a record in the Django cities_light city table, which is a Django module stored in the database models.

```
class CareHomeSetupForm(forms.ModelForm):
    location = forms.ModelChoiceField(queryset=City.objects.filter(country__code2='GB'))
    class Meta:
        model = CareHome
        fields = ['name', 'location']
```

Flowchart of the registration view



Login View

If the request method is GET, the first thing the login view does is `request.user` and checks for user authentication.

The function initially checks if the user is already logged in. Valid Manager and Carer authorised users are redirected to their correct dashboard, while Carers who were logged in but now are outside of their access date are rejected.

The method inside the carer model checks for valid access.

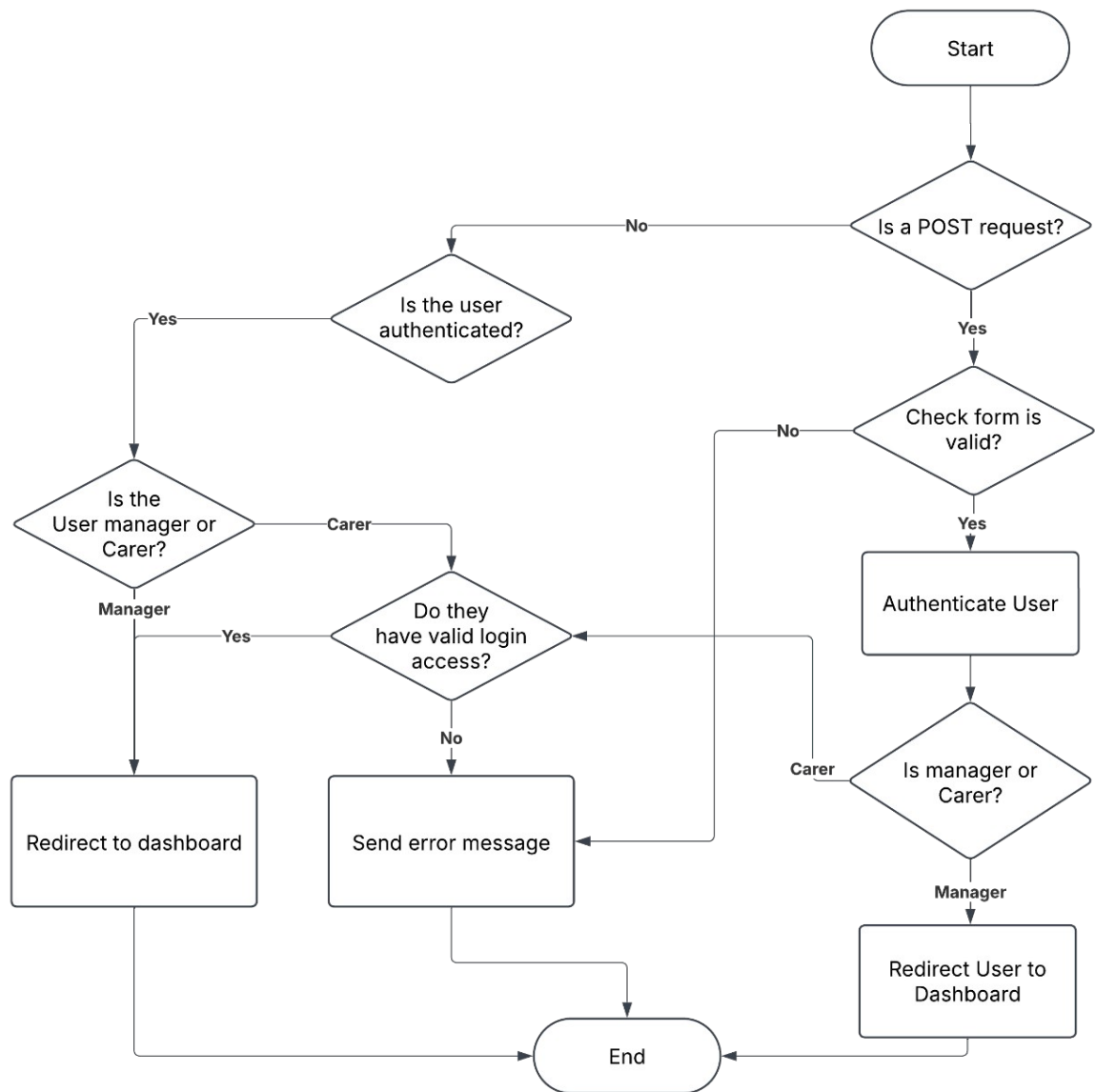
```
#Method inside the Carer user model
def has_valid_access(self):
    if self.indefinite_access:
        return True
    if self.access_end_date and timezone.now() < self.access_end_date:
        return True
    return False
```

Initial User authentication check

```
user = request.user
# if user is logged in, they are redirected to the dashboard
if request.user.is_authenticated:
    try:
        manager = request.user.manager
        return redirect('manager_dashboard')
    except Manager.DoesNotExist:
        try:
            carer = request.user.carer
            # Check if carer's access is still valid
            if not carer.has_valid_access:
                logout(request)
                messages.error(request, "Your access period has expired. Please contact your manager.")
                return redirect('login')
            return redirect('carer_dashboard')
        except Carer.DoesNotExist:
            logout(request)
            messages.error(request, "User account type not recognized. Please contact support.")
            return redirect('login')
```

Processing Form

It uses the Django AuthenticationForm, which by default takes in username and password fields and then authenticates the user. Then after authenticating, it will check user type and redirect them if they have valid access.



Invitation code system:

Managers use the “Manage_invites.html” page to generate carehome codes. When creating a code, they submit the form which takes in how many days they want the code to be valid for: *Manage_invitation_codes view in views.py*

```
if request.method == 'POST':
    if 'generate_code' in request.POST:
        form = GenerateInvitationCodeForm(request.POST)
        if form.is_valid():
            invitation = form.generate_code(carehome, request.user)
            messages.success(request, f"Invitation code {invitation.code} generated successfully!")
            return redirect('manage_invitation_codes')
```

The invitation code can be set at a maximum of a 30 day period, however by default its set at 7 days.

GenerateInvitationCodeForm from Forms.py

```
class GenerateInvitationCodeForm(forms.Form):
    expiry_days = forms.IntegerField(
        min_value=1,
        max_value=30,
        initial=7,
        label="Invitation valid for (days)",
```

Within the form is the method for code generation:

```
def generate_code(self, carehome, created_by):

    # Set expiry date
    expiry_days = self.cleaned_data['expiry_days']
    expires_at = timezone.now() + timedelta(days=expiry_days)

    # Generate code
    code = InvitationCode.generate_unique_code()

    # Create invitation
    invitation = InvitationCode.objects.create(
        code=code,
        carehome=carehome,
        created_by=created_by,
        expires_at=expires_at
    )
    return invitation
```

Key steps:

Set Expiry Date:

1. The form collects input from the manager specifying how many days the code should remain valid.

2. The method calculates the expiry date by adding the specified number of days to the current date.

Generate unique code:

The method calls the `generate_unique_code` method, which is a method of the invitation code model. The unique code is generated using the random library, creating a 8-digit alphanumeric code. The database query:

```
if not cls.objects.filter(code=code).exists():  
    return code
```

Checks that the code just generated doesn't already exist in the database, if it doesn't the loop is exited and code returned, if not a new code is generated.

Found in class InvaitonCode in models.py

```
@classmethod  
def generate_unique_code(cls):  
    #generates a unique, 8 letter long alphanumeric code  
    import random  
    import string  
  
    while True:  
        # Generate an 8-character alphanumeric code  
        code = ''.join(random.choices(string.ascii_uppercase + string.digits, k=8))  
  
        # Makes sure code is unique and doesnt already exist.  
        if not cls.objects.filter(code=code).exists():  
            return code
```

Resident Management

Resident creation

Resident creation is handled through the `add_resident` function.

The function processes the Resident form and saves it to the carehome creating a record of the resident in the database. The `form.is_valid()` check makes sure all required information is provided.

Form processing section of the `add_resident` function in `views.py`

```
if request.method == 'POST':
    form = ResidentForm(request.POST, request.FILES, carehome=carehome)
    if form.is_valid():
        resident = form.save(commit=False)
        resident.carehome = carehome
        resident.save()
        messages.success(request, 'Resident added successfully.')
        return redirect('manager_dashboard_residents')
    else:
        form = ResidentForm(carehome=carehome)
```

The resident form is designed to take in all the residents information.

```
class ResidentForm(forms.ModelForm):
    class Meta:
        model = Resident
        fields = ['name', 'photo', 'date_of_birth', 'conditions', 'allergies', 'room_number', 'group']
        widgets = {
            'date_of_birth': forms.DateInput(attrs={'type': 'date'}),
            'conditions': forms.Textarea(attrs={'rows': 3, 'placeholder': 'List any medical conditions'}),
            'allergies': forms.Textarea(attrs={'rows': 3, 'placeholder': 'List any allergies'}),
            'room_number': forms.TextInput(attrs={'placeholder': 'Enter room number'}),
        }
```

The form includes within its `__init__` two key aspects:

First it defines the optional fields, resident's might not have any allergies and so these fields are optional. This means the form will validate even if these fields are empty.

```
self.fields['allergies'].required = False
```

The form also filters the available groups within the carehome:

```
self.fields['group'].queryset = ResidentGroup.objects.filter(carehome=carehome)
```

Restricts the dropdown options to only groups belonging to the carehome

```
self.fields['group'].label_from_instance = lambda obj: obj.name
```

This controls how each option appears, the lambda function ensures only group names are displayed.

The residents model includes a `Date_of_birth` field which stores their data of birth. From this, the model implements a method called `Calculate_age` to dynamically calculate the residents age.

```
@property
def calculate_age(self):
    if self.date_of_birth:
        today = date.today()
        return today.year - self.date_of_birth.year - ((today.month, today.day) <
```

```
(self.date_of_birth.month, self.date_of_birth.day))  
    return None
```

The age calculation first calculates the difference in years between the current data and date of birth.

It then subtracts 1 if the birthday is yet to occur in the year yet.

This is through the expression:

```
((today.month, today.day) < (self.date_of_birth.month, self.date_of_birth.day))
```

Editing a resident

A resident is edited using the edit_resident view. It utilises the same form used to create a resident.

The function has the decorator `@carer_permission_required('can_edit_residents')` which ensures only authorised carers can edit residents data.

```
def edit_resident(request, resident_id):
```

The function is passed in an existing residents ID from a record in the database.

```
resident = get_object_or_404(Resident, id=resident_id, carehome=carehome)
```

This performs a secure database query to retrieve a resident using their id.

The equivalent in SQL would be:

```
SELECT * FROM MedTracker_Resident  
WHERE (MedTracker_Resident.id = {resident_id} AND  
MedTracker_Resident.carehome_id = {carehome.id})
```

If this query returns no rows, an exception is raised.

The form is passed the resident instance that the database returns and the form is processed using the POST method.

```
form = ResidentForm(request.POST, request.FILES, instance=resident, carehome=carehome)
```

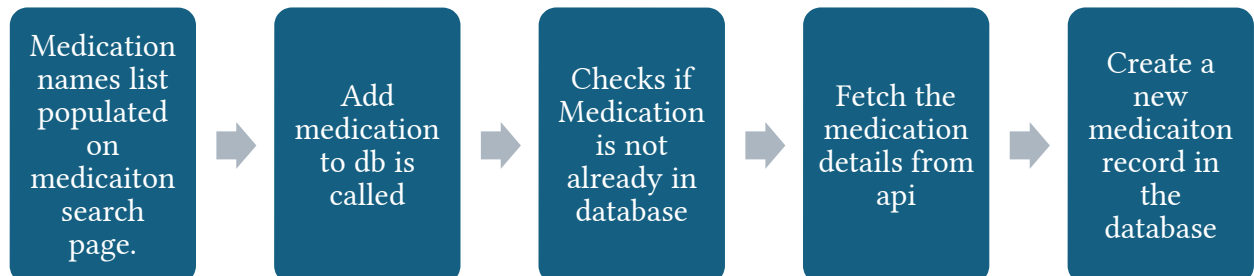
If the form is valid, the edits made to a resident are saved to the database.

Medicaition management

Medication inventory

Adding a medication

The execution steps for adding medication to the carehome inventory can be defined as such:



Search Medciation

The process starts with the user search for medication. This is done using the search medication view. The search for medication is populated using the NHS medication A-Z API. The function first calls the API and takes in the JSON response as its data set
The API call algorithm using requests library, found in get_medicines.py

```
def get_medicine(category):
    try:
        headers = { "subscription-key": API_KEY, "Accept": "application/json" }
        params = {
            "category": f"{category}",
            "page": 1,
            "per-page": 100
        }

        response = requests.get(API_URL, headers=headers, params=params)
        response.raise_for_status()

        return response.json()
    except requests.RequestException as e:
        return f"Error fetching medicines: {e}"
```

The API call uses categorisation. The API dataset is sorted into alphabetical sets based on the first letter of the medication name. Users can select which category they want to search. If the API server is not accessible, the error is handled using an exception.

```
def get_medicine_names(data):
    medicines = []
    for item in data.get('significantLink', []):
        if (item.get('mainEntityOfPage', {}).get('genre', []))
            and 'Medicine' in item['mainEntityOfPage']['genre']:
            url = item.get('url', "")
            med_id = url.split('/')[-2] if url else None

            if med_id:
                medicines.append({
                    'name': item['name'],
                    'id': med_id
                })
```

```
return medicines
```

The `get_medicine_names` utility filters the JSON response data and returns a list of the medicine names.

Example input data, Json response snippet

```
{
  "@type": "LinkRole",
  "name": "Adalimumab",
  "articleStatus": "published",
  "linkRelationship": "Result",
  "url": "https://api.service.nhs.uk/nhs-website-content/medicines/adalimumab/",
  "description": "Brand names: Humira, Amgevita, Hyrimoz, Idacio, Imraldi, Yuflyma. Find out how adalimumab treats swelling (inflammation), and how to take it.",
  "mainEntityOfPage": {
    "genre": ["Medicine"],
    "lastReviewed": null,
    "dateModified": "2023-03-15T14:52:10+00:00",
    "@type": "MedicalWebPage"
  }
}
```

The relevant section of the JSON response in the ‘significantlink’ array, which contains the details of each medicine. To find and store the medication data, a linear search with filtering is used.

The for loop iterates through each of the SignificantLink arrays, filtering the item’s MainEntityOfPage array and verifying the genre is “Medicine”.

```
if (item.get('mainEntityOfPage', {}).get('genre', []))
  and 'Medicine' in item['mainEntityOfPage']['genre']:
```

If True, then the algorithm extracts the ID used in the URL endpoint of the Medicines A-Z list:

```
url = item.get('url', "")
med_id = url.split('/')[2] if url else None
```

Therefore, a medication like “Aspirin”

Found at URL: <https://www.nhs.uk/medicines/aspirin-for-pain-relief/>

Will generate the `med_id`: aspirin-for-pain-relief, and will be stored within the medication record.

Adding the medication

When the user clicks “add to database” on the medication search frontend, it sends a POST request to the `add_medication_to_db` view. The view:

- Retrieves the `nhs_id` from the POST request
- Checks the Users authorisation
- Checks the database for if the medication already exists

```
if Medication.objects.filter(nhs_id=nhs_id, carehome=carehome).exists():
    messages.warning(request, "This medication already exists in your carehome's database.")
    return redirect('search_medications', 'A')
```

SQL query check for existing medication (medication.objects.filter)

```
SELECT *
FROM MedTracker_medication
```



```
WHERE nhs_id = '<nhs_id>'
AND carehome_id = <carehome_id>;
```

- Fetches the medication data from the API (runs the get_medicine api function)
Parses the correct category that the medication originates from
data = get_medicine(first_letter)

- Creates medication in database, getting the medication name from the API
SQL statement inserting the medication into the database

```
INSERT INTO MedTracker_medication (name, nhs_id, carehome_id)
VALUES ('<medication_name>', '<nhs_id>', <carehome_id>);
```

- Redirect back to medication inventory

Creating a Medication variant

The program treats a MedicationInventoryItem as a different variant of each medication a carehome stocks.

The form defines the different fields that are stored about each inventory item:

```
class MedicationInventoryItemForm(forms.ModelForm):
    class Meta:
        model = MedicationInventoryItem
        fields = ['dosage', 'form', 'stock', 'reorder_level']
```

Fields:

dosage: The dosage of the medication (e.g., 500mg).

form: The form of the medication (e.g. a tablet)

stock: The initial stock level.

reorder_level: The stock level at which low stock alert is triggered (managed by a method within the InventoryItem model)

```
def is_low_stock(self):
    return self.stock <= self.reorder_level
```

)

The add_inventory_item view takes in a medication_id as a argument. This is parsed when a user clicks on the add variant button on the frontend.

The view retrieves the medication medication = get_object_or_404(Medication, id=medication_id) from the corresponding medication in this database.

```
(SELECT * FROM MedTracker_medication WHERE id = <medication_id>)
```

The form data is then processed and the medication is inserted into the database.

```
if form.is_valid():
    inventory_item = form.save(commit=False)
    inventory_item.medication = medication
    inventory_item.save()
    messages.success(request, f"Variant for {medication.name} added successfully.")
    return redirect('medication_inventory')
```

Example SQL statement of a medication variant being stored in the database.

```
INSERT INTO MedTracker_medicationinventoryitem (medication_id, dosage, form, stock, reorder_level)
VALUES (1, '500mg', 'tablet', 100, 10);
```

Medication Administration

Creating a schedule

Creating a medication schedule starts with the `add_medication_schedule` view.

This view takes in a `resident_id`, rendering and validating the `ScheduleForm`

Steps:

- The `resident_id` is used to fetch the resident from the database
- If the request is a POST request, the form data received is validated and a `MedicationSchedule` record is created in the `Schedule` table.

```
if request.method == 'POST':
    form = MedicationScheduleForm(request.POST, carehome=carehome)
    if form.is_valid():
        schedule = form.save(commit=False)
        schedule.resident = resident
        schedule.save()
```

- The `times []` from the POST data is used to create records in the `MedicationTime` table.

```
# Process time slots
times = request.POST.getlist('times[]')
for time_str in times:
    if time_str:
        try:
            time_obj = datetime.strptime(time_str, '%H:%M').time()
            MedicationTime.objects.create(
                schedule=schedule,
                time=time_obj
            )
        except ValueError:
            pass # Skip invalid time values
```

The `times` array contains a list of strings of the time inputs by the user (e.g. 8:00, 12:00)

The initial iteration goes through each time string in the array.

If the time is invalid it is skipped, if it's a valid time then it's converted into a time object in the format Hour Minute representing 24 hour clock.

A medication time record is then stored in the database, assigning the schedule as a foreign key and the time instance.

Medication Schedule Form

The form defines the fields for a valid form.

```
class Meta:
    model = MedicationSchedule
    fields = ['medication', 'inventory_item', 'frequency', 'route']
    widgets = {
        'frequency': forms.NumberInput(attrs={'min': 1, 'max': 6}),
        'route': forms.Select(attrs={'class': 'form-control'})
    }
```

The form is able to dynamically update its fields based on the selected medication. This is implemented in the `__init__` section of the form.

```
# Filter medications by care home
if carehome:
    self.fields['medication'].queryset = Medication.objects.filter(carehome=carehome)

# Start with an empty inventory item queryset
self.fields['inventory_item'].queryset = MedicationInventoryItem.objects.none()

# Populate inventory items if a medication is selected
if self.is_bound and 'medication' in self.data:
    try:
        medication_id = int(self.data.get('medication'))
        self.fields['inventory_item'].queryset = MedicationInventoryItem.objects.filter(
            medication_id=medication_id
        )
```

This dynamically queries the database for all the medication assigned to a carehome, and then populates the variants dropdown with the stored inventoryitems of that medication.

Recording medication administration

The carer dashboard provides carers with a upcoming list of medications for residents in their assigned group. This list dynamically generates based on the:

- Medication schedules for residents in their assigned group
- The associated time slots in MedicationTime for each schedule

The dashboard displays these medications, sorted by urgency and displays:

- Resident name
- Medication
- The scheduled time
- A form to submit it as “given” and provide notes.

Displaying the medication

Time urgency is calculated using the `get_upcoming_medications` function

It takes in a carehome as the input and outputs a list of dictionaries, each representing an upcoming medication including the schedule, timeslot, urgency and if it has or hasn't been given.

Execution steps:

```
now = timezone.now()
```

This gets the current server time and sets it in the Now variable.

```
upcoming_meds = []
```

a empty meds list is created, then all the residents associated with the carehome are retrieved (`residents = carehome.residents.all()`)

The schedules are fetched with this algorithm.

```
schedules = MedicationSchedule.objects.filter(
    resident__in=residents,
    is_active=True
```

```

).select_related(
    'resident',
    'medication'
).prefetch_related('times')

```

The `select_related` method optimises the database queries by preloading the related resident and medication records. The `prefetch_related` preloads the related times for each schedule.

Each schedule is iterated over, and for each schedule its associated time slots are processed.

```

for schedule in schedules:
    for time_slot in schedule.times.all():
        med_time = timezone.make_aware(
            datetime.combine(now.date(), time_slot.time)
        )
        # time difference between when medication is due and the current time
        time_diff = med_time - now

        # If medication is more than 12 hours overdue, skip
        if time_diff.total_seconds() < -43 200: # 12 hours in seconds
            continue

        # Check if medication was already given today by checking the log
        was_given = MedicationLog.objects.filter(
            schedule=schedule,
            scheduled_time=time_slot,
            given_at__date=now.date()
        ).exists()

        # Calculate urgency level
        urgency = calculate_urgency(was_given, time_diff)

        upcoming_meds.append({
            'schedule': schedule,
            'time_slot': time_slot,
            'planned_time': med_time,
            'time_diff': time_diff,
            'urgency': urgency,
            'was_given': was_given
        })

```

The `med_time` variable calculates the scheduled time for the medication using the current date and the time from the medication time record. This ensures that the scheduled time is date-specific and timezone-aware.

Example data:

If the `now` variable was: 2023-10-15 14:30 00+00:00 (date and time in UTC)

`Time_slot.time`: 8:00:00 (recorded time medication was scheduled)

The final output would be:

Medtime = datetime(2023, 10, 15, 8, 0, 0, tzinfo=<UTC> (This is a timezone aware datetime record)

Medications that are overdue by more than 12 hours are skipped to avoid cluttering up the dashboard.

A check is done, querying the medicationlog table to check if the medication for the given schedule and time slot hasn't already been administered:

```
was_given = MedicationLog.objects.filter(
    schedule=schedule,
    scheduled_time=time_slot,
    given_at__date=now.date()
).exists()
```

Urgency is calculated using the calculate urgency function:

```
def calculate_urgency(was_given, time_diff):
    if was_given:
        return 'given'
    thresholds = [
        (0, 'overdue'),
        (1800, 'urgent'), # 30 minutes
        (3600, 'warning') # 1 hour
    ]
    for limit, label in thresholds:
        if time_diff.total_seconds() < limit:
            return label
    return 'normal'
```

This sets parameters, time_diff is a timedelta representing the difference between the scheduled time and the current time, is parsed from the variable:

```
time_diff = med_time - now
```

The thresholds for urgency are also defined, with now meaning the medication is now due/overdue, 30 minutes warning the user and letting it know it's a priority and a 1 hour mark giving a different colour code letting the user be aware of the medication being required in the near future.

The function iterates through the thresholds and if the time_diff (in seconds) is less than a threshold, the corresponding urgency label is returned.

And if no threshold is met, the medication is returned as "normal".

Once the medication urgency has been calculated, its details are appended to the upcoming meds array

```
upcoming_meds.append({
    'schedule': schedule,
    'time_slot': time_slot,
    'planned_time': med_time,
    'time_diff': time_diff,
    'urgency': urgency,
```

```
'was_given': was_given
})
```

It is stored as a dictionary, including the schedule, timeslot, planned time (scheduled time), time difference between current and scheduled time, urgency level, and a flag for if the medication has been administered.

```
upcoming_meds.merge_sort_upcoming_meds(upcoming_meds)
```

This calls a sort of the scheduled medication.

The sort is done using a merge sort algorithm, which has the time complexity of $O(n \log n)$. It works by dividing the list into smaller sublists, until each only contain one element, then the sublists are merged together into one sorted list.

It uses recursive division

```
if len(upcoming_meds) <= 1:
```

```
    return upcoming_meds
```

If the list has one or no elements, its already sorted, if not the list is split into two halves. The midpoint is calculated and the list is divided into the left_half and right_half. The function is called recursively on each half until sublists contain only one element.

```
mid = len(upcoming_meds) // 2
```

```
left_half = merge_sort_upcoming_meds(upcoming_meds[:mid])
```

```
right_half = merge_sort_upcoming_meds(upcoming_meds[mid:])
```

Once sublists are reduced to single elements, the merge function is called to combine them back into a sorted list.

```
while left and right:
```

```
    if left[0]['planned_time'] <= right[0]['planned_time']:
```

```
        sorted_list.append(left.pop(0))
```

```
    else:
```

```
        sorted_list.append(right.pop(0))
```

The first element of each sublist is compared based on the planned time key. The smaller element is removed from the sublist and added to the sorted list.

Dashboard updates

Dashboard updates are handled by the get_dashboard_updates view.

```
@login_required
```

```
def get_dashboard_updates(request):
```

```
    carehome = get_user_carehome(request.user)
```

```
    upcoming_meds = get_upcoming_medications(carehome)
```

```
    html = render_to_string('includes/medication_list.html', {'upcoming_meds': upcoming_meds})
```

```
    current_time = timezone.now().strftime('%H:%M:%S')
```

```
    return JsonResponse({'html': html, 'current_time': current_time})
```

This view fetches the latest list of upcoming medications and returns it as a json response.

It calls the get_upcoming_medications function and renders the medication list using the render_to_string function. It renders the template with the upcoming_meds as context, it generates the html for the updated medication list to be sent to the frontend.

The server time is updated and included in the response, to be displayed on the carers dashboard.

Marking the medication as 'Given'

The function first fetches the medication schedule and time slot associated with the POST request just sent from the user Client:

```
schedule = get_object_or_404(MedicationSchedule, id=schedule_id)
```

```
time_slot = get_object_or_404(MedicationTime, id=time_slot_id)
```

If either is invalid, an error is raised.

The program verifies the Carers access to the resident first, checking if they are associated with the resident.

```
if not request.user.carer.resident_group or \
    schedule.resident not in request.user.carer.resident_group.residents.all():
    messages.error(request, "Unauthorized access")
    return redirect('carer_dashboard')
```

Checks associated resident group and if the resident of the schedule is associated with the carer.

A medication log of the administration is recorded in the database:

```
MedicationLog.objects.create(
    schedule=schedule,
    scheduled_time=time_slot,
    given_by=request.user.carer,
    notes=notes
)
```

The inventory is updated, displaying that a medication has been used and decreasing the stock:

```
inventory_item = schedule.inventory_item
```

```
if inventory_item.stock > 0:
```

```
    inventory_item.stock -= 1
```

```
    inventory_item.save()
```

and user is provided feedback in the form of a success message, in addition to the page refreshing to display the medication as newly given.

```
messages.success(request, f"Medication recorded for {schedule.resident.name}")
```

```
return redirect('carer_dashboard')
```

Medical Administration Record (MAR chart) generation

The process begins with the `generate_mar_chart` view in `views.py`

The steps involved:



The PDF creation occurs in the `MARGenerator` class in `mar_generator.py`. This uses the `reportlab` library.

```
class MARGenerator:
    def __init__(self, resident, start_date):
        self.resident = resident
        self.start_date = start_date
        self.styles = getSampleStyleSheet()

        self.styles.add(ParagraphStyle(
            'Wrapped',
            parent=self.styles['Normal'],
            wordWrap='CJK'
        ))
```

This configures the file to be a PDF, and stores the resident and start date for the chart.

The PDF file format means that the document cannot be as easily tampered or edited, unlike a .doc file for example.

```
def generate_mar_chart(self, output_path):
    doc = SimpleDocTemplate(
        output_path,
        pagesize=A4,
        rightMargin=30,
        leftMargin=30,
        topMargin=30,
        bottomMargin=30
    )

    elements = []
    elements.extend(self._create_header())
    elements.extend(self._create_medication_tables())

    doc.build(elements)
```

1. The document is setup by creating a `SimpleDocTemplate` in a4 size.
2. The header element is created with is populated with resident details like name, DOB and allergies.
3. The medication table generates based off the residents medication schedules
4. The different elements are combined and the PDF is written to the buffer

Header creation

```
def _create_header(self):
    header_data = [
        ['Name:', self.resident.name, 'DOB:', self.resident.date_of_birth.strftime('%d/%m/%Y') if self.resident.date_of_birth
        else ""],
        ['Room:', self.resident.room_number or 'Not assigned', 'Care Home:', self.resident.carehome.name],
        ['Group:', self.resident.group.name if self.resident.group else 'Not assigned', 'Start Date:',
        self.start_date.strftime('%d/%m/%Y')],
        ['Allergies:', self.resident.allergies or 'None known', "", ""],
        ['Conditions:', self.resident.conditions or 'None known', "", ""],
    ]
    header_table = Table(header_data, colWidths=[1.2*inch, 2.8*inch, 1.2*inch, 2.2*inch])
    header_table.setStyle(TableStyle([...]))
    return [header_table, Paragraph("<br/><br/>", self.styles['Normal'])]
```

- Displays all the resident information in a table format.

Medication table creation

```
def _create_medication_tables(self):
    elements = []
    medications = self.resident.medicationschedule_set.all()

    for medication in medications:
        elements.extend(self._create_single_medication_table(medication))
        elements.append(Paragraph("<br/><br/>", self.styles['Normal']))

    return elements
```

- This iterates through all the medication schedules for a resident and generates a table for each of them

Single medication table creation

```
def _create_single_medication_table(self, medication_schedule):

    dates = [(self.start_date + timedelta(days=x)) for x in range(31)]
    admin_times = medication_schedule.times.all()

    # Split dates into weeks
    weeks = [dates[i:i + 7] for i in range(0, len(dates), 7)]

    for week_dates in weeks:
        table_data = [['Time'] + [d.strftime('%d/%m') for d in week_dates]]

        for time_slot in admin_times:
            row = [time_slot.time.strftime('%H:%M')]

            for date in week_dates:
                given = MedicationLog.objects.filter(
                    schedule=medication_schedule,
                    scheduled_time=time_slot,
                    given_at__date=date
                ).first()
```

```

if given:
    initials = f"{given.given_by.user.first_name[0]}{given.given_by.user.last_name[0]}.upper()"
    row.append(initials)

else:
    row.append("")
table_data.append(row)

admin_table = Table(table_data, colWidths=[1*inch] + [0.8*inch]*len(week_dates))
admin_table.setStyle(TableStyle([...]))
elements.append(admin_table)

```

Key steps:

- Creates rows for each administration time and columns for each day within the month
 - o The first column of each row displays the time (e.g. 8:00)
- Medication logs model is queried to check if medication was administered at that time and date
 - o If administered, the initials of the carer are added to the cell
 - o If not the cell is left blank

The role of the memory buffer:

```

# Create in-memory PDF
buffer = BytesIO()
MedicationLog.generate_mar_chart(resident, start_date, buffer)

# Set buffer's cursor at the beginning
buffer.seek(0)

# Return the PDF from memory
return FileResponse(
    buffer,
    as_attachment=True,
    filename=f'MAR_{resident.name}_{start_date.strftime("%B_%Y")}.pdf'
)

```

The initial call that occurs in views.py to generate a mar chart uses the bytesIO library to create a memory buffer in the server. This critically ensures that the MAR chart data that is generated is not stored on the servers filesystem.

The buffer holds the PDF data in memory only for the duration of the request. Once the response is sent to the client the buffer is discarded.

This minimises the risk of unauthorised access to sensitive resident data, meeting GDPR compliance.

Additionally, writing to memory is faster than writing to disk, slightly improving the serverside response time.

Permissions and Security:

The program uses multiple different systems in order to make sure only authorised users can access specific pages and complete operations they have permissions for.

Authentication layer

The first layer is basic authentication handled by Django's decorator `@login_required`. This is used on every dashboard view and ensures only authenticated (logged in) users can access these endpoints.

User role verification

The second layer is checking their role, if they are a Manager or Carer user.

```
if not hasattr(request.user, 'manager'):
    messages.error(request, "Unauthorized access")
    return redirect('carer_dashboard')
```

This is used in views like the Manager Carer dashboard, as only manager users should be able to access the management for care staff.

Carehome Association:

The `get_user_carehome` utility retrieves the current associated carehome and ensures that users only access the data of their associated carehome.

Get_user_carehome found in /Utils

```
carehome = get_user_carehome(request.user)
if not carehome:
    messages.error(request, "Unauthorized access")
    return redirect('/')
```

Carer Permissions:

Carers have permissions defined in the `CarerPermissions` model:

- `can_edit_residents` - Can modify resident information
- `can_edit_medications` - Can modify medication schedules
- `can_view_inventory` - Can view medication inventory
- `can_adjust_stock` - Can modify medication stock levels

These are enforced using the custom `@carer_permission_required` decorator:

This appears in functions and views like:

```
@login_required
@carer_permission_required('can_adjust_stock')
def adjust_inventory_stock(request, inventory_item_id):
```

When someone tries to access this view, or in this example execute the function, the decorator receives the permission name 'can_adjust_stock' as an argument, it creates a `_wrapped_view` function which executes first.

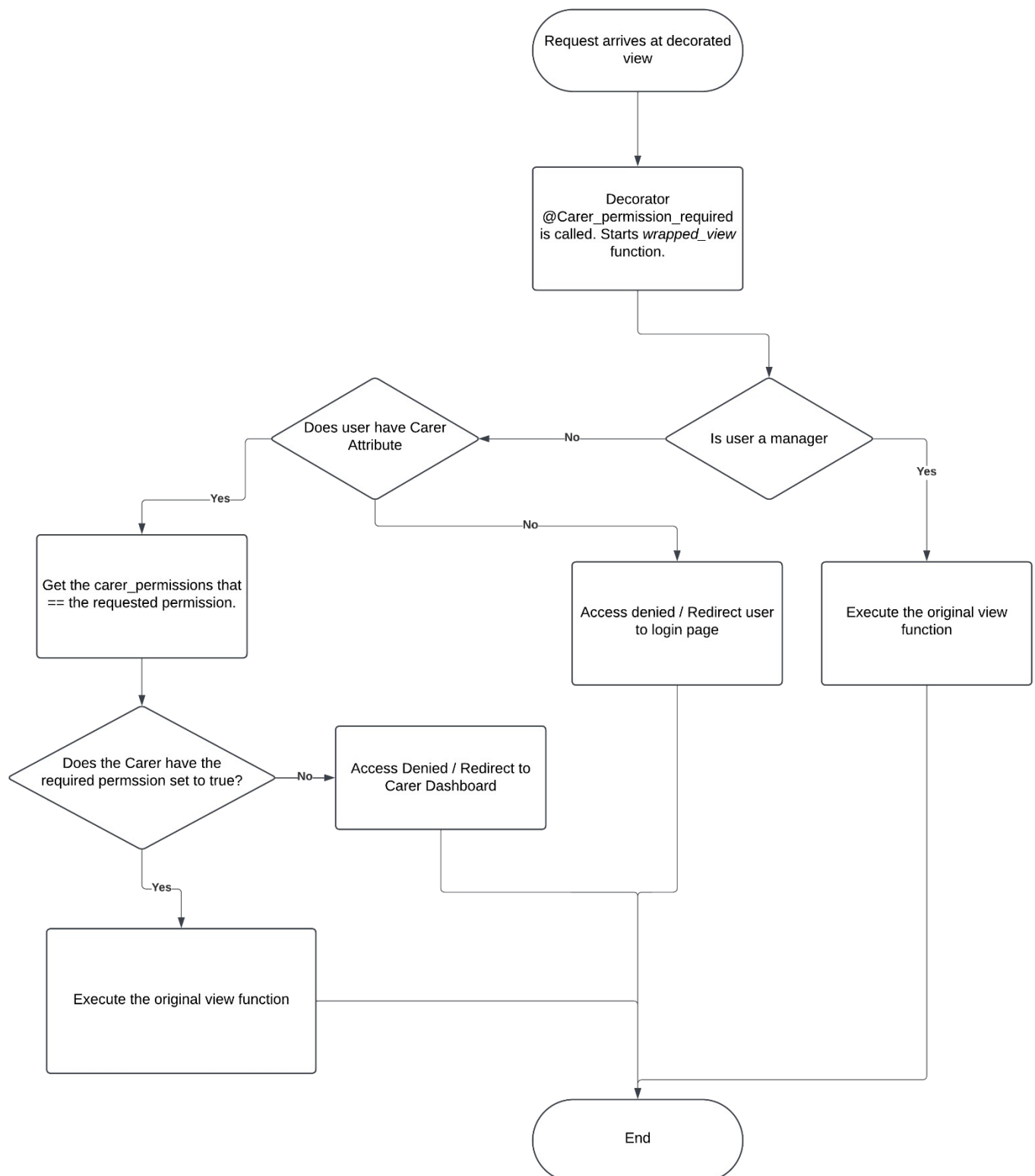
Inside the `_wrapped_view`, the decorator performs the checks:

- Manager Override
- Carer Authentication Check
- Permission specific check
- Allowing access if the checks clear

In action, the behaviour of the decorator is:
It accesses the carer's permissions relation

Uses `getattr()` to dynamically check the specified permission attribute
(like `can_edit_residents`)
If the permission is missing, shows an error and redirects to the carer dashboard

Flowchart of Carer permission decorator:



Logout system

The `logout_view` found in `views.py`

```
def logout_view(request):  
    logout(request)  
    return redirect('/')
```

This function calls the built-in Django logout function and then redirects users to the landing page.

Djangos `logout()` function performs several security operations. It clears the session key and any session data that's stored on the device's cache, any cookies, which are used for automatically logging the user in, are automatically expired and the user data of the user who clicked log-out is cleared from the request.

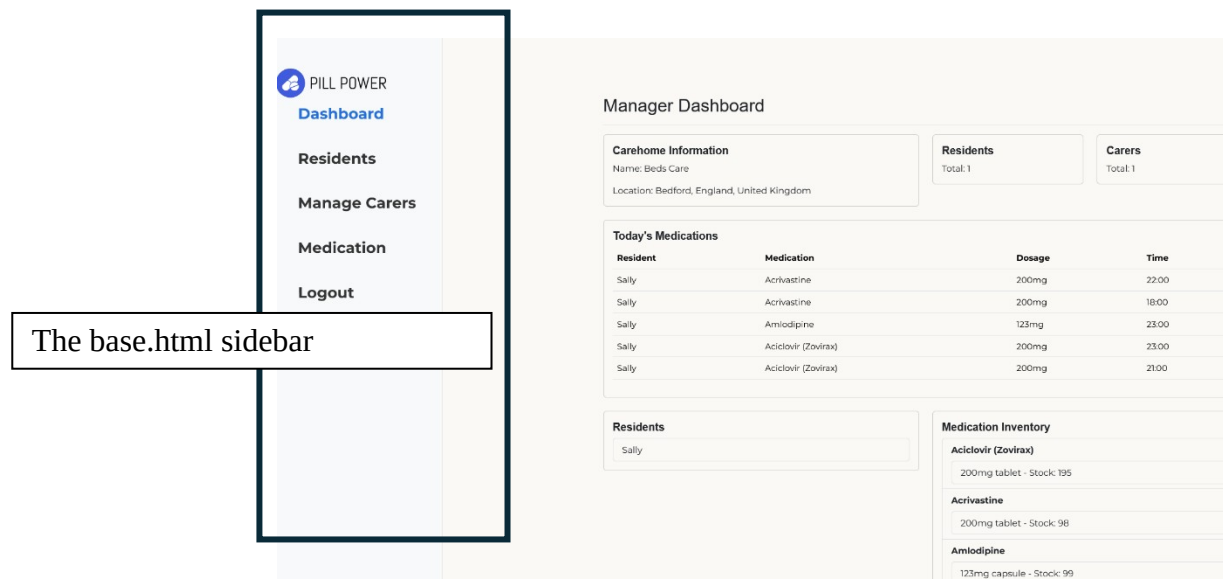
The details of how Django implements this is:

```
def logout(request):  
  
    session_key = request.session.session_key  
  
    request.session.flush()  
  
    if session_key and settings.SESSION_COOKIE_NAME in request.COOKIES:  
        response.delete_cookie(settings.SESSION_COOKIE_NAME)  
  
    if hasattr(request, 'user'):  
        request.user = AnonymousUser()
```

Frontend Layer

In this project, as a client-server system the user interacts with the program solely through web interfaces. Jinja has been used to dynamically render HTML content on the server side and sent it to the client browser. This allows python logic to be embedded within the HTML and parse data to and from the user to the server.

Base.html



The base.html file serves as a template base for almost all of the dashboard templates. It follows the DRY (don't repeat yourself) approach and reduces the need for duplication of code.

It defines the main content area that child templates override to insert page specific content.

```
{% block content %}
```

```
<!-- Child templates will override this block -->
```

```
{% endblock %}
```

It also defines the bootstrap CSS and javascript styling that all the project UI is based on. Other templates extend the base.html using `{% extends 'base.html' %}`. This allows the pages to inherit the sidebar navigation and styling.

Form rendering:

Jinja plays a crucial role in the rendering of the dynamic forms on the frontend. Jinja uses the `{% csrf_token %}` tag, which is a key part of the form design and included in all forms that the user interacts with in order to ensure secure form submissions. This significantly reduces cross-site forgery requests and helps with GDPR data safety compliance.

```
<form method="POST">
```

```
{% csrf_token %}
```

```
<!-- Form fields -->
```

```
</form>
```

An example of Jinja handling form rendering is in the sign-up form. The page handles two different registration forms through the provided context from the sign-up view.

```
{{ form1.field }}
```

Defines a form input box for a field from the manager form.

`{{ form2.field }}`

Defines a form input box for a field from the carer form.

The image shows a registration form for 'Pill Power'. At the top, there are two tabs: 'Manager' and 'Carer'. The form contains the following fields:

- First Name:** A text input field with the placeholder 'Enter your first name'. A callout points to this field with the code `{{ form1.first_name }}`.
- Last Name:** A text input field with the placeholder 'Enter your last name'.
- Email Address:** A text input field with the placeholder 'Enter your email address'.
- Password:** A text input field with the placeholder 'Create a password'. A callout points to this field with the code `{{ form1.password1 }}`. Below this field is a note: 'Your password must be 8+ characters and contain letters and numbers.'
- Confirm Password:** A text input field with the placeholder 'Confirm your password'.
- Submit Button:** A blue button labeled 'Register as Manager'. A callout points to this button with the text 'Button with type = "Submit"'. Another callout points to the entire form with the text 'Entire html form element.'

Figure 18: The registration form, an example of the Django template being rendered with jinja

Error handling:

```
{% if messages %}
<div class="messages z-3">
  {% for message in messages %}
    <div class="alert alert-{{ message.tags|default:'danger' }}">
      {{ message }}
    </div>
  {% endfor %}
</div>
{% endif %}
```

Using an if statement inside templates, dynamic error messages are displayed using jinja. The for loop iterates through error messages, if there are multiple errors made in a instance. The `{{ Message.tags }}` allows for a custom error message to be displayed from data parsed from the backend.

Example of a rendered error message:

The image shows a screenshot of the 'Pill Power' website. At the top, there is a logo with two pills and the text 'PILL POWER'. Below the logo, there is a message box with the text 'An account with that email already exists.' and a close button (X). At the bottom, there are two tabs: 'Manager' and 'Carer'.

Role of jinja in dashboards:

The data that the user interacts with is rendered through jinja and the context provided by the backend view models.

Jinja is able to dynamically render lists, like in the Residents dashboard:

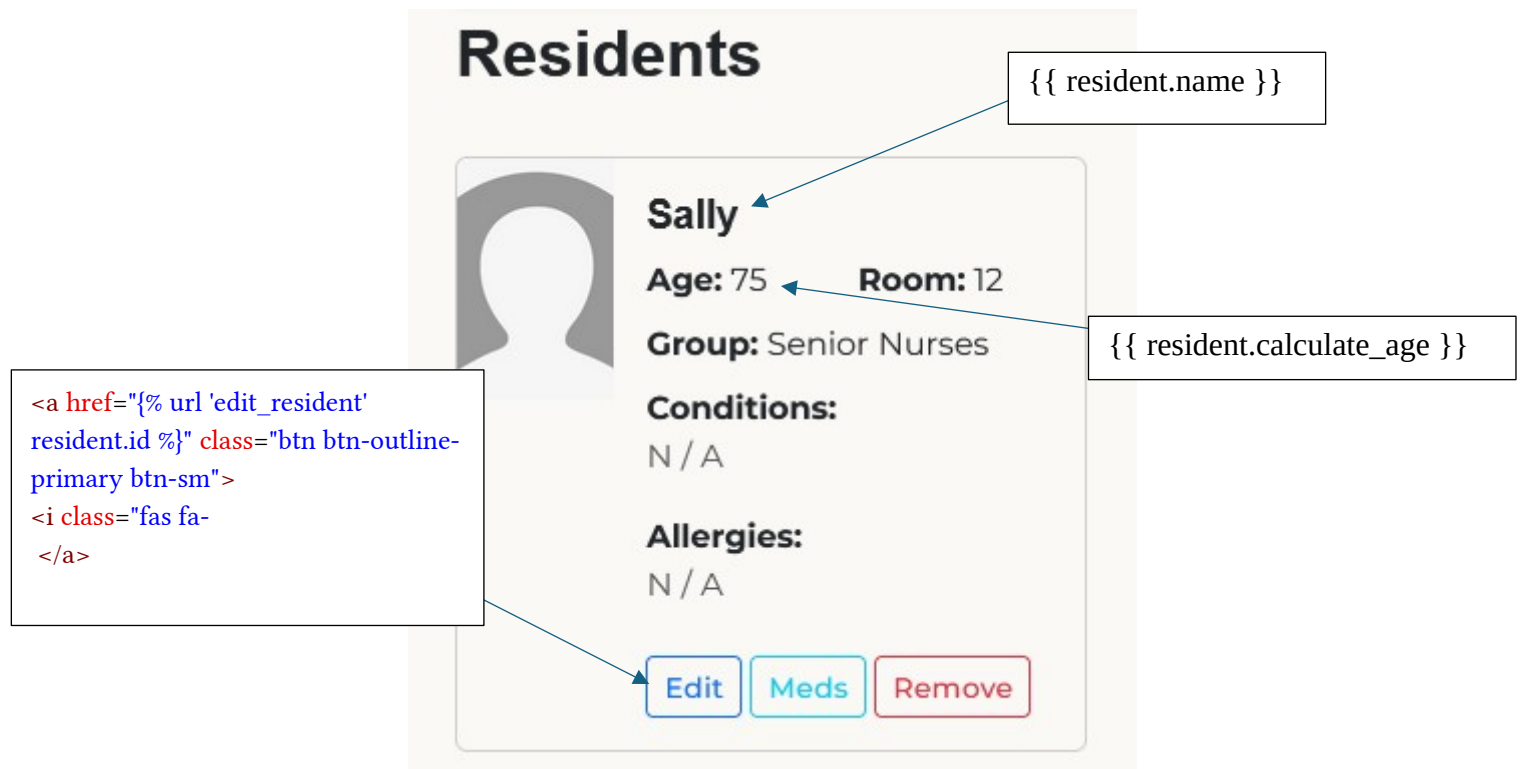
```
<div class="row g-3">
  {% for resident in residents %}
    <div class="col-md-6">
      <div class="card h-100">
        <div class="card-body">
          <h5 class="card-title">{{ resident.name }}</h5>
          <p class="card-text">
            <strong>Age:</strong> {{ resident.calculate_age }}<br>
            <strong>Room:</strong> {{ resident.room_number|default:"N/A" }}
          </p>
          <a href="{% url 'edit_resident' resident.id %}" class="btn btn-outline-primary btn-sm">Edit</a>
        </div>
      </div>
    </div>
  {% endfor %}
</div>
```

Here:

{% for resident in residents %} loops through the list of residents parsed from the backend

{{ resident.name }} displays the resident name

{% url 'edit_resident' resident.id %} generates a URL for editing the resident, sent to the backend as a POST request.



a screenshot from the residents dashboard, displaying a carehome resident on the frontend

Within the Residents page, Javascript also plays a role in rendering the colour preview:

Add New Group

Name

Senior Nurses

Colour

Preview

Add Group

```
document.addEventListener('DOMContentLoaded', function() {
  const colorInput = document.querySelector('input[type="color"]');
  const preview = document.querySelector('.color-preview');

  colorInput.addEventListener('input', function(e) {
    preview.style.backgroundColor = e.target.value;
  });
});
```

```
});  
});
```

```
const colorInput = document.querySelector('input[type="color"]');
```

This refers to the <input> element with the type 'color', referring to the colour input field that the user uses to select a colour.

The ColourInput.AddEventListener waits for when a colour input event is triggered and updates the preview element based on the e.target.value which it gets in hexadecimal format.

Jinja is also used in Inline actions, such as confirmation dialogs. When deleting a resident in the resident dashboard:

```
onsubmit="return confirm('Are you sure you want to remove this resident?');">
```

This adds a confirmation dialog before the form.

```
<input type="hidden" name="resident_id" value="{{ resident.id }}">  
<button type="submit" name="remove_resident" class="btn btn-outline-danger btn-sm">  
  <i class="fas fa-trash"></i> Remove  
</button>
```

Medication search rendering

The search boxes in the medication search are dynamically generated using Jinja. These boxes display the API data as a result of a medication search.

```
<div id="results">  
  {% if results %}  
    <div class="row">  
      {% for med in results %}  
        <div class="col-md-6 mb-3">  
          <div class="card">  
            <div class="card-body">  
              <h5 class="card-title">  
                {{ med.name }}  
              </h5>  
              <form method="POST" action="{% url 'add_medication_to_db' %}">  
                {% csrf_token %}  
                <input type="hidden" name="nhs_id" value="{{ med.id }}">  
                <button type="submit" class="btn btn-success">Add to Database</button>  
              </form>  
            </div>  
          </div>  
        </div>  
      {% endfor %}  
    </div>  
    {% elif search_term %}  
      <p>No medications found starting with "{{ search_term }}"</p>  
    {% endif %}  
  </div>
```

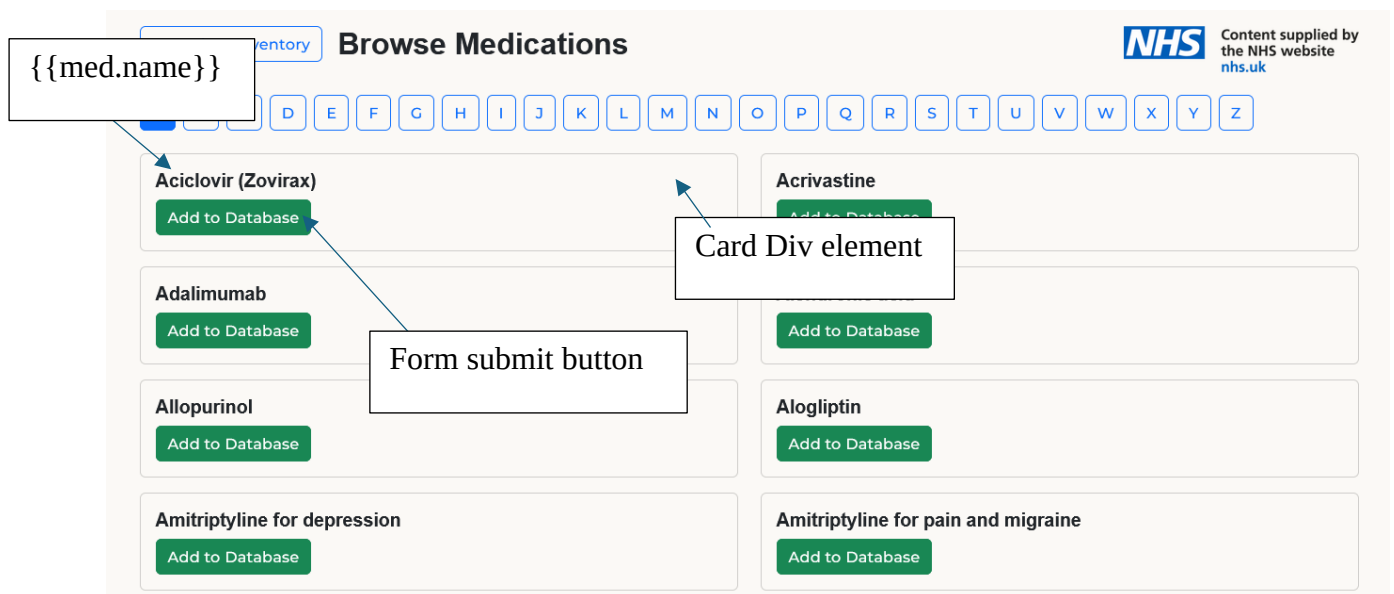
Jinja loops through the list of medications (results) passed from the backend and renders each as a card element.

{{ med.name }} this displays the medication name

`<input type="hidden" name="nhs_id" value="{{ med.id }}">`: This embeds the NHS id of the medication has a hidden form input field. When 'add to db' is pressed, the POST request is sent to the server with this data.

Conditional rendering also means that if there are no search results, errors are not thrown, instead it shows no medication found:

```
{% if results %}
    <!-- Render search results -->
{% elif search_term %}
    <p>No medications found starting with "{{ search_term }}"</p>
{% endif %}
```



Carer Dashboard

Displaying the upcoming medications within the carer dashboard works through a Jinja iteration loop and uses the parsed context from the backend to render.

The `/includes/medication_List.html` is the main template used to render the medication:

```
<ul>
    {% for med in upcoming_meds %}
        <li class="urgency-{{ med.urgency }}">
            {{ med.schedule.medication.name }} - {{ med.planned_time|date:"H:i" }}
            {% if med.was_given %}
                <span class="badge bg-success">Given</span>
            {% else %}
                <form method="post" action="{% url 'record_medication' med.schedule.id med.time_slot.id %}">
                    {% csrf_token %}
                    <button type="submit" class="btn btn-primary btn-sm">Mark as Given</button>
                </form>
            {% endif %}
        </li>
    {% endfor %}
</ul>
```

The *for med in upcoming meds* loops through the list of medications from the backend, the `med.schedule.medication.name` dynamically displays the individual; names of each medication, if `med.was_given` is a conditional render, dynamically rendering the form based on if the medication has been given.

```
<li class="urgency-{{ med.urgency }}">
```

Jinja is also used in the CSS classes, rendering the urgency of the medications based on the calculated urgency.

This is all displayed in the Carer_dashboard through:

```
<div class="medication-list" id="medication-list">
    {% include 'includes/medication_list.html' %}
</div>
```

There is also Javascript present in the Frontend of the Carer dashboard, this is used to fetch real-time updates without the need of page reloading:

```
document.addEventListener('DOMContentLoaded', function() {
    function updateDashboard() {
        fetch('{% url "get_dashboard_updates" %}')
            .then(response => response.json())
            .then(data => {
                if (data.error) {
                    console.error(data.error);
                    return;
                }
                // Update the medication list
                document.querySelector('#medication-list').innerHTML = data.html;
                // Update the current time
                document.querySelector('#current-time').textContent = data.current_time;
            })
            .catch(error => console.error('Error fetching dashboard updates:', error));
    }

    // Refresh the dashboard every 30 seconds
    setInterval(updateDashboard, 30000);
});
```

The `UpdateDashboard` function is used to call a json response from the backend and parse it to the template to render using Jinja.

```
Fetch{%url "get_dashboard_updates"%}'
```

This sends an AJAX request to the backend to fetch updated medication data. The advantage of AJAX is that the web page can update the medication list without needing to fully reload the page. The `Data.html` contains that updated data response.

The `setInterval(updateDashboard, 30000)` sets a automatic update interval to send new data to the carer dashboard and maintains that users are not seeing out of date information. The script also includes a update call using a listener which listens for a “submit”, this is when a medication is marked as given, updating the dashboard without a page reload.

```
document.addEventListener('submit', function(e) {
```

The screenshot shows a web application for medication management. At the top, a header bar displays 'Your Group: Senior Nurses' and 'Residents: 1'. Below this is a sidebar with 'Actions' and 'Logout' buttons. The main content area lists medication schedules for a resident named 'Sally'. Each entry includes the medication name, dosage, and a 'Mark Given' button. Annotations with arrows point to various parts of the interface: 'Your Group: {{ carer.resident_group.name }}' points to the group name; 'Residents: {{ residents.count }}' points to the resident count; '{{ med.schedule.resident.name }}' points to the resident name in the first entry; '{{ med.schedule.medication.name }} - {{ med.schedule.dosage }}' points to the medication and dosage in the second entry; and 'Time based urgency' points to the time-based urgency indicator in the second entry.

Your Group: Senior Nurses
Residents: 1

Actions ▾

Logout

Sally 6:00 PM

{{ med.schedule.resident.name }}

Sally 9:00 PM
Aciclovir (Zovirax) - 200mg
Add notes (optional) Mark Given

{{ med.schedule.medication.name }} - {{ med.schedule.dosage }}

10:00 PM

Given at

Sally 11:00 PM
Amlodipine - 123mg
Given at

Time based urgency

Sally 11:00 PM
Aciclovir (Zovirax) - 200mg
Add notes (optional) Mark Given

Implementation

For this section, I have included the technical solution files that make up my Django project, some files have been excluded, such as the Django settings.py and redundant python/html files, as they are standardized Django configuration files that are automatically generated when creating a new project. These files contain default settings and basic project configuration that were not modified significantly from their original state and therefore do not represent my original programming work for this NEA project.

The included files focus on the custom code and components I developed to implement the core functionality of my medical tracking system. I have organised them as follows, which reflects their original file structure:

Contents

IMPLEMENTATION.....	72
<i>Contents</i>	72
<i>MedTracker/</i>	73
<i>/utils/</i>	73
<i>/Get_medication_inventory.py</i>	73
<i>/Get_medicine_names.py</i>	73
<i>/Get_medicines.py</i>	74
<i>/Get_upcoming_medications.py</i>	74
<i>/Get_user_carehome.py</i>	77
<i>/Mar_generator.py</i>	77
<i>/decoratorscustom.py</i>	81
<i>/Forms.py</i>	81
<i>/models.py</i>	90
<i>/urls.py</i>	95
<i>/views.py</i>	97
<i>Templates/</i>	120
<i>/Auth/</i>	120
<i>Businesssignup.html</i>	120
<i>login.html</i>	125
<i>/Dashboards/</i>	127
<i>carer_dashboard.html</i>	127
<i>manage_carer_access.html</i>	130
<i>manage_carer_permissions.html</i>	131
<i>manager_dashboard_carers.html</i>	132
<i>manager_dashboard_residents.html</i>	135
<i>manager_dashboard.html</i>	138
<i>setup_carehome.html</i>	142
<i>/Includes/</i>	143
<i>medication_list.html</i>	143
<i>/Manager/</i>	144
<i>manage_invites.html</i>	144
<i>/Medication/</i>	147
<i>add_inventory_item.html</i>	147
<i>add_medication_schedule.html</i>	149
<i>audit_log.html</i>	152
<i>medication_inventory.html</i>	153
<i>search_medications.html</i>	156
<i>/Resident/</i>	158
<i>add_resident.html</i>	158
<i>edit_resident.html</i>	160
<i>resident_medications.html</i>	160
<i>/Staticpages/</i>	162
<i>base.html</i>	162
<i>index.html</i>	165

MedTracker/

/utils/

/Get_medication_inventory.py

```
import json
from django.http import JsonResponse
from django.shortcuts import get_object_or_404
from django.contrib.auth.decorators import login_required
from MedTracker.models import Medication, MedicationInventoryItem
from MedTracker.utils.get_user_carehome import get_user_carehome

# This function retrieves the inventory items for a specific medication.
# It checks if the user has access to the carehome associated with the medication.
# It returns a JSON response with the inventory items.
@login_required
def get_medication_inventory_items(request, medication_id):
    medication = get_object_or_404(Medication, id=medication_id)
    carehome = get_user_carehome(request.user)

    # Make sure user has access to this medication
    if medication.carehome != carehome:
        return JsonResponse({'error': 'Unauthorized'}, status=403)

    # Get all inventory items for this medication
    inventory_items = MedicationInventoryItem.objects.filter(
        medication_id=medication_id
    ).values('id', 'dosage', 'form', 'stock')

    return JsonResponse(list(inventory_items), safe=False)
```

/Get_medicine_names.py

```
#Search algorithm to find medicine
#elements in the api response, and get the names of the medicines
#and the URL id of the medicines

def get_medicine_names(data):
    medicines = []
    for item in data.get('significantLink', []):
        # Check if item has mainEntityOfPage with Medicine in genre, which filters out non-medicine items
```



```

if (item.get('mainEntityOfPage', {}).get('genre', []))
    and 'Medicine' in item['mainEntityOfPage']['genre']:
    # Extract ID from URL
    url = item.get('url', "")
    med_id = url.split('/')[ -2] if url else None

    if med_id:
        medicines.append({
            'name': item['name'],
            'id': med_id
        })

return medicines

/Get_medicines.py
import requests
from django.conf import settings

API_URL = settings.API
API_KEY = settings.API_KEY

#API call function
#This function takes a category as input
#and returns a list of medicines in that category
#It uses the requests library to make a GET request to the API
def get_medicine(category):
    try:
        headers = { "subscription-key": API_KEY, "Accept": "application/json" }
        params = {
            "category": f"{category}",
            "page": 1,
            "per-page": 100
        }

        response = requests.get(API_URL, headers=headers, params=params)
        response.raise_for_status()

```

```

        return response.json()

#error handling
except requests.RequestException as e:
    return f"Error fetching medicines: {e}"

/Get_upcoming_medications.py
from datetime import datetime
from django.utils import timezone
from ..models import MedicationSchedule, MedicationLog

def merge_sort_upcoming_meds(upcoming_meds):
    if len(upcoming_meds) <= 1:
        return upcoming_meds

    # Split the list into two halves
    mid = len(upcoming_meds) // 2
    left_half = merge_sort_upcoming_meds(upcoming_meds[:mid])
    right_half = merge_sort_upcoming_meds(upcoming_meds[mid:])

    # Merge the sorted halves
    return merge(left_half, right_half)

def merge(left, right):
    sorted_list = []
    while left and right:
        # Compare planned_time and append the smaller one
        if left[0]['planned_time'] <= right[0]['planned_time']:
            sorted_list.append(left.pop(0))
        else:
            sorted_list.append(right.pop(0))

    # Append the remaining elements
    sorted_list.extend(left)
    sorted_list.extend(right)

    return sorted_list

def get_upcoming_medications(carehome):
    now = timezone.now()

```

```

upcoming_meds = []

# Get all residents in the care home
residents = carehome.residents.all()

# Get all medication schedules for these residents
schedules = MedicationSchedule.objects.filter(
    resident__in=residents,
    is_active=True
).select_related(
    'resident',
    'medication'
).prefetch_related('times')

# Organize medications by time slots
for schedule in schedules:
    for time_slot in schedule.times.all():
        med_time = timezone.make_aware(
            datetime.combine(now.date(), time_slot.time)
        )
        # time difference between when medication is due and the current time
        time_diff = med_time - now

        # If medication is more than 12 hours overdue, skip
        if time_diff.total_seconds() < -43 200: # 12 hours in seconds
            continue

        # Check if medication was already given today by checking the log
        was_given = MedicationLog.objects.filter(
            schedule=schedule,
            scheduled_time=time_slot,
            given_at__date=now.date()
        ).exists()

        # Calculate urgency level
        urgency = calculate_urgency(was_given, time_diff)

        upcoming_meds.append({
            'schedule': schedule,
            'time_slot': time_slot,
            'planned_time': med_time,
            'time_diff': time_diff,
            'urgency': urgency,
            'was_given': was_given
        })

```

```

# Sort by time
upcoming_meds = merge_sort_upcoming_meds(upcoming_meds)

return upcoming_meds

```

```

def calculate_urgency(was_given, time_diff):
    if was_given:
        return 'given'
    thresholds = [
        (0, 'overdue'),
        (1800, 'urgent'), # 30 minutes
        (3600, 'warning') # 1 hour
    ]
    for limit, label in thresholds:
        if time_diff.total_seconds() < limit:
            return label
    return 'normal'

```

/Get_user_carehome.py

#reusable function to get the carehome of a user

```

def get_user_carehome(user):
    if hasattr(user, 'manager'):
        return user.manager.carehome
    elif hasattr(user, 'carer'):
        return user.carer.carehome
    else:
        return None

```

/Mar_generator.py

```

from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from datetime import datetime, timedelta
from ..models import MedicationLog

```

#Mar chart generator funtion using reportlab library to create a PDF document.

#This function takes a resident and a start date as input

#and generates a MAR chart for the resident from the start date to the end of the month

#The document is structured with a header containing resident information and a table for each medication schedule

#The tables show the administration times and the initials of the staff who administered the medication

```

class MARGenerator:
    def __init__(self, resident, start_date):
        self.resident = resident
        self.start_date = start_date
        self.styles = getSampleStyleSheet()

        self.styles.add(ParagraphStyle(
            'Wrapped',
            parent=self.styles['Normal'],
            wordWrap='CJK'
        ))

    def generate_mar_chart(self, output_path):
        doc = SimpleDocTemplate(
            output_path,
            pagesize=A4,
            rightMargin=30,
            leftMargin=30,
            topMargin=30,
            bottomMargin=30
        )

        elements = []
        elements.extend(self._create_header())
        elements.extend(self._create_medication_tables())

        doc.build(elements)

    def _create_header(self):
        conditions = Paragraph(
            self.resident.conditions or 'None known',
            self.styles['Wrapped']
        )
        allergies = Paragraph(
            self.resident.allergies or 'None known',

```

```

        self.styles['Wrapped']
    )

    # Create header table with resident information
    header_data = [
        ['Name:', self.resident.name, 'DOB:', self.resident.date_of_birth.strftime('%d/%m/%Y') if
self.resident.date_of_birth else ""],
        ['Room:', self.resident.room_number or 'Not assigned', 'Care Home:', self.resident.carehome.name],
        ['Group:', self.resident.group.name if self.resident.group else 'Not assigned', 'Start Date:',
self.start_date.strftime('%d/%m/%Y')],
        ['Allergies:', allergies, ", "],
        ['Conditions:', conditions, ", "]
    ]

    header_table = Table(header_data, colWidths=[1.2*inch, 2.8*inch, 1.2*inch, 2.2*inch])
    header_table.setStyle(TableStyle([
        ('GRID', (0, 0), (-1, -1), 1, colors.black),
        ('FONTNAME', (0, 0), (-1, -1), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, -1), 10),
        ('BACKGROUND', (0, 0), (0, -1), colors.lightgrey),
        ('BACKGROUND', (2, 0), (2, -1), colors.lightgrey),
        ('SPAN', (1, 3), (-1, 3)),
        ('SPAN', (1, 4), (-1, 4)),
        ('VALIGN', (0, 0), (-1, -1), 'TOP'),
    ]))

    return [header_table, Paragraph("<br/><br/>", self.styles['Normal'])]

def _create_medication_tables(self):
    elements = []

    # Get all medications for the resident
    medications = self.resident.medicationschedule_set.all()

    for medication in medications:
        elements.extend(self._create_single_medication_table(medication))

```

```

elements.append(Paragraph("<br/><br/>", self.styles['Normal']))

return elements

def _create_single_medication_table(self, medication_schedule):

    med_header = [
        Paragraph(f"<b>Medication:</b> {medication_schedule.medication.name}",
self.styles['Normal']),
        Paragraph(
            f"<b>Dosage:</b> {medication_schedule.dosage} | "
            f"<b>Route:</b> {medication_schedule.get_route_display()} | "
            f"<b>Frequency:</b> {medication_schedule.frequency} times daily",
            self.styles['Normal']
        )
    ]

    med_table = Table(med_header, colWidths=[7*inch])
    med_table.setStyle(TableStyle([
        ('BOX', (0, 0), (-1, -1), 1, colors.black),
        ('FONTSIZE', (0, 0), (-1, -1), 10),
        ('BACKGROUND', (0, 0), (-1, -1), colors.lightgrey),
    ]))

    # Create full month administration table
    dates = [(self.start_date + timedelta(days=x)) for x in range(31)]
    admin_times = medication_schedule.times.all()

    elements = [med_table, Paragraph("<br/>", self.styles['Normal'])]

    # Split dates into weeks

```

```

weeks = [dates[i:i + 7] for i in range(0, len(dates), 7)]

# Create a table for each week
for week_dates in weeks:

    # Create the table header with dates
    table_data = [['Time'] + [d.strftime('%d/%m') for d in week_dates]]

    # For each administration time
    for time_slot in admin_times:
        row = [time_slot.time.strftime('%H:%M')]

        # For each date in the week
        for date in week_dates:

            # Check if medication was given
            given = MedicationLog.objects.filter(
                schedule=medication_schedule,
                scheduled_time=time_slot,
                given_at__date=date
            ).first()

            if given:
                # This gets the initials for the Nurse/Carer who gave the medication.
                initials = f"{given.given_by.user.first_name[0]}{given.given_by.user.last_name[0]}.upper()"
                row.append(initials)
            else:
                row.append("")

        table_data.append(row)

# Creates the weekly table
admin_table = Table(table_data, colWidths=[1*inch] + [0.8*inch]*len(week_dates))
admin_table.setStyle(TableStyle([
    ('GRID', (0, 0), (-1, -1), 1, colors.black),
    ('BACKGROUND', (0, 0), (-1, 0), colors.lightgrey),

```



```

('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
('FONTSIZE', (0, 0), (-1, -1), 8),
('ALIGN', (0, 0), (-1, -1), 'CENTER'),
)))

```

```

elements.append(admin_table)
elements.append(Paragraph("<br/>", self.styles['Normal']))

```

```

return elements

```

/decoratorscustom.py

```

from functools import wraps
from django.shortcuts import redirect
from django.contrib import messages

#custom django decorator that uses the carer permissions table
#to control view access. Takes a string input of a carer permission as a arguement.
def carer_permission_required(permission):
    def decorator(view_func):
        @wraps(view_func)
        def _wrapped_view(request, *args, **kwargs):
            # Allow access for managers
            if hasattr(request.user, 'manager'):
                return view_func(request, *args, **kwargs)

            # Check carer permissions
            if not hasattr(request.user, 'carer'):
                messages.error(request, "Access denied")
                return redirect('login')

            carer_permissions = request.user.carer.permissions
            if not getattr(carer_permissions, permission):
                messages.error(request, "You don't have permission to perform this action")
                return redirect('carer_dashboard')

            return view_func(request, *args, **kwargs)
    
```

```
        return _wrapped_view
    return decorator
```

/Forms.py

```
from django import forms
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.core.exceptions import ValidationError
from django.core.exceptions import ValidationError
from django.contrib.auth.password_validation import validate_password
from django.forms.fields import EmailField
from django.forms.forms import Form
```

```
from django.utils import timezone
from datetime import timedelta
from .models import InvitationCode
```

```
from .models import *
```

```
#form for generating a invitation code to be used by a carer during signup
```

```
class GenerateInvitationCodeForm(forms.Form):
    expiry_days = forms.IntegerField(
        min_value=1,
        max_value=30,
        initial=7,
        label="Invitation valid for (days)",
        widget=forms.NumberInput(attrs={'class': 'form-control'})
    )
```

```
def generate_code(self, carehome, created_by):
```

```
    # Set expiry date
    expiry_days = self.cleaned_data['expiry_days']
    expires_at = timezone.now() + timedelta(days=expiry_days)
```

```
    # Generate code
    code = InvitationCode.generate_unique_code()
```

```
    # Create invitation
```

```

invitation = InvitationCode.objects.create(
    code=code,
    carehome=carehome,
    created_by=created_by,
    expires_at=expires_at
)

return invitation

#User signup forms
#fields: first name, last name, email, password, and password confirmation
#inherits from the Django UserCreationForm
#extends the function of Django's default form, taking in first name/last name/email
class CustomUserForm(UserCreationForm):
    first_name = forms.CharField(
        max_length=15,
        required=True,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your first name'
        })
    )

    last_name = forms.CharField(
        max_length=15,
        required=True,
        widget=forms.TextInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your last name'
        })
    )

    email = forms.EmailField(
        required=True,
        widget=forms.EmailInput(attrs={
            'class': 'form-control',
            'placeholder': 'Enter your email address'
        })
    )

    password1 = forms.CharField(

```

```

        widget=forms.PasswordInput(attrs={
            'class': 'form-control',
            'placeholder': 'Create a password'
        })
    )

```

```

password2 = forms.CharField(
    widget=forms.PasswordInput(attrs={
        'class': 'form-control',
        'placeholder': 'Confirm your password'
    })
)

```

```

class Meta:
    model = User
    fields = ('first_name', 'last_name', 'email', 'password1', 'password2')

```

#form for taking in manager signup details, inherits from the custom user form.

#email is set to the username and manager is created in the database.

```

class ManagerSignUpForm(CustomUserForm):

```

#inherits the fields from the custom user form

```

class Meta(CustomUserForm.Meta):
    model = User

```

```

def save(self, commit=True):
    user = super().save(commit=False)
    user.username = user.email.lower()
    if commit:
        user.save()
        Manager.objects.create(user=user)
    return user

```

#form for taking in carer signup details, inherits from the custom user form.

The carehome code is used to check that the carehome exists, before allowing a carer to be created

#data is cleaned, email is set to the username and carer is created in the database.

```

class CarerSignUpForm(CustomUserForm):

```

```

invitation_code = forms.CharField(
    max_length=10,
    required=True,
    widget=forms.TextInput(attrs={
        'class': 'form-control',
        'placeholder': 'Enter your invitation code'
    })
)

class Meta(CustomUserForm.Meta):
    model = User

def clean_invitation_code(self):
    code = self.cleaned_data.get('invitation_code')
    now = timezone.now()

    # Check if code exists and is valid
    try:
        invitation = InvitationCode.objects.get(
            code=code,
            is_used=False,
            expires_at__gt=now
        )
    except InvitationCode.DoesNotExist:
        raise forms.ValidationError(
            "Invalid or expired invitation code. Please check and try again."
        )

    return code

def save(self, commit=True):
    user = super().save(commit=False)
    user.username = user.email.lower()
    if commit:
        user.save()

    # Get the invitation code and its related carehome
    invitation = InvitationCode.objects.get(code=self.cleaned_data['invitation_code'])
    carehome = invitation.carehome

```

```
# Create carer object
Carer.objects.create(user=user, carehome=carehome)
```

```
# Mark invitation as used
invitation.mark_as_used(user)
```

```
return user
```

```
#carehome setup form
```

```
#form for setting up a carehome, takes in name and location
```

```
#location data from django-cities-light plugin, query search filter is used to only show cities in the UK
```

```
class CareHomeSetupForm(forms.ModelForm):
    location = forms.ModelChoiceField(queryset=City.objects.filter(country__code2='GB'))
    class Meta:
        model = CareHome
        fields = ['name', 'location']
```

```
#resident form, takes in name, photo, age and conditions. The photo and group are optional
```

```
class ResidentForm(forms.ModelForm):
    class Meta:
        model = Resident
        fields = ['name', 'photo', 'date_of_birth', 'conditions', 'allergies', 'room_number', 'group']
        widgets = {
            'date_of_birth': forms.DateInput(attrs={'type': 'date', }),
            'conditions': forms.Textarea(attrs={'rows': 3, 'placeholder': 'List any medical conditions'}),
            'allergies': forms.Textarea(attrs={'rows': 3, 'placeholder': 'List any allergies'}),
            'room_number': forms.TextInput(attrs={'placeholder': 'Enter room number'}),
        }
```

```
def __init__(self, *args, **kwargs):
    carehome = kwargs.pop('carehome', None)

    super().__init__(*args, **kwargs)
```

```

self.fields['photo'].required = False
self.fields['group'].required = False
self.fields['allergies'].required = False
self.fields['conditions'].required = False
self.fields['room_number'].required = False

```

```

if carehome:
    self.fields['group'].queryset = ResidentGroup.objects.filter(carehome=carehome)
    self.fields['group'].label_from_instance = lambda obj: obj.name

```

```

self.fields['date_of_birth'].input_formats = ['%Y-%m-%d']

```

#form for creating a resident group, takes in name and colour, the colour is a hex code
#and is displayed as a colour picker in the webpage.

```

class ResidentGroupForm(forms.ModelForm):

```

```

    class Meta:
        model = ResidentGroup
        fields = ['name', 'colour']
        widgets = {
            'colour': forms.TextInput(attrs={
                'type': 'color',
                'class': 'form-control form-control-color',
                'title': 'Choose group colour'
            })
        }

```

#defines the fields for a medication variant in the database

```

class MedicationInventoryItemForm(forms.ModelForm):

```

```

    class Meta:
        model = MedicationInventoryItem
        fields = ['dosage', 'form', 'stock', 'reorder_level']
        widgets = {
            'dosage': forms.TextInput(attrs={'class': 'form-control', 'placeholder': '500mg'}),
            'form': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'tablet'}),
            'stock': forms.NumberInput(attrs={'class': 'form-control', 'min': '0'}),
            'reorder_level': forms.NumberInput(attrs={'class': 'form-control', 'min': '1', 'value': '10'}),
        }

```

#form for creating a medication schedule, takes in medication, dosage and frequency

#the frequency is a number between 1 and 6, referring to each instance a medicine should be dispensed a day

#the route (way resident takes the medication) is a dropdown field, with the predefined options
being oral, topical, inhalation and injection

```
class MedicationScheduleForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = MedicationSchedule
```

```
        fields = ['medication', 'inventory_item', 'frequency', 'route']
```

```
        widgets = {
```

```
            'frequency': forms.NumberInput(attrs={'min': 1, 'max': 6}),
```

```
            'route': forms.Select(attrs={'class': 'form-control'})
```

```
        }
```

```
def __init__(self, *args, **kwargs):
```

```
    # Extract carehome
```

```
    carehome = kwargs.pop('carehome', None)
```

```
    super().__init__(*args, **kwargs)
```

```
    # Set medication choices to those in the carehome
```

```
    if carehome:
```

```
        self.fields['medication'].queryset = Medication.objects.filter(carehome=carehome)
```

```
    # Start with empty inventory item queryset
```

```
    self.fields['inventory_item'].queryset = MedicationInventoryItem.objects.none()
```

```
    # If form has medication field and is POST.
```

```
    if self.is_bound and 'medication' in self.data:
```

```
        try:
```

```
            medication_id = int(self.data.get('medication'))
```

```
            # Set the inventory item based on the selected medication
```

```
            self.fields['inventory_item'].queryset = MedicationInventoryItem.objects.filter(
```

```
                medication_id=medication_id
```

```
            )
```

```
        except (ValueError, TypeError):
```

```
            pass
```

```
    # If editing an existing instance with a medication
```

```
    elif self.instance.pk and self.instance.medication:
```

```
        self.fields['inventory_item'].queryset = MedicationInventoryItem.objects.filter(
```

```
            medication=self.instance.medication
```



```

    )

#carer permissions set as booleans
#displayed as checkboxes on the webpage
class CarerPermissionsForm(forms.ModelForm):
    class Meta:
        model = CarerPermissions
        fields = [
            'can_edit_residents',
            'can_edit_medications',
            'can_view_inventory',
            'can_adjust_stock'
        ]
        labels = {
            'can_edit_residents': 'Can edit resident information',
            'can_edit_medications': 'Can edit medication schedules',
            'can_view_inventory': 'Can view medication inventory',
            'can_adjust_stock': 'Can adjust medication stock levels'
        }

class CarerAccessForm(forms.ModelForm):
    access_duration = forms.ChoiceField(
        choices=[
            ('24h', '24 Hours'),
            ('48h', '48 Hours'),
            ('1w', '1 Week'),
            ('2w', '2 Weeks'),
            ('1m', '1 Month'),
            ('indefinite', 'Indefinite')
        ],
        widget=forms.RadioSelect,
        required=True
    )

    class Meta:
        model = Carer
        fields = ['indefinite_access']

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.fields['indefinite_access'].widget = forms.HiddenInput()

```

```

# Initialize the access_duration field based on current settings
instance = kwargs.get('instance')
if instance:
    if instance.indefinite_access:
        self.fields['access_duration'].initial = 'indefinite'
    elif instance.access_end_date:
        # Calculate remaining time and select closest option
        remaining = instance.access_end_date - timezone.now()
        days = remaining.days
        if days <= 1:
            self.fields['access_duration'].initial = '24h'
        elif days <= 2:
            self.fields['access_duration'].initial = '48h'
        elif days <= 7:
            self.fields['access_duration'].initial = '1w'
        elif days <= 14:
            self.fields['access_duration'].initial = '2w'
        else:
            self.fields['access_duration'].initial = '1m'

def save(self, commit=True):
    carer = super().save(commit=False)
    access_duration = self.cleaned_data.get('access_duration')

    if access_duration == 'indefinite':
        carer.indefinite_access = True
        carer.access_end_date = None
    else:
        carer.indefinite_access = False
        now = timezone.now()

        if access_duration == '24h':
            carer.access_end_date = now + timedelta(hours=24)
        elif access_duration == '48h':
            carer.access_end_date = now + timedelta(hours=48)
        elif access_duration == '1w':
            carer.access_end_date = now + timedelta(weeks=1)
        elif access_duration == '2w':
            carer.access_end_date = now + timedelta(weeks=2)
        elif access_duration == '1m':
            carer.access_end_date = now + timedelta(days=30)

```

```

    if commit:
        carer.save()
    return carer

```

/models.py

```

from django.db import models
from django.forms import ModelForm
from django.contrib.auth.forms import UserCreationForm
from django import forms
from django.conf import settings
from django.contrib.auth.models import User, AbstractUser, BaseUserManager
from django.utils.translation import gettext_lazy as _
import random
from datetime import date
from cities_light.models import *
from django.utils import timezone

#carehome model
class CareHome(models.Model):
    name = models.CharField(max_length=100)
    location = models.ForeignKey(City, on_delete=models.SET_NULL, null=True,
related_name='carehomes')
    manager = models.OneToOneField(User, on_delete=models.SET_NULL, null=True,
related_name='managed_carehomes')

class InvitationCode(models.Model):
    code = models.CharField(max_length=10, unique=True)
    carehome = models.ForeignKey(CareHome, on_delete=models.CASCADE,
related_name='invitation_codes')
    created_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True,
related_name='created_invitations')
    created_at = models.DateTimeField(auto_now_add=True)
    expires_at = models.DateTimeField()
    is_used = models.BooleanField(default=False)
    used_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True,
related_name='used_invitation')
    used_at = models.DateTimeField(null=True, blank=True)

    @classmethod
    def generate_unique_code(cls):

```

```

#generates a unique, 8 letter long alphanumeric code
import random
import string

while True:
    # Generate an 8-character alphanumeric code
    code = ''.join(random.choices(string.ascii_uppercase + string.digits, k=8))

    # Makes sure code is unique and doesnt already exist.
    if not cls.objects.filter(code=code).exists():
        return code

def mark_as_used(self, user):
    #Marks a code as used
    from django.utils import timezone

    self.is_used = True
    self.used_by = user
    self.used_at = timezone.now()
    self.save()

#defines the resident group model, associated with a colour and a carehome
class ResidentGroup(models.Model):
    name = models.CharField(max_length=100)
    carehome = models.ForeignKey(CareHome, on_delete=models.CASCADE,
related_name='resident_groups')
    colour = models.CharField(max_length=7, default='#FF0000')

#User models
class Manager(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    carehome = models.ForeignKey(CareHome, on_delete=models.CASCADE, related_name='managers',
null=True, blank=True)

class CarerPermissions(models.Model):

```

```

can_edit_residents = models.BooleanField(default=False)
can_edit_medications = models.BooleanField(default=False)
can_view_inventory = models.BooleanField(default=False)
can_adjust_stock = models.BooleanField(default=False)
carer = models.OneToOneField('Carer', on_delete=models.CASCADE, related_name='permissions')

def __str__(self):
    return f"Permissions for {self.carer.user.username}"

#Carer iser model, created after a carer successfully signs up, one-to-one realtion with a Django-User.
class Carer(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    carehome = models.ForeignKey(CareHome, on_delete=models.CASCADE, related_name='carers')
    resident_group = models.ForeignKey(ResidentGroup, on_delete=models.SET_NULL, null=True,
blank=True, related_name='carers')
    access_end_date = models.DateTimeField(null=True, blank=True)
    indefinite_access = models.BooleanField(default=False)

    def save(self, *args, **kwargs):
        is_new = self.pk is None
        super().save(*args, **kwargs)
        if is_new:
            CarerPermissions.objects.create(carer=self)

#used to check if the carer has access to the carehome
#if the carer has indefinite access, returns true
#additonally, if the access end date is more than the current date, returns true
#otherwise returns false
@property
def has_valid_access(self):
    if self.indefinite_access:
        return True
    if self.access_end_date and timezone.now() < self.access_end_date:
        return True
    return False

#This model is used to store the information of the residents in the carehome.
#It contains the name, photo, conditions, carehome, group, date of birth, allergies and room number.
class Resident(models.Model):
    name = models.CharField(max_length=100)
    photo = models.ImageField(upload_to='resident_photos/', null=True, blank=True,
default='resident_photos/default.png')
    conditions = models.TextField(blank=True)
    carehome = models.ForeignKey(CareHome, on_delete=models.CASCADE, related_name='residents')
    group = models.ForeignKey(ResidentGroup, on_delete=models.SET_NULL, null=True, blank=True,
related_name='residents')

```

```

date_of_birth = models.DateField(null=True, blank=True)
allergies = models.TextField(blank=True, help_text="List any known allergies")
room_number = models.CharField(max_length=10, null=True, blank=True)

def __str__(self):
    return self.name

#used to calculate the age of the resident using the current date and the date of birth
@property
def calculate_age(self):
    if self.date_of_birth:
        today = date.today()
        return today.year - self.date_of_birth.year - ((today.month, today.day) < (self.date_of_birth.month,
self.date_of_birth.day))
    return None

#Medication models

#medication model, takes in name, description, nhs_id, stock and carehome
#the carehome is set to the carehome that the medication belongs to,
class Medication(models.Model):
    name = models.CharField(max_length=100)
    nhs_id = models.CharField(max_length=50, default=None)
    carehome = models.ForeignKey(CareHome, on_delete=models.CASCADE)

    def __str__(self):
        return self.name

#Stores a variant of a carehomes stored medication.
class MedicationInventoryItem(models.Model):
    medication = models.ForeignKey(Medication, on_delete=models.CASCADE,
related_name='inventory_items')
    dosage = models.CharField(max_length=100) # e.g., "500mg"
    form = models.CharField(max_length=100, blank=True) # e.g., "tablet", "capsule", "liquid"
    stock = models.PositiveIntegerField(default=0)
    reorder_level = models.PositiveIntegerField(default=10) # Threshold for reordering

class Meta:
    unique_together = ['medication', 'dosage', 'form']

```

```

def __str__(self):
    return f"{self.medication.name} {self.dosage} {self.form} - Stock: {self.stock}"

def is_low_stock(self):
    return self.stock <= self.reorder_level

#medication schedule model, takes in resident, medication, dosage, frequency and is_active
#the is_active is a boolean to check if the schedule is active
class MedicationSchedule(models.Model):
    ROUTE_CHOICES = [
        ('oral', 'Oral'),
        ('topical', 'Topical'),
        ('injection', 'Injection'),
        ('inhaled', 'Inhaled'),
        ('other', 'Other'),
    ]

    resident = models.ForeignKey(Resident, on_delete=models.CASCADE)
    medication = models.ForeignKey(Medication, on_delete=models.CASCADE)
    inventory_item = models.ForeignKey(MedicationInventoryItem, on_delete=models.CASCADE,
default=None)
    frequency = models.CharField(max_length=100)
    route = models.CharField(max_length=20, choices=ROUTE_CHOICES, default='oral')
    is_active = models.BooleanField(default=True)

def __str__(self):
    return f"{self.resident.name} - {self.inventory_item} - {self.frequency} times daily"

@property
def dosage(self):
    return self.inventory_item.dosage

#medication time model, records every instance/time in a daily schedule were medicaiton needs to me
dispensed.
class MedicationTime(models.Model):
    schedule = models.ForeignKey(MedicationSchedule, on_delete=models.CASCADE,
related_name='times')
    time = models.TimeField()

```

```

#medication log model
#It records the time the medication was given, the carer who gave it and any
#used to keep track of medication transactions.
#contains the method called when generating a mar chart.
class MedicationLog(models.Model):
    schedule = models.ForeignKey(MedicationSchedule, on_delete=models.CASCADE)
    scheduled_time = models.ForeignKey(MedicationTime, on_delete=models.CASCADE)
    given_at = models.DateTimeField(auto_now_add=True)
    given_by = models.ForeignKey(Carer, on_delete=models.CASCADE)
    notes = models.TextField(blank=True)

class Meta:
    ordering = ['-given_at']

    @classmethod
    def generate_mar_chart(cls, resident, start_date, output_path):
        from .utils.mar_generator import MARGenerator

        generator = MARGenerator(resident, start_date)
        generator.generate_mar_chart(output_path)

```

/urls.py

```

from django.urls import path
from MedTracker import views
from django.conf.urls import handler404
from MedTracker.views import custom_404
from django.conf import settings
from django.conf.urls.static import static
from MedTracker.utils.get_medication_inventory import *

handler404 = custom_404

urlpatterns = [
    path("", views.index, name="index"),
    path("aboutus/", views.aboutus, name="aboutus"),
    path('logout/', views.logout_view, name='logout'),

    path('register/', views.registration, name='register'),

    path('carehomesetup/', views.setup_carehome, name='setup_carehome'),
    path('login/', views.login_view, name='login'),

```



```

path('carerdashboard/', views.carer_dashboard, name='carer_dashboard'),

path('record-medication/<int:schedule_id>/<int:time_slot_id>/',
     views.record_medication,
     name='record_medication'),


path('dashboard/updates/', views.get_dashboard_updates, name='get_dashboard_updates'),
path('managerdashboard/', views.manager_dashboard, name='manager_dashboard'),
path('manager/residents/', views.manager_dashboard_residents,
name='manager_dashboard_residents'),
path('manager/residents/edit/<int:resident_id>/', views.edit_resident, name='edit_resident'),
path('manager/carers/', views.manager_dashboard_carers, name='manager_dashboard_carers'),


path('manager/manage_invites/', views.manage_invitation_codes, name='manage_invitation_codes'),


path('resident/<int:resident_id>/medications/', views.resident_medications,
name='resident_medications'),
path('resident/<int:resident_id>/add-medication/', views.add_medication_schedule,
name='add_medication_schedule'),


path('medications/', views.medication_inventory, name='medication_inventory'),


path('medications/add/', views.add_medication_to_db, name='add_medication_to_db'),
path('medications/search/<str:search_term>/', views.search_medications, name='search_medications'),
path('medications/audit-log/', views.medication_audit_log, name='medication_audit_log'),


path('medication-schedule/<int:schedule_id>/delete/', views.delete_medication_schedule,
name='delete_medication_schedule'),


path('manage-carer-permissions/<int:carer_id>/', views.manage_carer_permissions,
name='manage_carer_permissions'),


path('resident/<int:resident_id>/mar-chart/', views.generate_mar_chart, name='generate_mar_chart'),

path('resident/add/', views.add_resident, name='add_resident'),


path('api/medication/<int:medication_id>/inventory-items/', get_medication_inventory_items,
name='get_medication_inventory_items'),
path('medications/inventory/add/<int:medication_id>/', views.add_inventory_item,
name='add_inventory_item'),

```

```

    path('medications/inventory/adjust-stock/<int:inventory_item_id>/', views.adjust_inventory_stock,
name='adjust_inventory_stock'),

    path('medications/inventory/<int:inventory_item_id>/delete/', views.delete_inventory_item,
name='delete_inventory_item'),
    path('medications/delete/<int:medication_id>/', views.delete_medication, name='delete_medication'),

    path('manage-carer-access/<int:carer_id>/', views.manage_carer_access,
name='manage_carer_access'),
]

```

/views.py

```

from django.shortcuts import render, redirect, \
    get_object_or_404
from django.db import IntegrityError
from django.views.decorators.http import require_http_methods
from django.contrib.auth import login, authenticate, logout
from django.http import HttpResponseRedirect, JsonResponse, FileResponse
from django.forms import inlineformset_factory
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.contrib.auth.models import User
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.conf import settings
from django.core.cache import cache
from django.db import transaction
from django.utils import timezone
from datetime import datetime, date, timedelta
from django.views.decorators.csrf import csrf_protect, ensure_csrf_cookie
import logging
from django.db.models import Prefetch
from MedTracker.decoratorscustom import carer_permission_required
from django.template.loader import render_to_string
from django.template import RequestContext
from django.shortcuts import render
import os
from io import BytesIO

from .utils.get_medicines import get_medicine
from .utils.get_medicine_names import get_medicine_names
from .utils.get_upcoming_medications import get_upcoming_medications, calculate_urgency
from .utils.get_user_carehome import get_user_carehome
from .forms import *
from .models import *

```

```

#Basic website views, 404, logout, index, about us

def custom_404(request, exception=None):
    return render(request, 'staticpages/404.html', status=404)

#logout, sends user back to landing page
def logout_view(request):
    logout(request)
    return redirect('/')

#this is the landing page
def index(request):
    template = 'staticpages/index.html'
    return render(request, template)

#Little about us page to describe what the website is for
def aboutus(request):
    template = 'staticpages/aboutcompany.html'
    return render(request, template)


#after a manager has been created, they create a carehome within the database
@login_required
@ensure_csrf_cookie
def setup_carehome(request):
    #initial check for the user type, only accepts managers
    user = request.user
    try:
        manager = user.manager
    except Manager.DoesNotExist:
        return redirect('/')

    try:
        carehome = manager.carehome
    except CareHome.DoesNotExist:
        carehome = None

    #form to create a carehome, checks if the form is valid, generates a unique code for the carehome
    # and saves the carehome to the database.
    if request.method == 'POST':
        form = CareHomeSetupForm(request.POST)
        if form.is_valid():
            carehome = form.save(commit=False)

```

```

        carehome.manager = user
        carehome.save()
        manager.carehome = carehome
        manager.save()
        return redirect('manager_dashboard')
    else:
        form = CareHomeSetupForm()

    return render(request, 'dashboards/setup_carehome.html',{'form': form})

#view for generating the one time invitation codes for carers.
@login_required
def manage_invitation_codes(request):
    # Checking user authorisation
    carehome = get_user_carehome(request.user)

    if not carehome:
        messages.error(request, "Unauthorized access")
        return redirect('/')

    if not hasattr(request.user, 'manager'):
        messages.error(request, "Unauthorized access")
        return redirect('carer_dashboard')

    # Get active invitations
    active_invitations = InvitationCode.objects.filter(
        carehome=carehome,
        is_used=False,
        expires_at__gt=timezone.now()
    ).order_by('-created_at')

    # Get used invitations
    used_invitations = InvitationCode.objects.filter(
        carehome=carehome,
        is_used=True
    ).order_by('-used_at')[:10] # Show last 10 used codes

```

```

# Handle form submission
if request.method == 'POST':
    if 'generate_code' in request.POST:
        form = GenerateInvitationCodeForm(request.POST)
        if form.is_valid():
            invitation = form.generate_code(carehome, request.user)
            messages.success(request, f"Invitation code {invitation.code} generated successfully!")
            return redirect('manage_invitation_codes')

    elif 'delete_code' in request.POST:
        code_id = request.POST.get('code_id')
        try:
            code = InvitationCode.objects.get(id=code_id, carehome=carehome, is_used=False)
            code.delete()
            messages.success(request, "Invitation code deleted successfully!")
        except InvitationCode.DoesNotExist:
            messages.error(request, "Code not found or already used.")
    else:
        form = GenerateInvitationCodeForm()

context = {
    'form': GenerateInvitationCodeForm(),
    'active_invitations': active_invitations,
    'used_invitations': used_invitations,
    'carehome': carehome
}

return render(request, 'manager/manage_invites.html', context)

#View for logging in and authenticating a user.
@csrf_protect
def login_view(request):
    user = request.user
    # if user is logged in, they are redirected to the dashboard
    if request.user.is_authenticated:
        try:
            manager = request.user.manager
            return redirect('manager_dashboard')

```

```

except Manager.DoesNotExist:
    try:
        carer = request.user.carer
        # Check if carer's access is still valid
        if not carer.has_valid_access:
            logout(request)
            messages.error(request, "Your access period has expired. Please contact your manager.")
            return redirect('login')
        return redirect('carer_dashboard')
except Carer.DoesNotExist:
    logout(request)
    messages.error(request, "User account type not recognized. Please contact support.")
    return redirect('login')

# if the user is not logged in, they are shown the login form
if request.method == 'POST':
    form = AuthenticationForm(request, request.POST)
    if form.is_valid():

        user=form.get_user()
        username = form.cleaned_data.get('username')
        password = form.cleaned_data.get('password')

        user = authenticate(username=username,
                             password=password)

    if user is not None:
        login(request, user)
        #Redirects manager user and carer user to correct dashboard
        try:
            manager = request.user.manager
            return redirect('manager_dashboard')
        except Manager.DoesNotExist:
            try:
                #checks if the carer has valid access before redirecting them
                carer = request.user.carer
                if not carer.has_valid_access:
                    logout(request)
                    messages.error(request, "Your access period has expired. Please contact your manager.")
                    return redirect('login')

            except Carer.DoesNotExist:
                return redirect('carer_dashboard')
        except Carer.DoesNotExist:

```

```

        logout(request)
        messages.error(request, "User account type not recognized. Please contact support.")
    return redirect('login')

else:
    for field, errors in form.errors.items():
        for error in errors:
            messages.error(request, f"{error}")
else:
    form = AuthenticationForm()

return render(request, 'auth/login.html', {'form': form})

#registration page view, handles both the manager and carer registration form
#validates the form and saves the data to the database.
@csrf_protect
@ensure_csrf_cookie
def registration(request):

    #if the server request is a post request, the form is checked for validity
    if request.method == 'POST':
        #The registration page manages both user types, manager and carer
        form1 = ManagerSignUpForm(request.POST)
        form2 = CarerSignUpForm(request.POST)

        #if the manager form is submitted, the form is checked for validity
        #the user is then created and logged in
        #the user is then redirected to the carehome setup page as a new manager.
        try:
            if 'sign_up_manager' in request.POST:
                form = ManagerSignUpForm(request.POST)
                if form.is_valid():
                    user = form.save()
                    login(request, user)
                    return redirect('setup_carehome')
            else:
                for field, errors in form.errors.items():
                    for error in errors:
                        messages.error(request, f"{error}")

        #if the carer form is submitted, the form is checked for validity
        #the user is then created and logged in
        #the user is then redirected to the carer dashboard for their carehome.
        elif 'sign_up_carer' in request.POST:
            form = CarerSignUpForm(request.POST)

```

```

    if form.is_valid():
        user = form.save()
        login(request, user)
        return redirect('carer_dashboard')
    else:
        for field, errors in form.errors.items():
            for error in errors:
                messages.error(request, f"{error}")

except IntegrityError as e:
    #handles email already existing in database
    if "unique constraint" in str(e).lower():
        messages.error(request, "An account with that email already exists.")

else:
    form1 = ManagerSignUpForm()
    form2 = CarerSignUpForm()

return render(request, 'auth/businesssignup.html',
              {'form1': form1, 'form2': form2}, )

#manager dashboard, displays all the information for the manager
#this includes the residents, carers, and medication stock
#and displays the medication schedule for the day
#Context is made of a database query for the respective carehome components.
@login_required
def manager_dashboard(request):
    carehome = get_user_carehome(request.user)
    if hasattr(request.user, 'carer'):
        return redirect('carer_dashboard')

    if not carehome:
        messages.error(request, "Unauthorized access")
        return redirect('/')

```



```

# Fetch Context
medications = Medication.objects.filter(carehome=carehome).prefetch_related(
    Prefetch(
        'inventory_items',
        queryset=MedicationInventoryItem.objects.filter(
            medication__carehome=carehome
        )
    )
)
context = {
    'today': date.today(),
    'carehome': carehome,
    'residents_count': carehome.residents.count(),
    'carers_count': carehome.carers.count(),

    'carers': carehome.carers.all(),
    'residents': carehome.residents.all(),
    'medication_stock': medications,

    'today_medications': MedicationSchedule.objects.filter(
        resident__carehome=carehome
    ).prefetch_related(
        Prefetch('times', queryset=MedicationTime.objects.filter(
            time__hour__gte=0
        )).select_related('resident', 'medication'),

}

return render(request, 'dashboards/manager_dashboard.html', context)

```

#Primary view for carer users.

#Displays the medication schedule for the day, and allows the carer to record medication given to residents.

@login_required

def carer_dashboard(request):

If the user is not a carer, they are redirected to the login page

if not hasattr(request.user, 'carer'):

messages.error(request, "Unauthorized access")

return redirect('login')

Get the correct carer from the database and timezone

carer = request.user.carer

now = timezone.now()

```

# Get residents from carer's assigned group
residents = []
if carer.resident_group:
    residents = carer.resident_group.residents.all()

# Get all medication schedules for these residents
schedules = MedicationSchedule.objects.filter(
    resident__in=residents,
    is_active=True
).select_related(
    'resident',
    'medication'
).prefetch_related('times')

# Organize medications by time slots
upcoming_meds = []
for schedule in schedules:
    for time_slot in schedule.times.all():
        med_time = timezone.make_aware(
            datetime.combine(now.date(), time_slot.time)
        )
        # Time difference between when medication is due and the current time
        time_diff = med_time - now

        # If medication is more than 12 hours overdue, skip
        if time_diff.total_seconds() < -43 200: # 12 hours in seconds
            continue

        # Check if medication was already given today by checking the log
        was_given = MedicationLog.objects.filter(
            schedule=schedule,
            scheduled_time=time_slot,
            given_at__date=now.date()
        ).exists()

```

```

# Calculate urgency level
urgency = calculate_urgency(was_given, time_diff)

```

```

upcoming_meds.append({
    'schedule': schedule,
    'time_slot': time_slot,
    'planned_time': med_time,
    'time_diff': time_diff,
    'urgency': urgency,
    'was_given': was_given
})

```

```

# Sort by time
upcoming_meds.sort(key=lambda x: x['planned_time'])

```

```

context = {
    'carer': carer,
    'upcoming_meds': upcoming_meds,
    'current_time': now,
    'residents': residents,
}

```

```

return render(request, 'dashboards/carers_dashboard.html', context)

```

```

#Main view for adding, editing and removing residents
#as well as creating resident groups
#and navigating to the create schedule view.

```

```

@login_required
@carer_permission_required('can_edit_residents')
def manager_dashboard_residents(request):
    carehome = get_user_carehome(request.user)
    if not carehome:
        messages.error(request, "Unauthorized access")
        return redirect('carers_dashboard')

```

```

# This gets all the residents from the carehome
residents = Resident.objects.filter(carehome=carehome)

```

```

group_form = ResidentGroupForm()

if request.method == 'POST':
    #form for adding a new group to carehome
    if 'add_group' in request.POST:
        group_form = ResidentGroupForm(request.POST)
        if group_form.is_valid():
            group = group_form.save(commit=False)
            group.carehome = carehome
            group.save()
            messages.success(request, 'Group added successfully.')
            return redirect('manager_dashboard_residents')

    #POST request to remove resident from carehome
    elif 'remove_resident' in request.POST:
        resident_id = request.POST.get('resident_id')
        try:
            resident = Resident.objects.get(id=resident_id, carehome=carehome)
            resident.delete()
            messages.success(request, 'Resident removed successfully.')
        except Resident.DoesNotExist:
            messages.error(request, 'Resident not found.')
        return redirect('manager_dashboard_residents')

    else:
        group_form = ResidentGroupForm()
        resident_form = ResidentForm(carehome=carehome)

groups = ResidentGroup.objects.filter(carehome=carehome)

context = {
    'residents': residents,
    'group_form': group_form,
    'groups': groups,
}
return render(request, 'dashboards/manager_dashboard_residents.html', context)

#View for managing the time-based access for Carers
#Allows the manager to set a carer to have indefinite access, or set a date for their access to end.
@login_required
def manage_carer_access(request, carer_id):
    carehome = get_user_carehome(request.user)
    carer = get_object_or_404(Carer, id=carer_id, carehome=carehome)

```

```

if request.method == 'POST':
    form = CarerAccessForm(request.POST, instance=carer)
    if form.is_valid():
        form.save()
        if carer.indefinite_access:
            messages.success(request, f'Access for {carer.user.get_full_name()} set to indefinite')
        else:
            end_date = carer.access_end_date.strftime('%d %b %Y, %H:%M')
            messages.success(request, f'Access for {carer.user.get_full_name()} will expire on {end_date}')
        return redirect('manager_dashboard_carers')
    else:
        form = CarerAccessForm(instance=carer)

return render(request, 'dashboards/manage_carer_access.html', {
    'form': form,
    'carer': carer
})

```

#View for editing an existing resident in the carehome.
 #Uses the same resident form used to create a resident, instead resident data
 #is parsed to populate the form before editing.

```

@login_required
@carer_permission_required('can_edit_residents')
def edit_resident(request, resident_id):
    carehome = get_user_carehome(request.user)
    if not carehome:
        messages.error(request, "Unauthorized access")
        return redirect('/')

    resident = get_object_or_404(Resident, id=resident_id, carehome=carehome)

```

```

if request.method == 'POST':

    form = ResidentForm(request.POST, request.FILES, instance=resident, carehome=carehome)
    if form.is_valid():
        form.save()
        messages.success(request, 'Resident updated successfully:')
        return redirect('manager_dashboard_residents')
    else:
        form = ResidentForm(instance=resident, carehome=carehome)

```

```

return render(request, 'resident/edit_resident.html', {'form': form, 'resident': resident})

#View for mangement of care staff
#Allows the manager to assign a carer to a resident group, or remove them from a group.
#Also provides a overview for all the carers in the carehome.
@login_required
def manager_dashboard_carers(request):
    carehome = get_user_carehome(request.user)
    if not carehome:
        messages.error(request, "Unauthorized access")
        return redirect('/')
    if not hasattr(request.user, 'manager'):
        messages.error(request, "Unauthorized access")
        return redirect('carer_dashboard')

    carers = Carer.objects.filter(carehome=carehome)
    groups = ResidentGroup.objects.filter(carehome=carehome)

    if request.method == 'POST' and 'update_carer_group' in request.POST:
        carer_id = request.POST.get('carer_id')
        group_id = request.POST.get('group_id')

        try:
            carer = carers.get(id=carer_id)
            if group_id:
                group = groups.get(id=group_id)
                carer.resident_group = group
            else:
                carer.resident_group = None
            carer.save()
            messages.success(request, 'Carer group updated successfully.')
        except (Carer.DoesNotExist, ResidentGroup.DoesNotExist):
            messages.error(request, 'Error updating carer group.')

    context = {
        'carers': carers,
        'groups': groups,

```

```

        'carehome': carehome,
    }
    return render(request, 'dashboards/manager_dashboard_carers.html', context)

#Creates a medication schedule
@login_required
@carer_permission_required('can_manage_schedules')
def add_medication_schedule(request, resident_id):
    resident = get_object_or_404(Resident, id=resident_id)
    carehome = get_user_carehome(request.user)

    # Verify user has access to this resident
    if resident.carehome != carehome:
        messages.error(request, "Unauthorized access")
        return redirect('carer_dashboard')

    if request.method == 'POST':

        form = MedicationScheduleForm(request.POST, carehome=carehome)

        if form.is_valid():
            schedule = form.save(commit=False)
            schedule.resident = resident
            schedule.save()

            # Process time slots
            times = request.POST.getlist('times[]')
            for time_str in times:
                if time_str:
                    try:
                        #create a MedicationTime record for each time instance
                        time_obj = datetime.strptime(time_str, '%H:%M').time()
                        MedicationTime.objects.create(
                            schedule=schedule,
                            time=time_obj

```

```

    )
except ValueError:
    pass # Skip invalid time values

messages.success(request, f"Medication schedule added for {resident.name}")
return redirect('resident_medications', resident_id=resident_id)

else:
    form = MedicationScheduleForm(carehome=carehome)

return render(request, 'medication/add_medication_schedule.html', {
    'form': form,
    'resident': resident
})

@login_required
def resident_medications(request, resident_id):
    resident = get_object_or_404(Resident, id=resident_id)
    carehome = get_user_carehome(request.user)
    if not carehome:
        messages.error(request, "Unauthorized access")
        return redirect('/')

    # Verify the resident belongs to the correct carehome
    if carehome != resident.carehome:
        messages.error(request, "Unauthorized access")
        return redirect('manager_dashboard')

    schedules = MedicationSchedule.objects.filter(resident=resident)\
        .select_related('medication')\
        .prefetch_related('times')

    return render(request, 'resident/resident_medications.html', {
        'resident': resident,
        'schedules': schedules
    })

```


#View for the API based medication search screen. Provides the context to populate the
#html page with medication names.

@login_required

```
def search_medications(request, search_term):
    if not hasattr(request.user, 'manager'):
        messages.error(request, "Unauthorized access")
        return redirect('carer_dashboard')

    if len(search_term) == 1 and search_term.isalpha():
        try:
            data = get_medicine(search_term)
            medicine_names = get_medicine_names(data)
        except Exception as e:
            messages.error(request, f"Error fetching medicines: {str(e)}")
            medicine_names = []
        else:
            medicine_names = []

    return render(request, 'medication/search_medications.html', {
        'results': medicine_names,
        'search_term': search_term.upper() if search_term else ""
    })
```

#Runs the database query to store a new medication to the database

#Checks that medication doesn't already exist in the database

#Finds the medication in the API call from the id parsed from the medication search.

@login_required

```
def add_medication_to_db(request):
    if request.method == 'POST':
        nhs_id = request.POST.get('nhs_id')
        carehome = get_user_carehome(request.user)
        if not carehome:
            messages.error(request, "Unauthorized access")
            return redirect('/')

        if not nhs_id:
            messages.error(request, "Invalid medication ID")
            return redirect('search_medications', 'A')

        if Medication.objects.filter(nhs_id=nhs_id, carehome=carehome).exists():
            messages.warning(request, "This medication already exists in your carehome's database.")
            return redirect('search_medications', 'A')

        try:
            # Get first letter of medication name for API search
```

```
first_letter = nhs_id[0].upper()
data = get_medicine(first_letter)
```

```
for item in data.get('significantLink', []):
    if nhs_id in item.get('url', ""):
```

```
        medication = Medication.objects.create(
            name=item['name'],
            nhs_id=nhs_id,
            carehome=carehome
        )
```

```
    messages.success(request, f"Successfully added {medication.name} to the database.")
    return redirect('medication_inventory')
```

```
    messages.error(request, "Medication not found in API response.")
except Exception as e:
    messages.error(request, f"Error adding medication: {str(e)}")
```

```
return redirect('search_medications', 'A')
```

#View for deleting a residents medication schedule. Checks users carehome association.

@login_required

```
def delete_medication_schedule(request, schedule_id):
```

```
    if request.method == 'POST':
```

```
        schedule = get_object_or_404(MedicationSchedule, id=schedule_id)
```

```
        carehome = get_user_carehome(request.user)
```

ensure the user has permission to delete this schedule

```
if carehome != schedule.resident.carehome:
```

```
    messages.error(request, "Unauthorized access")
```

```

        return redirect('manager_dashboard')

    resident_id = schedule.resident.id
    schedule.delete()
    messages.success(request, "Medication schedule removed successfully")
    return redirect('resident_medications', resident_id=resident_id)

return redirect('manager_dashboard')

#View for adjusting the sock level of a medciation invetory item.
#Gets the medication item record associated with the request and edits its stock level.

@login_required
@carer_permission_required('can_adjust_stock')
def adjust_inventory_stock(request, inventory_item_id):
    try:
        inventory_item = MedicationInventoryItem.objects.get(id=inventory_item_id)
        new_stock = request.POST.get('stock')

        carehome = get_user_carehome(request.user)
        # Check medication is from user carehome
        if carehome != inventory_item.medication.carehome:
            messages.error(request, "Unauthorized access")
            return redirect('medication_inventory')

        if new_stock is not None:
            inventory_item.stock = int(new_stock)
            inventory_item.save()
            messages.success(request, f"Stock updated for {inventory_item.medication.name}
{inventory_item.dosage} {inventory_item.form}")
        else:
            messages.error(request, "Invalid stock value")
    except (MedicationInventoryItem.DoesNotExist, ValueError):
        messages.error(request, "Invalid update request")
    return redirect('medication_inventory')

#View for adding a new inventory item to the database.

```

```

#Processes the medicationinventoryitem form
#saves the data to the database
#has a check for duplicate records.
@login_required
@carer_permission_required('can_adjust_stock')
def add_inventory_item(request, medication_id):
    medication = get_object_or_404(Medication, id=medication_id)
    carehome = get_user_carehome(request.user)

    if carehome != medication.carehome:
        messages.error(request, "Unauthorized access")
        return redirect('medication_inventory')

    if request.method == 'POST':
        form = MedicationInventoryItemForm(request.POST)
        if form.is_valid():
            inventory_item = form.save(commit=False)
            inventory_item.medicament = medication

            try:
                inventory_item.save()
                messages.success(request, f"Added {inventory_item.dosage} {inventory_item.form} to {medication.name}")
            except IntegrityError:
                messages.error(request, "This dosage and form combination already exists")

        return redirect('medication_inventory')
    else:
        form = MedicationInventoryItemForm()

    return render(request, 'medication/add_inventory_item.html', {
        'form': form,
        'medication': medication
    })

#View for deleting a medication inventory item of a medication.
#Checks if the item is used in any active medication schedules before deleting.
#If the item is used in an active schedule, the user is notified and redirected to the inventory page.

```

```

@login_required
@carer_permission_required('can_adjust_stock')
def delete_inventory_item(request, inventory_item_id):
    try:
        inventory_item = get_object_or_404(MedicationInventoryItem, id=inventory_item_id)
        medication_name = inventory_item.medication.name
        variant_info = f"{inventory_item.dosage} {inventory_item.form}"

        # Verify user has permission to delete this item
        carehome = get_user_carehome(request.user)
        if carehome != inventory_item.medication.carehome:
            messages.error(request, "Unauthorized access")
            return redirect('medication_inventory')

        # Check if inventory item is used in any active medication schedules
        if MedicationSchedule.objects.filter(inventory_item=inventory_item, is_active=True).exists():
            messages.error(request, f"Cannot delete {variant_info} as it is currently used in active medication schedules")
            return redirect('medication_inventory')

        # Delete the inventory item
        inventory_item.delete()
        messages.success(request, f"Successfully removed {medication_name} {variant_info} from inventory")

    except Exception as e:
        messages.error(request, f"Error deleting inventory item: {str(e)}")

    return redirect('medication_inventory')

#View for deleting a medication from the database, similar to delete_inventory_item.
#Checks if the medication is used in any active schedules before deleting.
@login_required
@carer_permission_required('can_adjust_stock')
def delete_medication(request, medication_id):
    if request.method == 'POST':
        medication = get_object_or_404(Medication, id=medication_id)
        carehome = get_user_carehome(request.user)

```

```

# Verify user has permission to delete this item
if carehome != medication.carehome:
    messages.error(request, "Unauthorized access")
    return redirect('medication_inventory')

# Check if any medication inventory items are used in active schedules
active_schedules = MedicationSchedule.objects.filter(
    medication=medication,
    is_active=True
)

if active_schedules.exists():
    messages.error(request,
        f"Cannot delete {medication.name} as it is being used in {active_schedules.count()} active
medication schedules")
    return redirect('medication_inventory')

# Get the medication name before deletion for the success message
medication_name = medication.name

# Delete the medication (this will cascade delete all inventory items)
medication.delete()

messages.success(request, f"Successfully removed {medication_name} and all its variants from
inventory")

return redirect('medication_inventory')

#View for rendering the medication inventory.
@login_required
@carer_permission_required('can_view_inventory')
def medication_inventory(request):

```

```

carehome = get_user_carehome(request.user)
if not carehome:
    messages.error(request, "Unauthorized access")
    return redirect('/')

medications = Medication.objects.filter(carehome=carehome).prefetch_related(
    Prefetch(
        'medicationschedule_set',
        queryset=MedicationSchedule.objects.select_related('resident')
    )
)

return render(request, 'medication/medication_inventory.html', {
    'medications': medications
})

#View for processing the Carer permisisions form and updating the record.
@login_required
def manage_carer_permissions(request, carer_id):
    carehome = get_user_carehome(request.user)
    carer = get_object_or_404(Carer, id=carer_id, carehome=carehome)

    if request.method == 'POST':
        form = CarerPermissionsForm(request.POST, instance=carer.permissions)
        if form.is_valid():
            form.save()
            messages.success(request, f'Permissions updated for {carer.user.username}')
            return redirect('manager_dashboard_carers')
        else:
            form = CarerPermissionsForm(instance=carer.permissions)

    return render(request, 'dashboards/manage_carer_permissions.html', {
        'form': form,
        'carer': carer
    })

#Records the instance of medication being dispensed to a carer

```

```

#Called when carer clicks 'mark as given' on a medication.
#Creates a medication log, including care notes and adjusts the stock level.
@login_required
def record_medication(request, schedule_id, time_slot_id):
    if request.method == 'POST':
        schedule = get_object_or_404(MedicationSchedule, id=schedule_id)
        time_slot = get_object_or_404(MedicationTime, id=time_slot_id)

        # Verify carer has access to this resident
        if not request.user.carer.resident_group or \
            schedule.resident not in request.user.carer.resident_group.residents.all():
            messages.error(request, "Unauthorized access")
            return redirect('carer_dashboard')

        notes = request.POST.get('notes', '')

        # Create medication log
        MedicationLog.objects.create(
            schedule=schedule,
            scheduled_time=time_slot,
            given_by=request.user.carer,
            notes=notes
        )

        # Decrease medication stock
        inventory_item = schedule.inventory_item
        if inventory_item.stock > 0:
            inventory_item.stock -= 1
            inventory_item.save()

        messages.success(request, f"Medication recorded for {schedule.resident.name}")
        return redirect('carer_dashboard')

    return redirect('carer_dashboard')

#Used to dynamically update the Carer dashboard.

```



```

#Used in the Carer dashboard html file, called by javascript in the update dashboard function.
@login_required
def get_dashboard_updates(request):
    carehome = get_user_carehome(request.user)
    upcoming_meds = get_upcoming_medications(carehome)
    html = render_to_string('includes/medication_list.html', {'upcoming_meds': upcoming_meds})
    current_time = timezone.now().strftime('%H:%M:%S')
    return JsonResponse({'html': html, 'current_time': current_time})

#Generates the mar charts for the resident. it uses a in-memeory buffer because the data
#is sensitive and should not be stored on the server.
@login_required
def generate_mar_chart(request, resident_id):
    if not hasattr(request.user, 'manager'):
        messages.error(request, "Unauthorized access")
        return redirect('/')

    carehome = get_user_carehome(request.user)
    resident = get_object_or_404(Resident, id=resident_id)

    # Verify user has permission to access this resident's data
    if carehome != resident.carehome:
        messages.error(request, "Unauthorized access")
        return redirect('manager_dashboard')

    start_date = datetime.now().replace(day=1) # Start from first of current month

    # Create in-memory PDF
    buffer = BytesIO()
    MedicationLog.generate_mar_chart(resident, start_date, buffer)

    # Set buffer's cursor at the beginning
    buffer.seek(0)

    # Return the PDF from memory
    return FileResponse(
        buffer,

```

```

        as_attachment=True,
        filename=f'MAR_{resident.name}_{start_date.strftime("%B_%Y")}.pdf'
    )

#View for adding a new resident to the carehome.
#Uses the same resident form as the edit resident view, but does not populate it with data.
#The form is then saved to the database with the carehome association.
@login_required
def add_resident(request):
    if not hasattr(request.user, 'manager'):
        messages.error(request, "Unauthorized access")
        return redirect('/')

    carehome = get_user_carehome(request.user)
    if not carehome:
        messages.error(request, "Unauthorized access")
        return redirect('/')

    if request.method == 'POST':
        form = ResidentForm(request.POST, request.FILES, carehome=carehome)
        if form.is_valid():
            resident = form.save(commit=False)
            resident.carehome = carehome
            resident.save()
            messages.success(request, 'Resident added successfully.')
            return redirect('manager_dashboard_residents')
        else:
            form = ResidentForm(carehome=carehome)

    return render(request, 'resident/add_resident.html', {'form': form})

#View for the medication audit log.
#This is a read only view for the manager to see all the medication logs for the carehome.
##The logs are filtered by date, and the user can select a date to view the logs for that date.
#Queries the medicationlog table for data.
@login_required
def medication_audit_log(request):
    if not hasattr(request.user, 'manager'):
        messages.error(request, "Unauthorized access")
        return redirect('carer_dashboard')

    carehome = get_user_carehome(request.user)
    selected_date = request.GET.get('date')

```

```

try:
    if selected_date:
        selected_date = datetime.strptime(selected_date, '%Y-%m-%d').date()
    else:
        selected_date = date.today()
except ValueError:
    selected_date = date.today()

logs = MedicationLog.objects.filter(
    schedule__resident__carehome=carehome,
    given_at__date=selected_date
).select_related(
    'schedule__resident',
    'schedule__medication',
    'given_by__user',
    'scheduled_time'
).order_by('-given_at')

return render(request, 'medication/audit_log.html', {
    'logs': logs,
    'selected_date': selected_date,
})

```

Templates/

/Auth/

Businesssignup.html

{% load static %}

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Register - MedTracker</title>

<!-- Bootstrap CSS -->

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH" crossorigin="anonymous">

<style>

body {

background-color: #f5f5f5;

min-height: 100vh;

display: flex;

align-items: center;

}

.registration-container {

max-width: 450px;

margin: auto;

padding: 20px;

}

.card {

border-radius: 15px;

border: none;

box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);

}

```

.nav-tabs {
  border: none;
  margin-bottom: 20px;
}

.nav-link {
  color: #6c757d;
  border: none;
  padding: 10px 20px;
  border-radius: 20px;
  margin-right: 10px;
}

.nav-link.active {
  background-color: #0d6efd;
  color: white;
}

.form-control {
  border-radius: 10px;
  padding: 10px 15px;
  margin-bottom: 15px;
}

.btn-primary {
  border-radius: 10px;
  padding: 10px;
  width: 100%;
}

.logo {
  text-align: center;
  margin-bottom: 30px;
}

.logo img {
  max-width: 200px;
  height: auto;
}

.ms-1 {
  margin-left: (2 * .25) !important;
}
</style>
</head>

<body>
  <div class="registration-container">

```

```

<!-- Logo -->
<div class="logo ms-sm-auto">
  <a href="/"> </a>
</div>

<!-- Error Messages -->
<div class="mt-3">
  {% for message in messages %}
    <div class="alert alert-{{ message.tags|default:'danger' }} alert-dismissible fade show">
      {{ message }}
      <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
  {% endfor %}
</div>

<!-- Registration Card -->
<div class="card">
  <div class="card-body">
    <!-- Tab Navigation -->
    <ul class="nav nav-tabs nav-fill" role="tablist">
      <li class="nav-item" role="presentation">
        <button class="nav-link active" data-bs-toggle="tab" data-bs-target="#manager-tab"
type="button">
          Manager
        </button>
      </li>
      <li class="nav-item" role="presentation">
        <button class="nav-link" data-bs-toggle="tab" data-bs-target="#carer-tab" type="button">
          Carer
        </button>
      </li>
    </ul>

    <!-- Tab Content -->
    <div class="tab-content mt-4">

      <!-- Manager Form -->
      <div class="tab-pane fade show active" id="manager-tab">
        <form method="POST">
          {% csrf_token %}
          <div class="row g-3">
            <div class="col-6">
              <label class="form-label">First Name</label>
              {{ form1.first_name }}
            </div>
          </div>
        </form>
      </div>
    </div>
  </div>
</div>

```

```

</div>
<div class="col-6">
  <label class="form-label">Last Name</label>
  {{ form1.last_name }}
</div>
<div class="col-12">
  <label class="form-label">Email Address</label>
  {{ form1.email }}
</div>
<div class="col-12">
  <label class="form-label">Password</label>
  {{ form1.password1 }}
  <div class="form-text small text-muted">
    Your password must be 8+ characters and contain letters and numbers.
  </div>
</div>
<div class="col-12">
  <label class="form-label">Confirm Password</label>
  {{ form1.password2 }}
</div>
</div>
<button type="submit" name="sign_up_manager" class="btn btn-primary mt-4">
  Register as Manager
</button>
</form>
</div>

<!-- Carer Form -->
<div class="tab-pane fade" id="carer-tab">
  <form method="POST">
    {% csrf_token %}
    <div class="row g-3">
      <div class="col-12">
        <label class="form-label">Care Home Code</label>
        {{ form2.invitation_code }}
        <div class="form-text small text-muted">
          Enter the code provided by your care home manager.
        </div>
      </div>
      <div class="col-6">
        <label class="form-label">First Name</label>
        {{ form2.first_name }}
      </div>
      <div class="col-6">
        <label class="form-label">Last Name</label>
        {{ form2.last_name }}
      </div>
    </div>
  </form>
</div>

```

```

        <label class="form-label">Email Address</label>
        {{ form2.email }}
    </div>
    <div class="col-12">
        <label class="form-label">Password</label>
        {{ form2.password1 }}
        <div class="form-text small text-muted">
            Your password must be 8+ characters and contain letters and numbers.
        </div>
    </div>
    <div class="col-12">
        <label class="form-label">Confirm Password</label>
        {{ form2.password2 }}
    </div>
    </div>
    <button type="submit" name="sign_up_carer" class="btn btn-primary mt-4">
        Register as Carer
    </button>
</form>
</div>
</div>
</div>
</div>

```

```

<!-- Login Link -->
<div class="text-center mt-3">
    <a href="{% url 'login' %}" class="text-decoration-none">Already have an account? Login</a>
</div>
</div>

```

```

<!-- Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

login.html

```
{% load static %}
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<!-- Main stylesheet -->
```

```
<link rel="stylesheet" href="{% static 'style.css' %}">
```



```

<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
crossorigin="anonymous">

<!-- Bootstrap icons -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.min.css">

<script defer src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

<title>Pillpower.com</title>
</head>

<body class="body-login">
  <div class="login-background"></div>

  <!-- Header -->
  <header class="header p-3 position-absolute start-0 top-0 end-0">
    <div class="d-flex justify-content-between align-items-center">
      <a href="/" class="text-decoration-none fw-bold text-white">
        
      </a>
    </div>
  </header>
  <!-- End of header -->

  <div class="container d-flex justify-content-center align-items-center vh-100">
    <div class="card p-4 shadow-lg" style="max-width: 400px; width: 100%;">
      <h1 class="text-center mb-4">Sign In</h1>
      <form method="POST">
        {% csrf_token %}
        <div class="mb-3 text-center ">
          <label for="username" class="form-label">Email</label>
          <input type="text" name="username" id="username" class="form-control text-center"
required>
        </div>
        <div class="mb-3 text-center">
          <label for="password" class="form-label">Password</label>
          <input type="password" name="password" id="password" class="form-control text-center"
required>
        </div>
        <button type="submit" class="btn btn-primary w-100">Sign in</button>
      </form>
    </div>
  </div>

```

```

{% if messages %}
<div class="mt-3">
  {% for message in messages %}
    <div class="alert alert-{{ message.tags|default:'danger' }} alert-dismissible fade show">
      {{ message }}
      <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
    </div>
  {% endfor %}
</div>
{% endif %}
</div>
</body>

</html>

```

/Dashboards/

carer_dashboard.html

```
{% extends 'staticpages/base.html' %}
```

```

{% block content %}
<div class="container-fluid px-3 py-2">
  <!-- Header -->
  <div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Welcome, {{ carer.user.first_name }}</h4>
    <span class="badge bg-primary" id="current-time">{{ current_time|date:"g:i A" }}</span>
  </div>

  <!-- Group Info -->
  {% if carer.resident_group %}
  <div class="card mb-3">
    <div class="card-body">
      <h5 class="card-title">Your Group: {{ carer.resident_group.name }}</h5>
      <p class="card-text">Residents: {{ residents.count }}</p>
    </div>
  </div>
  {% endif %}

  <!-- Dropdown for actions -->
  <div class="dropdown mb-3">

```

```

        <button class="btn btn-secondary dropdown-toggle" type="button" id="dropdownMenuButton"
data-bs-toggle="dropdown" aria-expanded="false">
    Actions
</button>
<ul class="dropdown-menu" aria-labelledby="dropdownMenuButton">
    {% if carer.permissions.can_edit_residents or carer.permissions.can_edit_medications %}
        <li><a class="dropdown-item" href="{% url 'manager_dashboard_residents' %}">View
residents</a></li>
        {% endif %}

```

```

        {% if carer.permissions.can_view_inventory %}
        <li><a class="dropdown-item" href="{% url 'medication_inventory' %}">View
Inventory</a></li>
        {% endif %}
    </ul>
</div>
<div class="button">
    <a href="{% url 'logout' %}" class="btn btn-primary">Logout</a>
</div>

```

```

<!-- Medications List -->
<div class="medication-list" id="medication-list">
    {% include 'includes/medication_list.html' %}
</div>
</div>

```

```

<!-- styles to make it look better on mobile -->
<style>

```

```

.medication-card {
    transition: all 0.3s ease;
}

```

```

.medication-card:active {
    transform: scale(0.98);
}

```

```

.urgency-overdue {
  background-color: #ffe6e6;
}

.urgency-urgent {
  background-color: #fff3cd;
}

.btn {
  min-height: 44px;
  padding: 0.5rem 1rem;
}

.form-control {
  font-size: 16px;
}

@supports (padding: max(0px)) {
  .container-fluid {
    padding-bottom: max(0.5rem, env(safe-area-inset-bottom));
  }
}
</style>
{% endblock %}

{% block extra_js %}
<script>
  document.addEventListener('DOMContentLoaded', function() {
    // Function to update the dashboard
    function updateDashboard() {
      fetch('{% url "get_dashboard_updates" %}')
        .then(response => response.json())
        .then(data => {

```

```

    if (data.error) {
      console.error(data.error);
      return;
    }

    // Update the medication list
    document.getElementById('medication-list').innerHTML = data.html;

    // Update the current time
    document.getElementById('current-time').textContent = data.current_time;
  })
  .catch(error => console.error('Error:', error));
}

// Update every 30 seconds
setInterval(updateDashboard, 30 000);

// Handle form submissions
document.addEventListener('submit', function(e) {
  if (e.target.matches('.medication-card form')) {
    e.preventDefault();

    const form = e.target;
    const formData = new FormData(form);

    fetch(form.action, {
      method: 'POST',
      body: formData,
      headers: {
        'X-CSRFToken': formData.get('csrfmiddlewaretoken')
      }
    })
    .then(response => {
      if (response.ok) {
        // Update the dashboard immediately after recording medication
        updateDashboard();
      }
    });
  }
});

```

```

    }
  })
  .catch(error => console.error('Error:', error));
}
});
});
</script>
{% endblock %}

```

manage_carer_access.html

```
{% extends 'staticpages/base.html' %}
```

```
{% block content %}
```

```

<div class="container mt-4">
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header">
          <h2>Manage Access for {{ carer.user.get_full_name }}</h2>
        </div>
        <div class="card-body">
          <form method="post">
            {% csrf_token %}

            <div class="mb-4">
              <h5>Access Duration</h5>
              <p class="text-muted">Set how long this carer should have access to the system.</p>

              <div class="mt-3">
                {% for radio in form.access_duration %}
                  <div class="form-check mb-2">
                    {{ radio.tag }}
                    <label class="form-check-label" for="{{ radio.id_for_label }}">
                      {{ radio.choice_label }}
                    </label>
                  </div>
                {% endfor %}
              </div>

              {% if form.access_duration.errors %}
                <div class="alert alert-danger mt-2">

```

```

        {{ form.access_duration.errors }}
    </div>
    {% endif %}
</div>

{% if carer.access_end_date and not carer.indefinite_access %}
<div class="alert alert-info">
    Current access expires on: {{ carer.access_end_date|date:"d M Y, H:i" }}
</div>
{% elif carer.indefinite_access %}
<div class="alert alert-success">
    This carer currently has indefinite access.
</div>
{% endif %}

<div class="d-flex justify-content-between">
    <a href="{% url 'manager_dashboard_carers' %}" class="btn btn-secondary">Cancel</a>
    <button type="submit" class="btn btn-primary">Save Changes</button>
</div>
</form>
</div>
</div>
</div>
</div>
</div>
{% endblock %}

```

manage_carer_permissions.html

```
{% extends 'staticpages/base.html' %}
```

```
{% block content %}
```

```
<div class="container mt-4">
```

```
    <h2>Manage Permissions for {{ carer.user.username }}</h2>
```

```
<form method="post" class="mt-4">
```

```
    {% csrf_token %}
```

```
    <div class="card">
```

```
        <div class="card-body">
```

```
            {% for field in form %}
```

```
                <div class="form-check mb-3">
```

```
                    {{ field }}
```

```
                    <label class="form-check-label" for="{{ field.id_for_label }}">
```

```

        {{ field.label }}
    </label>
</div>
{% endfor %}

    <button type="submit" class="btn btn-primary">Save Permissions</button>
    <a href="{% url 'manager_dashboard_carers' %}" class="btn btn-secondary">Cancel</a>
</div>
</div>
</form>
</div>
{% endblock %}

manager_dashboard_carers.html
{% extends 'staticpages/base.html' %}

{% block title %}Carers - PillPower{% endblock %}
{% block content %}
<div class="container-fluid">
    <div class="row">
        <!-- error messages-->
        {% if messages %}
        <div class="messages z-3">
            {% for message in messages %}
            <div class="alert alert-{{ message.tags|default:'danger' }}">
                {{ message }}
            </div>
            {% endfor %}
        </div>
        {% endif %}
        <!-- end of error messages-->
        <!-- Carers List -->
        <div class="col-md-8">
            <h2 class="mb-4">Carers</h2>
            <div class="row">
                {% for carer in carers %}
                <div class="col-md-6 mb-4">
                    <div class="card">
                        <div class="card-body">
                            <h5 class="card-title">{{ carer.user.get_full_name }}</h5>
                            <p class="card-text">
                                <strong>Email:</strong> {{ carer.user.email }}<br>
                                <strong>Group:</strong> {{ carer.resident_group.name|default:"Not Assigned" }}
                            </p>
                            <form method="POST" class="d-inline">
                                {% csrf_token %}

```



```

<input type="hidden" name="carer_id" value="{{ carer.id }}">
<select name="group_id" class="form-select mb-2">
  <option value="">-- Select Group --</option>
  {% for group in groups %}
    <option value="{{ group.id }}" {% if carer.resident_group == group %}selected{%
endif %}>
      {{ group.name }}
    </option>
  {% endfor %}
</select>
<button type="submit" name="update_carer_group" class="btn btn-primary">Update
Group</button>
</form>
<p class="card-text mt-2">
  <strong>Access Status:</strong>
  {% if carer.indefinite_access %}
    <span class="badge bg-success">Permanent Staff</span>
  {% elif carer.access_end_date %}
    {% if carer.has_valid_access %}
      <span class="badge bg-warning">Temporary (Expires: {{ carer.access_end_date|
date:"d M Y" }})</span>
    {% else %}
      <span class="badge bg-danger">Expired</span>
    {% endif %}
  {% else %}
    <span class="badge bg-secondary">Not Set</span>
  {% endif %}
</p>
<div class="mt-2">
  <a href="{% url 'manage_carer_permissions' carer.id %}" class="btn btn-sm btn-info">
    Manage Permissions
  </a>
  <a href="{% url 'manage_carer_access' carer.id %}" class="btn btn-sm btn-warning">
    Manage Access
  </a>
</div>
</div>
</div>
</div>
{% empty %}
<div class="col-12">
  <p>No carers found.</p>
</div>
{% endfor %}
</div>
</div>
<!-- Care Home Code Display -->

```

```

<div class="col-md-4">
  <div class="card mb-4">
    <div class="card-body">
      <h3>Care Home Code</h3>
      <p class="lead">Share codes with carers to join your care home:</p>
      <div class="alert alert-info">
        <a href="{% url 'manage_invitation_codes' %}" class="btn btn-secondary">Manage Staff
Invitations</a>
      </div>
    </div>
  </div>
</div>

<!-- Groups List -->
<div class="card">
  <div class="card-body">
    <h3>Resident Groups</h3>
    <ul class="list-group">
      {% for group in groups %}
      <li class="list-group-item d-flex justify-content-between align-items-center">
        {{ group.name }}
        <span class="badge" style="background-color: {{ group.color }}">
          {{ group.residents.count }} residents
        </span>
      </li>
      {% empty %}
      <li class="list-group-item">No groups created yet.</li>
      {% endfor %}
    </ul>
  </div>
</div>
</div>
</div>
</div>

```

```
{% endblock %}
```

```
manager_dashboard_residents.html
```

```
{% extends 'staticpages/base.html' %}
```

```
{% block title %}Residents - PillPower{% endblock %}
```

```
{% block content %}
```

```
<div class="container-fluid">
```

```
<div class="row">
```

```
<!-- error messages-->
```

```
{% if messages %}
<div class="messages z-3">
  {% for message in messages %}
    <div class="alert alert-{{ message.tags|default:'danger' }}">
      {{ message }}
    </div>
  {% endfor %}
</div>
{% endif %}
```

```
<!-- Header with Add resident Button -->
<div class="col-12 mb-4">
  <div class="d-flex justify-content-between align-items-center">
    <h2>Residents</h2>
    <a href="{% url 'add_resident' %}" class="btn btn-primary">
      <i class="fas fa-plus"></i> Add New Resident
    </a>
  </div>
</div>
```

```
<!-- Residents List -->
<div class="col-md-8">
  <div class="row g-3">
    {% for resident in residents %}
      <div class="col-md-6">
        <div class="card h-100">
          <div class="row g-0">
            {% if resident.photo %}
              <div class="col-3">
                
              </div>
            {% endif %}
            <div class="col{% if resident.photo %}-9{% endif %}">
              <div class="card-body">
                <h5 class="card-title">{{ resident.name }}</h5>
                <div class="card-text">
                  <div class="row g-2">
                    <div class="col-6">
                      <strong>Age:</strong> {{ resident.calculate_age }}
                    </div>
                    <div class="col-6">
                      <strong>Room:</strong> {{ resident.room_number|default:"N/A" }}
                    </div>
                  </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    {% endfor %}
  </div>
```



```

<div class="col-md-4">
  <div class="card">
    <div class="card-body">
      <h3>Add New Group</h3>
      <form method="POST">
        {% csrf_token %}
        <!-- Name field -->
        <div class="mb-3">
          <label for="{{ group_form.name.id_for_label }}" class="form-label">
            {{ group_form.name.label }}
          </label>
          {{ group_form.name }}
          {% if group_form.name.errors %}
          <div class="alert alert-danger">
            {{ group_form.name.errors }}
          </div>
          {% endif %}
        </div>
        <!-- Color field with preview -->
        <div class="mb-3">
          <label for="{{ group_form.colour.id_for_label }}" class="form-label">
            {{ group_form.colour.label }}
          </label>
          <div class="d-flex align-items-center gap-2">
            {{ group_form.colour }}
            <span class="color-preview px-4 py-2 border rounded" style="background-color:
{{ group_form.colour.value|default:'#FF0000' }}">
              Preview
            </span>
          </div>
          {% if group_form.colour.errors %}
          <div class="alert alert-danger">
            {{ group_form.colour.errors }}
          </div>
          {% endif %}
        </div>
        <button type="submit" name="add_group" class="btn btn-primary">Add Group</button>
      </form>
    </div>
  </div>
</div>
{% endblock %}

<!-- JS script for the colour picker -->
{% block extra_js %}
<script>

```

```

document.addEventListener('DOMContentLoaded', function() {
  const colorInput = document.querySelector('input[type="color"]');
  const preview = document.querySelector('.color-preview');

  colorInput.addEventListener('input', function(e) {
    preview.style.backgroundColor = e.target.value;
  });
});
</script>
{% endblock %}

```

manager_dashboard.html

```

{% extends 'staticpages/base.html' %}
<body>
{% block title %}Dashboard - PillPower{% endblock %}
{% block content %}

```

```

<!-- Main content -->
<main class="col-md-9 ms-sm-auto col-lg-10 px-md-4">
  <div class="d-flex justify-content-between flex-wrap flex-md-nowrap align-items-center pt-3 pb-2
mb-3 border-bottom">
    <h1 class="h2">Manager Dashboard</h1>
  </div>

  <!-- Carehome Information -->
  <div class="row mb-4">
    <div class="col-md-6">
      <div class="card">
        <div class="card-body">
          <h5 class="card-title">Carehome Information</h5>
          <p class="card-text">Name: {{ carehome.name }}</p>
          <p class="card-text">Location: {{ carehome.location }}</p>
        </div>
      </div>
    </div>
    <div class="col-md-3">
      <div class="card">
        <div class="card-body">
          <h5 class="card-title">Residents</h5>
          <p class="card-text">Total: {{ residents_count }}</p>
        </div>
      </div>
    </div>
  </div>

```

```

<div class="col-md-3">
  <div class="card">
    <div class="card-body">
      <h5 class="card-title">Carers</h5>
      <p class="card-text">Total: {{ carers_count }}</p>
    </div>
  </div>
</div>

<!-- Today's Medications -->
<div class="row mb-4">
  <div class="col-12">
    <div class="card">
      <div class="card-body">
        <h5 class="card-title">Today's Medications</h5>
        <table class="table">
          <thead>
            <tr>
              <th>Resident</th>
              <th>Medication</th>
              <th>Dosage</th>
              <th>Time</th>
            </tr>
          </thead>
          <tbody>
            {% for schedule in today_medications %}
              {% for time in schedule.times.all %}
                <tr>
                  <td>{{ schedule.resident.name }}</td>
                  <td>{{ schedule.medication.name }}</td>
                  <td>{{ schedule.dosage }}</td>
                  <td>{{ time.time|time:"H:i" }}</td>
                </tr>
              {% endfor %}
            {% empty %}
              <tr>
                <td colspan="4">No medications scheduled for today.</td>
              </tr>
            {% endfor %}
          </tbody>
        </table>
      </div>
    </div>
  </div>
</div>

<!-- Medication Stock + residents -->

```

```

<div class="row">
  <div class="col-md-6 mb-4">
    <div class="card">
      <div class="card-body">
        <h5 class="card-title">Residents</h5>
        <ul class="list-group">
          {% for resident in residents %}
            <li class="list-group-item">{{ resident.name }}</li>
          {% empty %}
            <li class="list-group-item">No residents found.</li>
          {% endfor %}
        </ul>
      </div>
    </div>
  </div>
  <div class="col-md-6 mb-4">
    <div class="card">
      <div class="card-body">
        <h5 class="card-title">Medication Inventory</h5>
        <ul class="list-group">
          {% for medication in medication_stock %}
            <li class="list-group-item">
              <strong>{{ medication.name }}</strong>
              <ul class="list-group mt-2">
                {% for item in medication.inventory_items.all %}
                  <li class="list-group-item">
                    {{ item.dosage }} {{ item.form }} - Stock: {{ item.stock }}
                  </li>
                {% empty %}
                  <li class="list-group-item">No inventory items found.</li>
                {% endfor %}
              </ul>
            </li>
          {% empty %}
            <li class="list-group-item">No medications in inventory.</li>
          {% endfor %}
        </ul>
      </div>
    </div>
  </div>
</div>
</main>

{% endblock %}

</body>
</html>

```


setup_carehome.html

```
{% load widget_tweaks %}
{% block content %}

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}PillPower - Dashboard{% endblock %}</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
crossorigin="anonymous">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.min.css">
  <!-- custom CSS -->
  <link rel="stylesheet" href="/static/style.css">

</head>
<div class="container mt-5">
  <div class="row justify-content-center">
    <div class="col-md-8 col-lg-6">
      <div class="card shadow">
        <div class="card-body p-5">
          <div class="text-center mb-4">
            <h2 class="fw-bold">Set Up Your Care Home</h2>
            <p class="text-muted">Enter your care home details to get started</p>
          </div>

          <form method="post">
            {% csrf_token %}

            <div class="mb-3">
              <label for="{{ form.name.id_for_label }}" class="form-label">Care Home Name</label>
              {{ form.name|add_class:"form-control" }}
              {% if form.name.errors %}
                <div class="text-danger">
                  {{ form.name.errors }}
                </div>
              {% endif %}
            </div>

            <div class="mb-4">
              <label for="{{ form.location.id_for_label }}" class="form-label">Location</label>
```

```

    {{ form.location|add_class:"form-control" }}
    {% if form.location.errors %}
        <div class="text-danger">
            {{ form.location.errors }}
        </div>
    {% endif %}
</div>

{% if messages %}
    {% for message in messages %}
        <div class="alert alert-{{ message.tags }}">
            {{ message }}
        </div>
    {% endfor %}
{% endif %}

<div class="d-grid">
    <button type="submit" class="btn btn-primary btn-lg">
        Create Care Home
    </button>
</div>
</form>
</div>
</div>
</div>
</div>

</div>

{% endblock %}

```

/Includes/

medication_list.html

```

{% for med in upcoming_meds %}
<div class="card mb-2 medication-card {% if med.urgency == 'overdue' %}border-danger{% elif
med.urgency == 'urgent' %}border-warning{% endif %}">
    <div class="card-body">
        <div class="d-flex justify-content-between align-items-start">
            <div>
                <h5 class="card-title mb-1">{{ med.schedule.resident.name }}</h5>
                <h6 class="card-subtitle mb-2 text-muted">
                    {{ med.schedule.medication.name }} - {{ med.schedule.dosage }}
                </h6>
            </div>
            <span class="badge {% if med.urgency == 'overdue' %}bg-danger{% elif med.urgency == 'urgent' %}
bg-warning{% elif med.urgency == 'given' %}bg-success{% else %}bg-primary{% endif %}">
                {{ med.time_slot.time|time:"g:i A" }}
            </span>
        </div>
    </div>
</div>

```

```

</div>

{% if not med.was_given %}
<form method="post" action="{% url 'record_medication' med.schedule.id med.time_slot.id %}"
class="mt-2">
    {% csrf_token %}
    <div class="input-group">
        <input type="text" name="notes" class="form-control form-control-sm" placeholder="Add
notes (optional)">
        <button type="submit" class="btn btn-primary btn-sm">Mark Given</button>
    </div>
</form>
{% else %}
<div class="text-success mt-2">
    <small><i class="fas fa-check"></i> Given at {{ med.given_at|time:"g:i A" }}</small>
</div>
{% endif %}
</div>
</div>
{% empty %}
<div class="alert alert-info">
    No medications scheduled at this time.
</div>
{% endfor %}

```

/Manager/

manage_invites.html

```
{% extends 'staticpages/base.html' %}
```

```

{% block content %}
<div class="container">
    <!-- Error Messages -->
    <div class="mt-3">
        {% for message in messages %}
            <div class="alert alert-{{ message.tags|default:'danger' }} alert-dismissible fade show">
                {{ message }}
                <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
            </div>
        {% endfor %}
    </div>

    <h2>Manage Invitation Codes</h2>

    <p>Generate one-time invitation codes for new carers to join {{ carehome.name }}</p>

```

```

<div class="row">
  <div class="col-md-6">
    <div class="card mb-4">
      <div class="card-header">
        <h5>Generate New Invitation Code</h5>
      </div>
      <div class="card-body">
        <form method="post">
          {% csrf_token %}
          <div class="mb-3">
            {{ form.expiry_days.label_tag }}
            {{ form.expiry_days }}
            {% if form.expiry_days.errors %}
              <div class="invalid-feedback d-block">
                {{ form.expiry_days.errors }}
              </div>
            {% endif %}
          </div>
          <button type="submit" name="generate_code" class="btn btn-primary">Generate
Code</button>
        </form>
      </div>
    </div>
  </div>
</div>

```

```

<div class="col-md-6">
  <div class="card mb-4">
    <div class="card-header">
      <h5>Active Invitation Codes</h5>
    </div>
    <div class="card-body">
      {% if active_invitations %}
        <table class="table table-hover">
          <thead>
            <tr>
              <th>Code</th>
              <th>Created</th>
              <th>Expires</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody>
            {% for invitation in active_invitations %}

```

```

<tr>
  <td><code>{{ invitation.code }}</code></td>
  <td>{{ invitation.created_at|date:"M d, Y" }}</td>
  <td>{{ invitation.expires_at|date:"M d, Y" }}</td>
  <td>
    <form method="post" class="d-inline">
      {% csrf_token %}
      <input type="hidden" name="code_id" value="{{ invitation.id }}">
      <button type="submit" name="delete_code" class="btn btn-sm btn-danger">
        Delete
      </button>
    </form>
  </td>
</tr>
{% endfor %}
</tbody>
</table>
{% else %}
  <p>No active invitation codes.</p>
{% endif %}
</div>
</div>

```

```

<div class="card">
  <div class="card-header">
    <h5>Recently Used Codes</h5>
  </div>
  <div class="card-body">
    {% if used_invitations %}
      <table class="table table-sm">
        <thead>
          <tr>
            <th>Code</th>
            <th>Used By</th>
            <th>Used On</th>
          </tr>
        </thead>
        <tbody>
          {% for invitation in used_invitations %}
            <tr>
              <td><code>{{ invitation.code }}</code></td>
              <td>{{ invitation.used_by.email }}</td>
              <td>{{ invitation.used_at|date:"M d, Y" }}</td>
            </tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
  </div>

```

```

        </table>
    {% else %}
        <p>No used invitation codes.</p>
    {% endif %}
</div>
</div>
</div>
</div>
</div>
{% endblock %}

```

/Medication/

add_inventory_item.html

```
{% extends 'staticpages/base.html' %}
```

```
{% block title %}Add Medication Variant{% endblock %}
```

```
{% block content %}
```

```
<div class="container mt-4">
```

```
<div class="card">
```

```
<div class="card-header">
```

```
<h4>Add Variant for {{ medication.name }}</h4>
```

```
</div>
```

```
<div class="card-body">
```

```
<form method="POST">
```

```
{% csrf_token %}
```

```
<div class="mb-3">
```

```
<label for="{{ form.dosage.id_for_label }}" class="form-label">Dosage</label>
```

```
<div class="input-group">
```

```
{{ form.dosage }}
```

```
<span class="input-group-text">e.g. 500mg</span>
```

```
</div>
```

```
{% if form.dosage.errors %}
```

```
<div class="text-danger">{{ form.dosage.errors }}</div>
```

```
{% endif %}
```

```
</div>
```

```
<div class="mb-3">
```

```
<label for="{{ form.form.id_for_label }}" class="form-label">Form</label>
```

```
{{ form.form }}
```

```
<div class="form-text">E.g. tablet, capsule, liquid, cream</div>
```

```
{% if form.form.errors %}
```

```
<div class="text-danger">{{ form.form.errors }}</div>
```

```
{% endif %}
```

```
</div>
```

```

<div class="mb-3">
  <label for="{{ form.stock.id_for_label }}" class="form-label">Initial Stock</label>
  {{ form.stock }}
  {% if form.stock.errors %}
    <div class="text-danger">{{ form.stock.errors }}</div>
  {% endif %}
</div>

<div class="mb-3">
  <label for="{{ form.reorder_level.id_for_label }}" class="form-label">Reorder Level</label>
  {{ form.reorder_level }}
  <div class="form-text">Stock level at which to reorder</div>
  {% if form.reorder_level.errors %}
    <div class="text-danger">{{ form.reorder_level.errors }}</div>
  {% endif %}
</div>

<div class="d-grid gap-2 d-md-flex justify-content-md-end">
  <a href="{% url 'medication_inventory' %}" class="btn btn-secondary me-md-2">Cancel</a>
  <button type="submit" class="btn btn-primary">Add Variant</button>
</div>
</form>
</div>
</div>
{% endblock %}

```

add_medication_schedule.html

```

{% extends 'staticpages/base.html' %}
{% load widget_tweaks %}

```

```

{% block title %}Add Medication Schedule - PillPower{% endblock %}

```

```

{% block content %}

```

```

<div class="container mt-4">
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header">
          <h2 class="mb-0">Add Medication Schedule for {{ resident.name }}</h2>
        </div>
        <div class="card-body">
          <form method="POST" id="medicationForm">
            {% csrf_token %}

```

```

<!-- Medication Selection -->
<div class="mb-3">
  <label for="{{ form.medication.id_for_label }}" class="form-label">Medication* </label>
  {% render_field form.medication class="form-control" %}
  {% if form.medication.errors %}
    <div class="alert alert-danger">{{ form.medication.errors }}</div>
  {% endif %}
</div>

<!-- Inventory Item Selection -->
<div class="mb-3">
  <label for="{{ form.inventory_item.id_for_label }}" class="form-label">Inventory
Item* </label>
  {% render_field form.inventory_item class="form-control" disabled="true" %}
  {% if form.inventory_item.errors %}
    <div class="alert alert-danger">{{ form.inventory_item.errors }}</div>
  {% endif %}
</div>

<!-- Route -->
<div class="mb-3">
  <label for="{{ form.route.id_for_label }}" class="form-label">Administration
Route* </label>
  {% render_field form.route class="form-control" %}
  {% if form.route.errors %}
    <div class="alert alert-danger">{{ form.route.errors }}</div>
  {% endif %}
</div>

<!-- Frequency -->
<div class="mb-3">
  <label for="{{ form.frequency.id_for_label }}" class="form-label">Times Per Day* </label>
  {% render_field form.frequency class="form-control" min="1" max="6" %}
  {% if form.frequency.errors %}
    <div class="alert alert-danger">{{ form.frequency.errors }}</div>
  {% endif %}
</div>

<!-- Time inputs dynamically added -->
<div id="timeInputs" class="mb-3">
</div>

<div class="d-grid gap-2 d-md-flex justify-content-md-end">

```



```

        <a href="{% url 'resident_medications' resident.id %}" class="btn btn-
secondary">Cancel</a>
        <button type="submit" class="btn btn-primary">Save Schedule</button>
    </div>
</form>
</div>
</div>
</div>
</div>
{ % endblock %}

```

```

{% block extra_js %}
{{ block.super }}

```

```

<script>
document.addEventListener('DOMContentLoaded', function() {
    const frequencyInput = document.getElementById('{{ form.frequency.id_for_label }}');

```

```

function updateTimeInputs() {
    const frequency = parseInt(frequencyInput.value);
    const timeInputs = document.getElementById('timeInputs');
    timeInputs.innerHTML = '<h5 class="mb-3">Administration Times*</h5>';

```

```

    for(let i = 0; i < frequency; i++) {
        const div = document.createElement('div');
        div.className = 'mb-3';
        div.innerHTML = `
            <label class="form-label">Time ${i + 1}</label>
            <input type="time" name="times[]" class="form-control" required>
        `;
        timeInputs.appendChild(div);
    }
}

```

```

frequencyInput.addEventListener('change', updateTimeInputs);

```

```

if (frequencyInput.value) {

```

```

        updateTimeInputs();
    }

const medicationSelect = document.getElementById('{{ form.medications.id_for_label }}');
const inventoryItemSelect = document.getElementById('{{ form.inventory_item.id_for_label }}');

medicationSelect.addEventListener('change', function() {
    const medicationId = this.value;
    if (medicationId) {
        fetch(`/api/medication/${medicationId}/inventory-items/`)
            .then(response => response.json())
            .then(data => {
                // Clear existing options
                inventoryItemSelect.innerHTML = "";

                // Add new options
                data.forEach(item => {
                    const option = document.createElement('option');
                    option.value = item.id;
                    option.textContent = `${item.dosage} ${item.form} (Stock: ${item.stock})`;
                    inventoryItemSelect.appendChild(option);
                });

                //enable selection
                inventoryItemSelect.disabled = false;
            });
    } else {
        // Disable and clear the inventory item select if no medication is selected
        inventoryItemSelect.disabled = true;
        inventoryItemSelect.innerHTML = "";
    }
});
</script>
{% endblock %}

```

audit_log.html

```
{% extends 'staticpages/base.html' %}
```

```
{% block title %}Medication Audit Log - PillPower{% endblock %}
```

```
{% block content %}
```

```

<div class="container mt-4">
  <div class="d-flex justify-content-between align-items-center mb-4">
    <h2>Medication Audit Log</h2>
    <div>
      <form method="get" class="d-flex gap-2">
        <input type="date" name="date" class="form-control" value="{{ selected_date|date:'Y-m-d' }}">
        <button type="submit" class="btn btn-primary">Filter</button>
      </form>
    </div>
  </div>

  <div class="table-responsive">
    <table class="table table-striped">
      <thead>
        <tr>
          <th>Time Given</th>
          <th>Resident</th>
          <th>Medication</th>
          <th>Dosage</th>
          <th>Given By</th>
          <th>Scheduled Time</th>
          <th>Notes</th>
        </tr>
      </thead>
      <tbody>
        {% for log in logs %}
          <tr>
            <td>{{ log.given_at|date:"d/m/Y H:i" }}</td>
            <td>{{ log.schedule.resident.name }}</td>
            <td>{{ log.schedule.medication.name }}</td>
            <td>{{ log.schedule.dosage }}</td>
            <td>{{ log.given_by.user.get_full_name }}</td>
            <td>{{ log.scheduled_time.time|time:"H:i" }}</td>
            <td>{{ log.notes|default:"- " }}</td>
          </tr>
          {% empty %}
            <tr>
              <td colspan="7" class="text-center">No medication logs found for this date.</td>
            </tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
  </div>
{% endblock %}

```

medication_inventory.html

{% extends 'staticpages/base.html' %}

```

{% block content %}
<div class="container-fluid mt-4">
<!-- error messages-->
{% if messages %}
<div class="messages z-3">
  {% for message in messages %}
    <div class="alert alert-{{ message.tags|default:'danger' }}">
      {{ message }}
    </div>
  {% endfor %}
</div>
{% endif %}
<!-- end of error messages-->
<div class="d-flex justify-content-between align-items-center mb-4">
  <h2>Medication Inventory</h2>
  <div class="d-flex gap-2">
    <a href="{% url 'medication_audit_log' %}" class="btn btn-secondary">
      <i class="bi bi-journal-text"></i> Audit Log
    </a>
    <a href="{% url 'search_medications' 'A' %}" class="btn btn-primary">
      <i class="bi bi-plus-circle"></i> Add New Medication
    </a>
  </div>
</div>

{% if medications %}
  <div class="alert alert-info">
    <strong>Note:</strong> To create a new variant, click the “Add Variant” button next to the
medication
  </div>
  <div class="table-responsive">
    <table class="table table-striped">
      <thead>
        <tr>
          <th>Medication</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        {% for medication in medications %}
          <tr>
            <td>{{ medication.name }}</td>
            <td>
              <div class="d-flex gap-2">
                <a href="{% url 'add_inventory_item' medication.id %}" class="btn btn-sm btn-
primary">

```

```

        <i class="fas fa-plus"></i> Add Variant
    </a>
    <a href="https://www.nhs.uk/medicines/{{ medication.nhs_id }}"
        target="_blank"
        class="btn btn-sm btn-outline-info">
        NHS Info
    </a>
    <form method="POST" action="{% url 'delete_medication' medication.id %}"
        onsubmit="return confirm('Are you sure you want to remove
{{ medication.name }}? This will also remove all associated inventory items and cannot be undone.');"
        style="display: inline;">
        {% csrf_token %}
        <button type="submit" class="btn btn-sm btn-outline-danger">
            <i class="fas fa-trash"></i> Remove
        </button>
    </form>
</div>
</td>
</tr>
<!-- Show inventory items underneath each medication -->
{% for item in medication.inventory_items.all %}
<tr class="inventory-item">
    <td class="ps-4">
        {{ item.dosage }} {{ item.form }}
        {% if item.is_low_stock %}
        <span class="badge bg-danger ms-2">Low Stock</span>
        {% endif %}

    </td>

    <td class="d-flex">

        <form method="POST" action="{% url 'adjust_inventory_stock' item.id %}" class="d-
flex">
            {% csrf_token %}
            <input type="number" name="stock" value="{{ item.stock }}"
                class="form-control form-control-sm me-2" style="width: 80px; min="0">
            <button type="submit" class="btn btn-sm btn-outline-primary">
                Update
            </button>

```

```

        </form>
        <a href="#" class="btn btn-sm btn-danger ms-2"
            onclick="confirmDelete('{% url 'delete_inventory_item' item.id %}',
'{{ item.medication.name }} {{ item.dosage }} {{ item.form }}')">
            <i class="fas fa-trash"></i> Delete
        </a>

    </td>
</tr>
{% endfor %}
{% endfor %}
</tbody>
</table>
</div>
{% else %}
    <div class="alert alert-info">
        No medications in inventory.
        <a href="{% url 'search_medications' 'A' %}" class="alert-link">Add some medications</a>
    </div>
{% endif %}
</div>
{% endblock %}
{% block extra_js %}
<script>
function confirmDelete(deleteUrl, itemName) {
    if (confirm(`Are you sure you want to delete ${itemName}?`)) {
        window.location.href = deleteUrl;
    }
}
</script>
{% endblock %}

```

search_medications.html

```
{% extends 'staticpages/base.html' %}
```

```
{% block content %}
```

```
<div class="container mt-4">
```

```

<div class="d-flex justify-content-between align-items-start mb-4">
  <div class="d-flex align-items-center gap-3">
    <a href="{% url 'medication_inventory' %}" class="btn btn-outline-primary">
      <i class="bi bi-arrow-left"></i> Back to Inventory
    </a>
    <h2 class="mb-0">Browse Medications</h2>
  </div>

  <!-- NHS attribution -->
  <a href="https://www.nhs.uk/medicines/" target="_blank">
    
  </a>
</div>

<!-- error messages-->
{% if messages %}
<div class="messages z-3">
  {% for message in messages %}
    <div class="alert alert-{{ message.tags|default:'danger' }}">
      {{ message }}
    </div>
  {% endfor %}
</div>
{% endif %}
<!-- end of error messages-->

<!-- A-Z Navigation -->
<div class="mb-4">
  <div class="d-flex flex-wrap gap-2">
    {% for letter in "ABCDEFGHIJKLMNOPQRSTUVWXYZ" %}
      <a href="{% url 'search_medications' letter %}"
        class="btn {% if letter == search_term %}btn-primary{% else %}btn-outline-primary{% endif
%}">
        {{ letter }}
      </a>
    {% endfor %}
  </div>
</div>

<div id="results">

```

```

{% if results %}
  <div class="row">
    {% for med in results %}
      <div class="col-md-6 mb-3">
        <div class="card">
          <div class="card-body">
            <h5 class="card-title">
              {{ med.name }}
            </h5>
            <form method="POST" action="{% url 'add_medication_to_db' %}">
              {% csrf_token %}
              <input type="hidden" name="nhs_id" value="{{ med.id }}">
              <button type="submit" class="btn btn-success">Add to Database</button>
            </form>
          </div>
        </div>
      </div>
    {% endfor %}
  </div>
{% elif search_term %}
  <p>No medications found starting with "{{ search_term }}"</p>
{% endif %}
</div>
</div>

{% endblock %}

```

/Resident/

add_resident.html

```

{% extends 'staticpages/base.html' %}
{% load widget_tweaks %}

```

```

{% block title %}Add New Resident - PillPower{% endblock %}

```

```

{% block content %}
  <div class="container mt-4">
    <div class="row justify-content-center">
      <div class="col-md-8">
        <div class="card">
          <div class="card-header">
            <h2 class="mb-0">Add New Resident</h2>
          </div>
          <div class="card-body">
            <form method="POST" enctype="multipart/form-data">
              {% csrf_token %}

```



```

<!-- Personal Information -->
<h4 class="mb-3">Personal Information</h4>
<div class="row mb-3">
  <div class="col-md-6">
    <label for="{{ form.name.id_for_label }}" class="form-label">Full Name* </label>
    {% render_field form.name class="form-control" %}
    {% if form.name.errors %}
      <div class="alert alert-danger">{{ form.name.errors }}</div>
    {% endif %}
  </div>
  <div class="col-md-6">
    <label for="{{ form.date_of_birth.id_for_label }}" class="form-label">Date of
Birth* </label>
    {% render_field form.date_of_birth class="form-control" %}
    {% if form.date_of_birth.errors %}
      <div class="alert alert-danger">{{ form.date_of_birth.errors }}</div>
    {% endif %}
  </div>
</div>

<div class="row mb-3">
  <div class="col-md-6">
    <label for="{{ form.room_number.id_for_label }}" class="form-label">Room
Number </label>
    {% render_field form.room_number class="form-control" %}
  </div>
  <div class="col-md-6">
    <label for="{{ form.group.id_for_label }}" class="form-label">Resident Group</label>
    {% render_field form.group class="form-control" %}
  </div>
</div>

<!-- Medical Information -->
<h4 class="mb-3">Medical Information</h4>
<div class="mb-3">
  <label for="{{ form.conditions.id_for_label }}" class="form-label">Medical
Conditions</label>
  {% render_field form.conditions class="form-control" %}
</div>

<div class="mb-3">
  <label for="{{ form.allergies.id_for_label }}" class="form-label">Allergies</label>
  {% render_field form.allergies class="form-control" %}
</div>

```

```

<!-- Photo Upload -->
<h4 class="mb-3">Photo</h4>
<div class="mb-3">
  <label for="{{ form.photo.id_for_label }}" class="form-label">Resident Photo</label>
  {% render_field form.photo class="form-control" %}
</div>

<div class="d-grid gap-2 d-md-flex justify-content-md-end">
  <a href="{% url 'manager_dashboard_residents' %}" class="btn btn-
secondary">Cancel</a>
  <button type="submit" class="btn btn-primary">Add Resident</button>
</div>
</form>
</div>
</div>
</div>
</div>
</div>
{% endblock %}

```

edit_resident.html

```

{% extends 'staticpages/base.html' %}

{% block title %}Edit Resident - PillPower{% endblock %}

{% block content %}
<div class="container mt-4">
  <div class="row">
    <div class="col-md-8 offset-md-2">
      <div class="card">
        <div class="card-body">
          <h2 class="card-title">Edit Resident: {{ resident.name }}</h2>
          <form method="POST" enctype="multipart/form-data">
            {% csrf_token %}
            {% for field in form %}
              <div class="mb-3">
                <label for="{{ field.id_for_label }}" class="form-label">{{ field.label }}</label>
                {{ field }}
                {% if field.errors %}
                  <div class="alert alert-danger">
                    {{ field.errors }}
                  </div>
                {% endif %}
              </div>
            {% endfor %}
            <button type="submit" class="btn btn-primary">Save Changes</button>
            <a href="{% url 'manager_dashboard_residents' %}" class="btn btn-secondary">Cancel</a>
          </form>

```

```

        </div>
    </div>
</div>
</div>
</div>
{% endblock %}

```

resident_medications.html

```
{% extends 'staticpages/base.html' %}
```

```
{% block content %}
```

```

<div class="container-fluid mt-4">
    <div class="row">
        <!-- Medication schedules -->
        <div class="col-md-8">
            <div class="medication-header mb-4">
                <h2>Medications for {{ resident.name }}</h2>
                <div class="medication-actions mt-3">
                    <a href="{% url 'add_medication_schedule' resident.id %}" class="btn btn-primary me-2">
                        <i class="fas fa-plus"></i> Add Medication
                    </a>
                    <a href="{% url 'generate_mar_chart' resident.id %}" class="btn btn-secondary"
target="_blank">
                        <i class="fas fa-file-pdf"></i> Generate MAR Chart
                    </a>
                </div>
            </div>
        </div>
    </div>

```

```

{% if schedules %}
    <div class="row g-4">
        {% for schedule in schedules %}
            <div class="col-md-6">
                <div class="card">
                    <div class="card-body">
                        <div class="d-flex justify-content-between align-items-start mb-3">
                            <h5 class="card-title">{{ schedule.medication.name }}</h5>
                            <form method="POST" action="{% url 'delete_medication_schedule' schedule.id %}"
                                onsubmit="return confirm('Are you sure you want to remove this
medication?');">
                                <button type="submit" class="btn btn-sm btn-outline-danger">
                                    <i class="bi bi-trash"></i>
                                </button>
                            </form>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

```

        <p class="card-text">
            <strong>Dosage:</strong> {{ schedule.dosage }}<br>
            <strong>Frequency:</strong> {{ schedule.frequency }} times per day<br>
            <strong>Times:</strong>
            {% for time in schedule.times.all %}
                {{ time.time|time:"H:i" }}{% if not forloop.last %}, {% endif %}
            {% endfor %}
        </p>
    </div>
</div>
</div>
    {% endfor %}
</div>
    {% else %}
        <p>No medications scheduled for this resident.</p>
    {% endif %}
</div>

</div>
</div>
</div>
{% endblock %}

```

/Staticpages/

base.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}PillPower - Dashboard{% endblock %}</title>
    <!-- Bootstrap CSS -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
crossorigin="anonymous">
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.min.css">
    <!-- custom CSS -->
    <link rel="stylesheet" href="/static/style.css">
    {% block extra_css %}{% endblock %}
    <!-- Montserrat font -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Montserrat:wght@300;400;500;600;700&display=swap" rel="stylesheet">

```

```
<style>
  body {
    font-family: "Montserrat", -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
    font-weight: 400;
    font-size: 16px;
    line-height: 1.5;
  }

  h1, h2, h3, h4, h5, h6 {
    font-family: "proxima-nova", sans-serif;
    font-weight: 600;
  }

  .display-1, .display-2, .display-3, .display-4 {
    font-weight: 300;
  }

  .btn {
    font-weight: 500;
    letter-spacing: 0.3px;
  }

  .navbar-brand {
    font-weight: 600;
    letter-spacing: -0.2px;
  }

  .card-title {
    font-weight: 600;
    letter-spacing: -0.2px;
  }

  .form-control {
    font-size: 0.95rem;
  }

  .table {
    font-size: 0.95rem;
  }

  .nav-link {
    font-weight: 500;
  }
</style>
</head>
```

```

<body>
  <div class="container-fluid">
    <div class="row">
      <!-- Sidebar -->
      <div class="col-md-3 col-lg-2 d-md-block bg-light sidebar collapse-non-md">
        <div class="position-sticky pt-3">

          <ul class="nav flex-column">
            <li class="nav-item">
              <a href="/" class="text-decoration-none fw-bold text-white">
                
              </a>
            </li>

            <li class="nav-item">
              <a class="nav-link {% if request.resolver_match.url_name == 'manager_dashboard' %}
active{% endif %}"
href="{% url 'manager_dashboard' %}">
                Dashboard
              </a>
            </li>
            <li class="nav-item">
              <a class="nav-link {% if request.resolver_match.url_name ==
'manager_dashboard_residents' %}active{% endif %}"
href="{% url 'manager_dashboard_residents' %}">
                Residents
              </a>
            </li>
            {% if user.manager %}
            <li class="nav-item">
              <a class="nav-link {% if request.resolver_match.url_name ==
'manager_dashboard_carers' %}active{% endif %}"
href="{% url 'manager_dashboard_carers' %}">
                Manage Carers
              </a>
            </li>
            {% endif %}
            <li class="nav-item">
              <a class="nav-link {% if request.resolver_match.url_name == 'medication_inventory' %}
active{% endif %}"

```

```

        href="{% url 'medication_inventory' %}">
        Medication
    </a>
</li>

    <li class="nav-item">
        <a class="nav-link" href="{% url 'logout' %}">Logout</a>
    </li>
</ul>
</div>
</div>

```

```

<!-- Main content -->
<main class="col-md-9 ms-sm-auto col-lg-10 px-md-4">
    {% block content %}
    {% endblock %}
</main>
</div>
</div>

```

```

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
    {% block extra_js %}{% endblock %}
</body>
</html>

```

index.html

```

{% load static %}
{% load tz %}
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
        content="width=device-width, user-scalable=no, initial-scale=1.0, maximum-scale=1.0, minimum-
scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>PillPower.com</title>

```

```

<!--main style sheet-->
<link rel="stylesheet" href="{% static 'style.css' %}?{% now 'U' %}">
<!--bootstrap5 css-->

```

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
crossorigin="anonymous">
```

```
<!-- bootstrap icons -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.min.css">
```

```
<script defer src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
```

```
</head>
<body>
<!--header-->
<header class="header p-3 position-absolute start-0 top-0 end-0">
  <div class="d-flex justify-content-between align-items-center">
    <a href="/" class="text-decoration-none fw-bold text-white">
      
    </a>

    <div class="ms-auto">
      <a href="/login/" class="text-white nav-link me-4 navbar-hover-effect">
        Already with us? Login
      </a>
    </div>

    <div>
      <button class="navbar-toggler text-white" type="button" data-bs-toggle="collapse" data-bs-
target="#navbar" aria-controls="navbar" aria-expanded="false" aria-label="toggle navigation">
        <i class="bi bi-three-dots"></i>
      </button>
    </div>

  </div>
</header>
```



```

<!-- end of header-->

<!--navigation bar-->
<nav class="collapse navbar-collapse dropdown-nav" id="navbar">

    <div class="dropdown-nav_container container-xxl d-flex align-items-start align-items-md-center">

        <div class="row align-items-start">

            <!--navbar link1 -->
            <div class="col-2 col-sm-4 mt-4">
                <a href="#" class="row text-decoration-none">
                    <div class="col-2 col-sm-12 mb-4">
                        
                    </div>

                    <div class="col-10">
                        <h3>Track Your Medication</h3>
                        <p>Get your independence back.</p>
                        <p>Learn More <i class="bi bi-arrow-right-short"></i></p>
                    </div>
                </a>
            </div>

            <!--navbar link2 -->
            <div class="col-2 col-sm-4 mb-4 mt-4">
                <a href="/aboutus/" class="row text-decoration-none">
                    <div class="col-2 col-sm-12 mb-4">
                        
                    </div>

                    <div class="col-10">
                        <h3>Who we are</h3>
                        <p>Read about our company and founder.</p>
                        <p>Learn More <i class="bi bi-arrow-right-short"></i></p>
                    </div>
                </a>
            </div>

            <!--navbar link3 -->
            <div class="col-2 col-sm-4 mb-4 mt-4">
                <a href="/register/" class="row text-decoration-none">
                    <div class="col-2 col-sm-12 mb-4">
                        
                    </div>
                </a>
            </div>
        </div>
    </div>

```

```

    </div>

    <div class="col-10">
      <h3>Get Started</h3>
      <p>Login or Register here for free.</p>
      <p>Go now <i class="bi bi-arrow-right-short"></i></p>
    </div>
  </a>
</div>

```

```

</div>

```

```

<!--button to close navbar-->
<button class="navbar-toggler dropdown-nav_CloseNav" type="button" data-bs-toggle="collapse"
data-bs-target="#navbar" aria-controls="navbar" aria-expanded="false" aria-label="toggle navigation">
  <i class="bi bi-x"></i>
</button>

```

```

</div>

```

```

</nav>

```

```

<!--end of navigation bar-->

```

```

<!--hero section-->
<section class="hero">

```

```

  <div class="hero_overlay">

```

```

    <video playsinline="playsinline" autoplay="autoplay" muted="muted" loop="loop" class="hero_video">

```

```

        <source src="/static/img/hero-video-2.mp4" type="video/mp4">
    </video>
</div>

<div class="hero_main h-100 container_custom position-relative">
    <div class="d-flex h-100 align-items-center hero_width_content">
        <div class="text-black">
            <h1 class="hero_heading fw-bold mb-4">Managing medication can be a pain.</h1>
            <p class="lead fw">PillPower makes it simple and stress-free.</p>
            <a href="register/" class="mt-2 btn btn-lg btn-outline-dark " role="button">Get Started and
register with us</a>

```

```

            <a href="login/" class="mt-2 btn btn-lg btn-outline-dark" role="button">Already registered?
Login</a>

```

```

        </div>
    </div>
</div>

```

```

<a href="#Scroll-Down" class="hero_scroll-button">Our Features
    <i class="bi bi-arrow-down-short"></i>
</a>

```

```

</section>

```

```

<!--end of hero section-->

```

```

<!--features section-->

```

```

<a id="Scroll-Down"></a>

```

```

<!-- section one -->

```

```

<section class="features container_custom">

```

```

    <div class="row">

```

```

        <div class="col-sm-6 col-12 d-md-flex">

```

```

```

```

        </div>

```

```

    <div class="align-self-center col-12 justify-content-md-center col-sm-6">

```

```

        <div class="features-content-width">

```

```

            <span>01</span>

```

```

            <h1 class="features_heading mb-4">Track what's what.</h1>

```

```

            <p class="mb-4 features-text">Pill Power has robust handling of all your drug and medication
needs. To start, Simply add your medication to your Pill Power account, and let us handle the rest. <p>

```

```

        <a href="#">Get Started <i class="bi bi-arrow-right"></i></a>

    </div>

</div>

</div>

</section>
<!-- end of section one -->

<
<!-- section two -->
<section class="features features-background">
    <div class="container_custom">
        <div class="row">
            <div class="col-sm-6 col-12 d-md-flex order-sm-1">
                
            </div>

            <div class="align-self-center col-12 justify-content-md-center col-sm-6">
                <div class="features-content-width">
                    <span>02</span>
                    <h1 class="features_heading mb-4">Timing and control</h1>
                    <p class="mb-4 features-text">Make sure that medication is taken at the exact time and for
the correct dosage, all powered by Pill Power.<p>
                    <a href="#">Read More <i class="bi bi-arrow-right"></i></a>

                </div>

            </div>

        </div>

    </div>

</section>

<!-- end of section two -->

<!-- section three -->
<section class="features container_custom">

```

```

<div class="row">
  <div class="col-sm-6 col-12 d-md-flex">
    
  </div>

  <div class="align-self-center col-12 justify-content-md-center col-sm-6">
    <div class="features-content-width">
      <span>03</span>
      <h1 class="features_heading mb-4">Improving your well-being</h1>
      <p class="mb-4 features-text">Pill Power manages and tracks everything about your
medication or prescription. It really makes managing medication on small or large scales far easier. Peace
of mind.<p>
      <a href="#">Read More <i class="bi bi-arrow-right"></i></a>

    </div>

  </div>

</div>

</div>
</section>
<!-- end of section three -->

<!--end of features section-->

<!-- 3 columns section -->
<section class="bg-dark text-white py-4">
  <div class="container_custom my-4">
    <div class="row">

      <div class="col-12 col-sm-4 mb-4">
        
        <h2 class="fw-bold">NHS Industry Standard database</h2>
        <p>Industry Standard medication database, offered by the NHS, up-to-date with the latest
medication information.</p>
      </div>

      <div class="col-12 col-sm-4 mb-4">
        
        <h2 class="fw-bold">Widespread support</h2>
        <p>The design of PillPower.co.uk aims to offer a fantastic experience on both mobile and
desktop</p>
      </div>

      <div class="col-12 col-sm-4 mb-4">

```

```

        
        <h2 class="fw-bold">Visual Feedback</h2>
        <p>Get calendars and graphs representing what to take and when, right from your PillPower
account.</p>
    </div>

</div>
</div>
</section>
<!-- end of 3 columns section -->

<!--footer-->
<footer class="footer">
    <div class="container_custom d-flex justify-content-between align-items-center py-3 border-
highlights">
        <div class="col-md-4 d-flex align-items-center">
            <a href="#" class="me-2 text-muted">
                <i class="bi bi-capsule"></i>
                <span>&copy; 2024 PillPower.com</span>
            </a>
        </div>

        <ul class="nav col-md-4 justify-content-end list-unstyled d-flex text-white">
            <li class="ms-3"><a href="#" class="text-muted"><i class="bi bi-twitter-x"></i></a></li>
            <li class="ms-3"><a href="#" class="text-muted"><i class="bi bi-facebook"></i></a></li>
            <li class="ms-3"><a href="#" class="text-muted"><i class="bi bi-instagram"></i></a></li>

        </ul>
    </div>
</footer>

</body>
</html>

```

Testing

Testing was done with one device hosting the server, and another device accessing it from the same LAN, therefore representing the expected usage of the program as a client-server model.

Evidence Provided by Video, recorded on OBS¹² and edited with Microsoft clipchamp¹³

The video is available and hosted on YouTube:

The video can be found at: <https://youtu.be/F9oF-cDpiBM>

Test	Type	Expected Outcome	Passed?	Evidence
Authentication & Setup				
Attempt signup with valid, unique email and password.	Normal Data	User is created, redirected to Care Home Setup page or dashboard.	Yes	Video 0:03
Attempt signup with an already existing email.	Erroneous Data	Error message shown, user not created.	Yes	Video 0:30
Attempt signup with invalid email format.	Erroneous Data	Form validation error shown, user not created.	Yes	Video 0:46
Log in with correct credentials after signup.	Normal Data	Successful login, redirected to the appropriate dashboard (Setup or Main Manager Dashboard).	Yes	Video 1:11
Attempt login with incorrect password.	Erroneous Data	Error message shown, login fails.	Yes	Video 0:52
Attempt login with non-existent email.	Erroneous Data	Error message shown, login fails.	Yes	Video 1:03
Manager: Complete Care Home Setup form after first login.	Normal Data	Care home details are saved, manager redirected to the main dashboard.	Yes	Video 0:17
Manager: Generate a new invitation code.	Normal Data	A unique invitation code is generated and displayed in the 'Manage Invites' section.	Yes	Video 1:24
Carer: Attempt signup without a valid invitation code.	Erroneous Data	Error message shown, user not created.	Yes	Video 2:20
Carer: Attempt signup with an expired invitation code.	Boundary Data	Error message shown, user not created.	Yes	Video 2:10
Carer: Attempt signup with a valid, unexpired invitation code.	Normal Data	User is created, redirected to login or carer dashboard.	Yes	Video 1:52
Carer: Log in with correct credentials.	Normal Data	Successful login, redirected to Carer Dashboard.	Yes	Video 7:29

¹² Open Broadcast Software - <https://obsproject.com/>

¹³ Microsoft Clipchamp for Windows - <https://clipchamp.com/en/windows-video-editor/>

Carer: Attempt to Log In when access is expired.	Acceptance Testing	Error message shown, User not logged in.	Yes	Video 7:17
Carer: Attempt to Log In when within access period.	Normal Data	Successful login, redirected to the appropriate dashboard.	Yes	Video 7:29
Any User: Click Logout button.	Acceptance Testing	User is logged out and redirected to the landing page or login page.	Yes	Video 0:26
Manager Functions				
Manager: Navigate to 'Add Resident' page.	Acceptance Testing	Add Resident' form is displayed correctly.	Yes	Video 2:32
Manager: Add a new resident with all valid required details.	Normal Data	Resident is saved, success message shown, resident appears in the 'Residents' list.	Yes	Video 2:39
Manager: Attempt to add a resident with missing required fields.	Erroneous Data	Form validation error shown, resident not saved.	Yes	Video 3:41
Manager: Edit an existing resident's details.	Normal Data	Details are updated, success message shown, updated details reflected in the list.	Yes	Video 3:43
Manager: Create a Resident Group	Normal Data	Resident group with name and colour created	Yes	Video 6:06
Manager: Assign Carer to Resident Group	Normal Data	Carer profile is updated, Updated group status reflected in profile	Yes	Video 6:16
Manager: Assign Resident to Resident Group	Normal Data	Resident profile is updated, Updated group status reflected in profile	Yes	Video 6:19
Manager: View the list of Carers.	Acceptance Testing	List of registered carers associated with the care home is displayed.	Yes	Video 4:08
Manager: Grant specific permissions to a Carer.	Normal Data	Permissions are updated for the carer. (Verify by logging in as carer or checking carer's view).	Yes	Video 4:44
Manager: Revoke specific permissions from a Carer.	Normal Data	Permissions are updated for the carer. (Verify by logging in as carer).	Yes	Video 4:39
Manager: Set an access expiry date for a Carer.	Normal Data	Access end date is saved for the carer.	Yes	Video 4:13
Manager: Set indefinite access for a Carer.	Normal Data	Indefinite access flag is set for the carer.	Yes	Video 4:10
Manager: Navigate to 'Medication Inventory'.	Acceptance Testing	List of medication inventory items is displayed correctly.	Yes	Video 4:53
Manager: Add a new item to the Medication Inventory.	Normal Data	Item is added successfully, appears in the inventory list.	Yes	Video 5:27
Manager: Attempt to add an inventory item with missing fields.	Erroneous Data	Form validation error shown, item not added.	Yes	Video 5:11

Manager: View the Medication Audit Log.	Acceptance Testing	Log entries (e.g., administrations) are displayed correctly.	Yes	Video 7:42
Manager: Search for a specific medication.	Normal Data	Search results page displayed, relevant medications listed (if found).	Yes	Video 4:59
Manager: Add a medication schedule for a resident.	Normal Data	Schedule is created successfully, associated with the resident.	Yes	Video 5:39
Manager: Generate a MAR chart for a resident/period.	Acceptance Testing	A PDF MAR chart is generated and downloaded/displayed.	Yes	Video 8:00
Carer Functions				
Carer: Log in and view assigned residents on the dashboard.	Acceptance Testing	Only residents assigned to this carer (or all if permissions allow) are visible.	Yes	Video 6:25
Carer: View upcoming medication schedule for a resident.	Acceptance Testing	Correct medication times and details are displayed for the selected resident.	Yes	Video 6:35
Carer: Record administration of a scheduled medication.	Normal Data	Administration is logged successfully, timestamped, appears in audit log.	Yes	Video 6:40
Carer: Attempt to access a manager-only function.	Acceptance Testing	Access denied message shown, or page redirects/doesn't load.	Yes	Video 6:55

Evaluation

Fulfilment of Objectives:

Objective	Met?
1. User Account creation	
1.1. Provide a user registration module to create distinct user accounts	Yes
1.1.1. Present a manager registration form, allowing the creation with name, email and password	Yes
1.1.2. Present a Carer registration form, allowing account creation with name, email password and a existing carehome code	Yes
1.1.3. Process, clean and store user data in the database	Yes
1.1.3.1. Store User passwords in an encrypted format	Yes
1.1.4. Create a Manager/Carer entity from stored Users in the database	Yes
1.1.4.1. Present a carehome creation form process for Managers after registration	Yes
1.1.4.1.1. Require managers to setup carehome after registration	Yes
2. Role-Based access controls	
2.1. Implement role-based access to views, which are dependant on user permissions or role	Yes
2.1.1. Restrict carehome views to authenticated users associated with the correct carehome	Yes
2.1.2. implement permission-based access for Carers	Yes
2.1.2.1. limit the views and functions they can access based on permissions.	Yes
3. User session management	
3.1. Enable users to Login using an authentication form	Yes
3.1.1. Require both an email and password for logging in	Yes
3.2. Redirect authenticated users to the appropriate dashboard for their role	Yes
3.3. Redirect unauthorised access back to the login page	Yes
3.3.1. Display error messages displaying appropriate contextual message	Yes
4. Resident Management	
4.1. Enable Managers to add residents to the carehome	Yes
4.1.1. Display a form, requiring valid fields like name and d.o.b	Yes
4.1.2. Create a record of the resident in the database	Yes

4.2. Enable creation of carehome groups	Yes
4.2.1. Display a name and colour field	Yes
4.2.2. Allow residents to be assigned to groups associated with their carehome	Yes
5. Medication Schedule management	
5.1. Manage Medication Schedules and the administration	Yes
5.1.1. Provide a dashboard for Carers to view upcoming medication schedules	Yes
5.1.1.1. Only display the residents for residents of an associated group	Yes
5.1.2. Allow authorised carers to mark a medication as administered	Yes
5.1.2.1. Log the time of medication administration	Yes
5.1.2.2. Provide a optional field for care notes	Yes
6. Medication Inventory Tracking	Yes
6.1. Track the carehome inventory	Yes
6.1.1. Allow managers to add medication to the carehome inventory	Yes
6.1.1.1. Provide a form to define a variant of a medication	Yes
6.1.1.2. Store the variant of the medication in the database	Yes
6.1.1.3. Notify users when medication is low on stock/ requires reorder	Yes
7. Carer Permissions	
7.1. Implement controllable permissions for carer access	Yes
7.1.1. Allow managers to assign permissions for an individual carer	Yes
7.1.2. Allow managers to revoke permissions for an individual carer	Yes
7.1.3. Create a custom decorator to control permissions	Yes
7.1.3.1. Allow decorator to refuse/allow access dependant on different permissions	Yes
7.1.4. Trigger error message/redirect for unauthorised access	Yes
8. Medical Administration logging	
8.1. Implement a audit trail/log of medicine administration	Yes
8.1.1. Log every instance of given medication	Yes
8.1.1.1. Record timestamp at moment of administration	Yes
8.1.1.2. Record which carers staff provided medication	Yes
8.1.1.3. Record optional notes of medical administration	Yes
8.1.2. Allow audit logs to be filtered by date/time	Yes

9. Carer Dashboard	
9.1. Provide carer dashboard with dynamic updates	Yes
9.1.1. Automatically refresh the list of upcoming medication	Yes
9.2. Calculate urgency of pending medications	Yes
9.2.1. Display urgent medications with colour coded identifier	Yes

User feedback:

After my own testing, I had my end-user have hands on experience with the final product. After which I asked a series of questions to gather feedback:

Question:	Answer:
Was the program easy to use and navigate?	Overall, the user interface was straightforward, information was provided
Did you encounter any difficulties while using the program? If so, please describe them.	No, there was no difficulty in navigating the system.
Did the program meet your expectations in terms of functionality?	Yes, my expectations were met. I had the ability to add the usual medication our carehome uses, view them and update their stock. Also, I could manage carers and residents, setting permissions and access controls. Carers could provide notes with their dispensation of medication, and audit logs reflect that.
Did you experience any bugs or issues while using the program?	Small tweaks required, MAR chart didn't display DOB.in rare instances
Did the program perform efficiently and quickly?	Yes, there was no noticeable loadtimes while using the program.
Did you encounter any security concerns or vulnerabilities?	No, from what I saw, there were staff access controls and permissions which is a must. Initial registration required one time code from managers. I noticed that while logged in as a carer, I was unable to access the pages I would expect only a manager to have access to like the staff management page.
Are there any features you think are missing or could be improved?	Should be able to mark item as refused or not required, with reason, instead of only allowing for it to be marked as 'given'. (although the care notes allow for refusal to

	be noted in some way) In-Built training system, which would allow us to mark staff as competent with the software.
How satisfied are you with the program overall?	It has very good list of features, meets my expectations for security. Scheduling is straightforward and inventory management is versatile.

Overall evaluation:

Overall, the project successfully delivers a care home management features with the core functionality expected of the end-user and the initial objectives. End-User testing confirmed that it met expectations, highlighting the straightforward design and ease of navigation.

The platform has support for concurrent carehome use, through the use of a robust PostgreSQL database and model-view-controller client-server. User management is a key strength, and there is distinct user access between Manager and Carer roles, as well as secure invitation links (through a one-time code noted as a positive by the end-user) and effective permission control. In feedback, the User mentioned the controls successfully restricted access to sensitive areas like the staff management dashboard. Furthermore, considerations like password encryption, pseudonymisation, audit trails and time-based user access enhances the security in relation to GDPR sensitive data and accountability for access of the data.

Core functions like managing and recording resident information (adding, viewing, and editing) and medication inventory (variants and stock levels) were well received. Medication scheduling and logging administration, with added notes, was a feature the user found practical, however, raised a valuable improvement that medication could also be marked as 'refused' or 'not required' within reason, which currently the program doesn't consider.

While the program sufficiently met the needs of my end-user and left them satisfied, I do believe in future improvements can be made. One consideration is the scheduling system, while daily scheduling is functional, enhancing it with modularity (e.g. a weekly or pattern-based scheduling system for medicine on specific days of the week) would improve the flexibility for more carehomes and resident needs. Additionally, further security measures could also be implemented, be that two-factor authentication or database encryption for medical or residential data. Furthermore, A last addition would be asynchronous sockets for the carer dashboard. The current implementation is functional and the dashboard does update itself without need for user input, but it's not truly real time.

In conclusion, the system provides a solid foundation that meets the defined objectives and was successful in testing, in addition to this it earned positive feedback regarding usability. The improvements offer a clear pathway for expansion of the project in the future.